

```
{"metadata":{"kernelspec":{"display_name":"Python
3","language":"python","name":"python3"},"language_info":
{"codemirror_mode":
{"name":"ipython","version":3},"file_extension":".py","mimetype":"te
xt/x-
python","name":"python","nbconvert_exporter":"python","pygments_lexer":
"ipython3","version":"3.11.13"},"kaggle":
{"accelerator":"none","dataSources":
[{"sourceId":12826027,"sourceType":"datasetVersion","datasetId":8111
178},
{"sourceId":12843465,"sourceType":"datasetVersion","datasetId":81231
04},
{"sourceId":12847832,"sourceType":"datasetVersion","datasetId":81260
92},
{"sourceId":12855367,"sourceType":"datasetVersion","datasetId":81301
02},
{"sourceId":12855779,"sourceType":"datasetVersion","datasetId":81313
62},
{"sourceId":12855788,"sourceType":"datasetVersion","datasetId":81313
69}], "dockerImageVersionId":31089,"isInternetEnabled":true,"language
":"python","sourceType":"notebook","isGpuEnabled":false},"papermill"
:{"default_parameters":
{},"duration":12.16616,"end_time":"2025-08-24T12:38:57.809985","envi
ronment_variables":
{},"exception":null,"input_path":"__notebook__.ipynb","output_path":
"__notebook__.ipynb","parameters":
{},"start_time":"2025-08-24T12:38:45.643825","version":"2.6.0"},"wid
gets":{"application/vnd.jupyter.widget-state+json":{"state":
{"00fcfdb360f74a4d862dc013f56a64a6":{"model_module":"@jupyter-
widgets/
controls","model_module_version":"2.0.0","model_name":"CheckboxStyle
Model","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"CheckboxSty
leModel","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","backg
round":null,"description_width":""}},{"044f1791c73c4966ba64441ed51d2f
ba":{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"ButtonModel",
"state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"ButtonModel
","_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"ButtonView","
button_style":"","description":"Estimate
ETAs","disabled":false,"icon":"","layout":{"IPY_MODEL_e3d3f8a44f4641c
eabcba4f4a8c80263d","style":{"IPY_MODEL_578e91cb8b9d46138d2bab7db1a6f
62"},"tabbable":null,"tooltip":null}},"062c76fe6dba4049938d15ec5371e7
99":{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"TextareaModel
","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"TextareaMod
el","_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"TextareaView"
,"continuous_update":true,"description":"Vitals
JSON","description_allow_html":false,"disabled":false,"layout":{"IPY_
```

```
MODEL_23ec39404c004d46ad725e754c5bd68e","placeholder":  
","rows":null,"style":"IPY_MODEL_ae89f3ca57724884b0b62141526a2a71",  
"tabbable":null,"tooltip":null,"value":{"\"rr\":24,\"sbp\":95,\"gcs\":  
:14,\"hr\":120,\"temp\":38.6,\"avpu\":\"V\",  
\"map\":65,\"age\":68,\"sex\":\"M\",  
\"sao2\":97}}},"0645fad0130d4e28b163318b2657ad3a":  
{"model_module":"@jupyter-widgets/  
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta  
te":{"_model_module":"@jupyter-widgets/  
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_  
view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig  
n_content":null,"align_items":null,"align_self":null,"border_bottom"  
:null,"border_left":null,"border_right":null,"border_top":null,"bott  
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul  
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu  
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template  
_areas":null,"grid_template_columns":null,"grid_template_rows":null,  
"height":null,"justify_content":null,"justify_items":null,"left":nul  
l,"margin":null,"max_height":null,"max_width":null,"min_height":null  
,"min_width":null,"object_fit":null,"object_position":null,"order":n  
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili  
ty":null,"width":null}},"066b9d79860547ffabc26a71b6ea31ae":  
{"model_module":"@jupyter-widgets/  
controls","model_module_version":"2.0.0","model_name":"CheckboxModel  
","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/  
controls","_model_module_version":"2.0.0","_model_name":"CheckboxMod  
el","_view_count":null,"_view_module":"@jupyter-widgets/  
controls","_view_module_version":"2.0.0","_view_name":"CheckboxView"  
,"description":"Paresis / plaster  
cast","description_allow_html":false,"disabled":false,"indent":true,  
"layout":"IPY_MODEL_d9e4af503e0547678a9d5d3bb9279d91","style":"IPY_M  
ODEL_3fdb829ca8c84cc280c94170ce21566c","tabbable":null,"tooltip":nul  
l,"value":false}},"07a6726c673342bc99b03d630d7553be":  
{"model_module":"@jupyter-widgets/  
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta  
te":{"_model_module":"@jupyter-widgets/  
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_  
view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig  
n_content":null,"align_items":null,"align_self":null,"border_bottom"  
:null,"border_left":null,"border_right":null,"border_top":null,"bott  
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul  
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu  
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template  
_areas":null,"grid_template_columns":null,"grid_template_rows":null,  
"height":null,"justify_content":null,"justify_items":null,"left":nul  
l,"margin":null,"max_height":null,"max_width":null,"min_height":null  
,"min_width":null,"object_fit":null,"object_position":null,"order":n  
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili  
ty":null,"width":null}},"0817e2629d6148e4b11015c46e3edca5":  
{"model_module":"@jupyter-widgets/  
controls","model_module_version":"2.0.0","model_name":"HTMLModel","s  
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
```

```
controls", "_model_module_version": "2.0.0", "_model_name": "HTMLModel",
"_view_count": null, "_view_module": "@jupyter-widgets/
controls", "_view_module_version": "2.0.0", "_view_name": "HTMLView", "de
scription": "", "description_allow_html": false, "layout": "IPY_MODEL_b55
9e8ffc7b94ed3b17a1d140aa81b03", "placeholder": "
", "style": "IPY_MODEL_4f44cb5207e9482c911c674771b158d8", "tabbable": nu
ll, "tooltip": null, "value": "<b>Pregnancy-adapted YEARS pathway</
b>{}", "0927fa383af84923949a0beae319f902": {"model_module": "@jupyter-
widgets/
controls", "model_module_version": "2.0.0", "model_name": "TextStyleMode
l", "state": {"_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "TextStyleMo
del", "_view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "backg
round": null, "description_width": "", "font_size": null, "text_color": nul
l}}, "0afb9ed37ce84de383952257972f01ee": {"model_module": "@jupyter-
widgets/
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta
te": {"_model_module": "@jupyter-widgets/
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_
view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig
n_content": null, "align_items": null, "align_self": null, "border_bottom"
: null, "border_left": null, "border_right": null, "border_top": null, "bott
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nu
ll, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template
_areas": null, "grid_template_columns": null, "grid_template_rows": null,
"height": null, "justify_content": null, "justify_items": null, "left": nul
l, "margin": null, "max_height": null, "max_width": null, "min_height": null
, "min_width": null, "object_fit": null, "object_position": null, "order": n
ull, "overflow": null, "padding": null, "right": null, "top": null, "visibili
ty": null, "width": null}}, "0cf762a039b14ec498fa9c60a33c32fd":
{"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "HTMLStyleMode
l", "state": {"_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "HTMLStyleMo
del", "_view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "backg
round": null, "description_width": "", "font_size": null, "text_color": nul
l}}, "0eb05c5d88c74d08a2295e0f1766a308": {"model_module": "@jupyter-
widgets/
controls", "model_module_version": "2.0.0", "model_name": "TextStyleMode
l", "state": {"_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "TextStyleMo
del", "_view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "backg
round": null, "description_width": "", "font_size": null, "text_color": nul
l}}, "0eb31f30adb44afbb7561751873caa19": {"model_module": "@jupyter-
widgets/
controls", "model_module_version": "2.0.0", "model_name": "ButtonStyleMo
del", "state": {"_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "ButtonStyle
Model", "_view_count": null, "_view_module": "@jupyter-widgets/
```

```
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "button_color": null, "font_family": null, "font_size": null, "font_style": null, "font_variant": null, "font_weight": null, "text_color": null, "text_decoration": null}}, "10235bb45f7d41c8815c4b632ab563b9": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, "112565e2d4f34fddb795da4aad032c53": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "122605c8e19a4d1497f3eb41213bb18b": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "IntTextModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "IntTextModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "IntTextView", "continuous_update": false, "description": "Capacity", "description_allow_html": false, "disabled": false, "layout": "IPY_MODEL_c49d5de4436b40a0be9a25f09789e537", "step": 1, "style": "IPY_MODEL_194c8725ccf948eabd7921d8cd975fd4", "tabbable": null, "tooltip": null, "value": 12}}, "143ed71461854d6594f0b56ff94a0308": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null}}
```

```
ty":null,"width":null}}, "14e0e99c3d2f4bea9499f712d68f2d8c":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"CheckboxStyle
Model","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"CheckboxSty
leModel","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","backg
round":null,"description_width":""}}, "151894b90397400db1ddcedcf7b7cc
69":{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"TextModel","s
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"TextModel",
"_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"TextView","co
ntinuous_update":true,"description":"Patient
ID","description_allow_html":false,"disabled":false,"layout":"IPY_MO
DEL_1fa58b7d0dd342abb9efb0761021dd46","placeholder":"e.g.,
UKE-12345","style":"IPY_MODEL_6aac149e583a4e9d8153867d7a8b4033","tab
bable":null,"tooltip":null,"value":""}}, "1653cdfec14c486b8754d52399e
5d47e":{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"CheckboxModel
","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"CheckboxMod
el","_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"CheckboxView"
,"description":"Previous
DVT","description_allow_html":false,"disabled":false,"indent":true,"
layout":"IPY_MODEL_1db642809530431cb7b3cda50229c6f9","style":"IPY_MO
DEL_cf8f143ba3fb42bfba01506abc0e0f18","tabbable":null,"tooltip":null
,"value":false}}, "1694569d067f49428b2497d442d3dabe":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"ButtonModel",
"state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"ButtonModel
","_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"ButtonView",
"button_style":"","description":"Compute +
Log","disabled":false,"icon":"","layout":"IPY_MODEL_d5b3ab9b3f904b9e
88dbc2038f8a049c","style":"IPY_MODEL_77025d605f9e4f4185e5bac86452741
1","tabbable":null,"tooltip":null}}, "17e0c42dbcaa4e5d92699b9c81085ff
f":{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"HBoxModel","s
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"HBoxModel",
"_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"HBoxView", "bo
x_style":"","children":
["IPY_MODEL_1694569d067f49428b2497d442d3dabe","IPY_MODEL_20371b46207
24785baafef66a834eaaa"],"layout":"IPY_MODEL_3dab4615c04644e39dd753e8
93fb787f","tabbable":null,"tooltip":null}}, "194c8725ccf948eabd7921d8
cd975fd4":{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"DescriptionSt
yleModel","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"Description
```

```
StyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "descri  
ption_width": ""}}, "1964c0c2a3b44f54b62cc0060962f132":  
{ "model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta  
te": { "model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "_  
view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig  
n_content": null, "align_items": null, "align_self": null, "border_bottom"  
: null, "border_left": null, "border_right": null, "border_top": null, "bott  
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul  
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nul  
l, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_  
areas": null, "grid_template_columns": null, "grid_template_rows": null,  
"height": null, "justify_content": null, "justify_items": null, "left": nul  
l, "margin": null, "max_height": null, "max_width": null, "min_height": null  
, "min_width": null, "object_fit": null, "object_position": null, "order": n  
ull, "overflow": null, "padding": null, "right": null, "top": null, "visibili  
ty": null, "width": null}}, "1abc26d9099242e0aee3a00097a8cadf":  
{ "model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta  
te": { "model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "_  
view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig  
n_content": null, "align_items": null, "align_self": null, "border_bottom"  
: null, "border_left": null, "border_right": null, "border_top": null, "bott  
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul  
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nul  
l, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_  
areas": null, "grid_template_columns": null, "grid_template_rows": null,  
"height": "140px", "justify_content": null, "justify_items": null, "left":  
null, "margin": null, "max_height": null, "max_width": null, "min_height": n  
ull, "min_width": null, "object_fit": null, "object_position": null, "order  
": null, "overflow": null, "padding": null, "right": null, "top": null, "visib  
ility": null, "width": "100%"}}, "1be04ed5691e4069910c21ccf482f1ed":  
{ "model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta  
te": { "model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "_  
view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig  
n_content": null, "align_items": null, "align_self": null, "border_bottom"  
: null, "border_left": null, "border_right": null, "border_top": null, "bott  
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul  
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nul  
l, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_  
areas": null, "grid_template_columns": null, "grid_template_rows": null,  
"height": null, "justify_content": null, "justify_items": null, "left": nul  
l, "margin": null, "max_height": null, "max_width": null, "min_height": null  
, "min_width": null, "object_fit": null, "object_position": null, "order": n  
ull, "overflow": null, "padding": null, "right": null, "top": null, "visibili  
ty": null, "width": null}}, "1db642809530431cb7b3cda50229c6f9":
```

```
{"model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta  
te": {"_model_module": "@jupyter-widgets/  
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_  
view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig  
n_content": null, "align_items": null, "align_self": null, "border_bottom"  
: null, "border_left": null, "border_right": null, "border_top": null, "bott  
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul  
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nu  
ll, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template  
_areas": null, "grid_template_columns": null, "grid_template_rows": null,  
"height": null, "justify_content": null, "justify_items": null, "left": nul  
l, "margin": null, "max_height": null, "max_width": null, "min_height": null  
,"min_width": null, "object_fit": null, "object_position": null, "order": n  
ull, "overflow": null, "padding": null, "right": null, "top": null, "visibili  
ty": null, "width": null}}, "1ef699aa4db84ff8afee13265b131a0e":  
{"model_module": "@jupyter-widgets/  
output", "model_module_version": "1.0.0", "model_name": "OutputModel", "s  
tate": {"_dom_classes": [], "_model_module": "@jupyter-widgets/  
output", "_model_module_version": "1.0.0", "_model_name": "OutputModel",  
"_view_count": null, "_view_module": "@jupyter-widgets/  
output", "_view_module_version": "1.0.0", "_view_name": "OutputView", "la  
yout": "IPY_MODEL_07a6726c673342bc99b03d630d7553be", "msg_id": "", "outp  
uts":  
[], "tabbable": null, "tooltip": null}}, "1fa58b7d0dd342abb9efb0761021dd4  
6": {"model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta  
te": {"_model_module": "@jupyter-widgets/  
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_  
view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig  
n_content": null, "align_items": null, "align_self": null, "border_bottom"  
: null, "border_left": null, "border_right": null, "border_top": null, "bott  
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul  
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nu  
ll, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template  
_areas": null, "grid_template_columns": null, "grid_template_rows": null,  
"height": null, "justify_content": null, "justify_items": null, "left": nul  
l, "margin": null, "max_height": null, "max_width": null, "min_height": null  
,"min_width": null, "object_fit": null, "object_position": null, "order": n  
ull, "overflow": null, "padding": null, "right": null, "top": null, "visibili  
ty": null, "width": null}}, "20371b4620724785baafef66a834eaaa":  
{"model_module": "@jupyter-widgets/  
controls", "model_module_version": "2.0.0", "model_name": "ButtonModel",  
"state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "ButtonModel",  
"_view_count": null, "_view_module": "@jupyter-widgets/  
controls", "_view_module_version": "2.0.0", "_view_name": "ButtonView", "  
button_style": "", "description": "Apply HL7 → Labs  
JSON", "disabled": false, "icon": "", "layout": "IPY_MODEL_f18a06d7c1c54a0  
794737cebc44b5adf", "style": "IPY_MODEL_438acb80307b492d82d9f7f5eac1bf  
f9", "tabbable": null, "tooltip": null}}, "2213bebf52324a8faea0908b288897  
0e": {"model_module": "@jupyter-widgets/
```

```
controls","model_module_version":"2.0.0","model_name":"DescriptionSt
yleModel","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"Description
StyleModel","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","descr
iption_width":""}},"229861ac14594327887a3ecc1ed6ee93":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"HTMLStyleMode
l","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"HTMLStyleMo
del","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","backg
round":null,"description_width":"","font_size":null,"text_color":nul
l}},"23ec39404c004d46ad725e754c5bd68e":{"model_module":"@jupyter-
widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":"90px","justify_content":null,"justify_items":null,"left":n
ull,"margin":null,"max_height":null,"max_width":null,"min_height":nu
ll,"min_width":null,"object_fit":null,"object_position":null,"order"
:null,"overflow":null,"padding":null,"right":null,"top":null,"visibi
lity":null,"width":"100%"}}, "252fdf321a7e4fddb7aedic3190f18dd":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}}, "27493d29fded48c5990e6cd20288444a":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
```

```
n_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}}, "27904b40edeb417dab6c921cf25a51ce": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, "2795284af072414aa0fa6ed9e8c56682": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "TextareaModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "TextareaModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "TextareaView", "continuous_update": true, "description": "Plan text", "description_allow_html": false, "disabled": false, "layout": "IPY_MODEL_8fe1a2877e3c41bbac262e576aba1ec7", "placeholder": "", "rows": null, "style": "IPY_MODEL_eb35d19682304113bcabf87f48cbc73c", "tabbable": null, "tooltip": null, "value": ""}}, "280dfe33a3d54c1caec0d68fc56b7f35": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "HTMLModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "HTMLModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "HTMLView", "description": "", "description_allow_html": false, "layout": "IPY_MODEL_dc5b4cc4af754cb0abd700cc71fc63bf", "placeholder": "", "style": "IPY_MODEL_d7ffef5ed9d84732b24547d0075f7e1a", "tabbable": null, "tooltip": null, "value": "<i>Using Labs JSON from Phase-2 panel</i>"}}, "28db8905b1074f90b35840b2caf40eaa": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null
```

```
, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "2c25f650ebf6468fa0f4109d3f42749a": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "HTMLStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "HTMLStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": "", "font_size": null, "text_color": null}}, "2c54a712aeff46e8ae92b47b5a9d9c2c": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "ButtonModel", "state": {"_dom_classes": [], "model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "ButtonModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "ButtonView", "button_style": "", "description": "Add/Update unit", "disabled": false, "icon": "", "layout": "IPY_MODEL_ec00d56f7fd1448db276bbf9091f1f8e", "style": "IPY_MODEL_4812940bd97240a3a929b8e45eb85b58", "tabbable": null, "tooltip": null}}, "2f8613c70fec48e481405299f36c4632": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "ButtonModel", "state": {"_dom_classes": [], "model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "ButtonModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "ButtonView", "button_style": "primary", "description": "Run ML → OPS (inline)", "disabled": false, "icon": "", "layout": "IPY_MODEL_bb2302f602f24743aa9c7d0279908522", "style": "IPY_MODEL_d78025e056794eb39e8e98e4f580f750", "tabbable": null, "tooltip": null}}, "32bd503290e9479fa53baf9c1ba570e6": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "TextModel", "state": {"_dom_classes": [], "model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "TextModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "TextView", "continuous_update": true, "description": "Scan path", "description_allow_html": false, "disabled": false, "layout": "IPY_MODEL_8b8c19532f9c41adb0b30cd1bd240e48", "placeholder": "/mnt/data/scan.pdf (optional)", "style": "IPY_MODEL_ec63c072b818418388a6decd50eaeef22", "tabbable": null, "tooltip": null, "value": ""}}, "332760ac71994f7baf988074f9825be1": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxModel", "state": {"_dom_classes": [], "model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "CheckboxView", "description": "Prior VTE", "description_allow_html": false, "disabled": false, "indent": true, "layout": "IPY_MODEL_86f8e208eea24dda85c1ea5846f820da", "style": "IPY_MODEL_3d968bb2cffc48a3887cd5f6de21af8c", "tabbable": null, "tooltip": null, "value": false}}, "334ecaa4063a46e8b30f3e88db966f48":
```

```
{"model_module": "@jupyter-widgets/  
controls", "model_module_version": "2.0.0", "model_name": "HBoxModel", "s  
tate": {"_dom_classes": [], "_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "HBoxModel",  
"_view_count": null, "_view_module": "@jupyter-widgets/  
controls", "_view_module_version": "2.0.0", "_view_name": "HBoxView", "bo  
x_style": "", "children":  
["IPY_MODEL_70fd9c8a54834391af62898b924f5802", "IPY_MODEL_122605c8e19  
a4d1497f3eb41213bb18b", "IPY_MODEL_809833c0115a4a949368cb4153bb6d53"]  
}, "layout": "IPY_MODEL_fbda97691aad43cb8d2f5889c96ce4be", "tabbable": nu  
ll, "tooltip": null}}, "33ef02f2cddd480388295febf88291f0":  
{"model_module": "@jupyter-widgets/  
controls", "model_module_version": "2.0.0", "model_name": "HTMLModel", "s  
tate": {"_dom_classes": [], "_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "HTMLModel",  
"_view_count": null, "_view_module": "@jupyter-widgets/  
controls", "_view_module_version": "2.0.0", "_view_name": "HTMLView", "de  
scription": "", "description_allow_html": false, "layout": "IPY_MODEL_274  
93d29fded48c5990e6cd20288444a", "placeholder": "  
", "style": "IPY_MODEL_ba42ec43f8e54e89af822c8ab400214b", "tabbable": nu  
ll, "tooltip": null, "value": "<b>PE / PERC flags</  
b>"}}, "346f8b35c2dc4d4887a52dd6579f19a2": {"model_module": "@jupyter-  
widgets/  
controls", "model_module_version": "2.0.0", "model_name": "VBoxModel", "s  
tate": {"_dom_classes": [], "_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "VBoxModel",  
"_view_count": null, "_view_module": "@jupyter-widgets/  
controls", "_view_module_version": "2.0.0", "_view_name": "VBoxView", "bo  
x_style": "", "children":  
["IPY_MODEL_6137c34f40034564b65eaa353d487968", "IPY_MODEL_50b72a067e3  
c43ecbd566ac5bfe2406e", "IPY_MODEL_98e36cdd33834e57ba7ad71532969581",  
"IPY_MODEL_33ef02f2cddd480388295febf88291f0", "IPY_MODEL_465f32790523  
44feb7e5c5152701ea9e", "IPY_MODEL_4288779ab5374e80b172e7383bb48775", "  
IPY_MODEL_dca45eb8def6474a91c4f2cc05ff3c91", "IPY_MODEL_a49b7b8c235b4  
c5ba892c40bd0e275e3", "IPY_MODEL_867b28bb3c6646eea03c085eed5f9f33", "I  
PY_MODEL_58e54b651cfa450e8bd1fc41a20f73a3", "IPY_MODEL_17e0c42dbcaa4e  
5d92699b9c81085fff", "IPY_MODEL_d82b858d202e4280b8f317db1d7174ab", "IP  
Y_MODEL_9f204cd7c63e447c9d4b723b07da148f", "IPY_MODEL_e87d17e7a91b4b3  
286d31fad7c715cd1"], "layout": "IPY_MODEL_a7b0c06d501f4a90a6756d2e3df1  
3fb4", "tabbable": null, "tooltip": null}}, "36345b2a929f4820b12da82e1983  
1068": {"model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta  
te": {"_model_module": "@jupyter-widgets/  
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_  
view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig  
n_content": null, "align_items": null, "align_self": null, "border_bottom"  
: null, "border_left": null, "border_right": null, "border_top": null, "bott  
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul  
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nu  
ll, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template  
_areas": null, "grid_template_columns": null, "grid_template_rows": null,  
"height": null, "justify_content": null, "justify_items": null, "left": nul  
l, "margin": null, "max_height": null, "max_width": null, "min_height": null
```

```
, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "3bf6c1262f514b5b85c62a4dc2351c8b": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "3c06bd5f469a466a97d74ca044597c3d": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "CheckboxView", "description": "Active cancer", "description_allow_html": false, "disabled": false, "indent": true, "layout": "IPY_MODEL_69990097297445d5be67e83ba0ccb07a", "style": "IPY_MODEL_93e1800fcf2b43209e970cd40625b2b9", "tabbable": null, "tooltip": null, "value": false}}, "3cf589dab4934038b35eaec7dd4d415": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "ButtonStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "ButtonStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "button_color": null, "font_family": null, "font_size": null, "font_style": null, "font_variant": null, "font_weight": null, "text_color": null, "text_decoration": null}}, "3d6ce717b57545bc9834426e29d822b1": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null
```

```
, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "3d968bb2cffc48a3887cd5f6de21af8c": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, "3dab4615c04644e39dd753e893fb787f": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "3e6de6539d49495cb92388132aebb37b": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, "3ea7d9e280c245caa5df65be01c7b95d": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "3fdb829ca8c84cc280c94170ce21566c": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyleModel", "state": {"_model_module": "@jupyter-widgets/
```

```
controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, "412fd5f9d964411389cbc8f70a24435e": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "CheckboxView", "description": "Pitting edema (symptomatic leg)", "description_allow_html": false, "disabled": false, "indent": true, "layout": "IPY_MODEL_ca98ac24ef274745a601b095cfbd67b5", "style": "IPY_MODEL_e5d63e456cce4058946d8d40787b060c", "tabbable": null, "tooltip": null, "value": false}}, "4288779ab5374e80b172e7383bb48775": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "HBoxModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "HBoxModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "HBoxView", "box_style": "", "children": ["IPY_MODEL_cd4adcd15ec54d968ac3ea5a66e66004", "IPY_MODEL_7366b63e56e540a99689904335d7f578", "IPY_MODEL_eed78e5883254d32825fbb625bb7308"]}, "layout": "IPY_MODEL_cadbc032e3d444c0ba061a0411dd2335", "tabbable": null, "tooltip": null}}, "438acb80307b492d82d9f7f5eac1bff9": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "ButtonStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "ButtonStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "button_color": null, "font_family": null, "font_size": null, "font_style": null, "font_variant": null, "font_weight": null, "text_color": null, "text_decoration": null}}, "4606304b4fe245cda7f243b40c113fc9": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "CheckboxView", "description": "Bedridden ≥3d / surgery ≤12w", "description_allow_html": false, "disabled": false, "indent": true, "layout": "IPY_MODEL_3ea7d9e280c245caa5df65be01c7b95d", "style": "IPY_MODEL_e7ff180c1fff480bb25d68c6741e26fd", "tabbable": null, "tooltip": null, "value": false}}, "465f3279052344feb7e5c5152701ea9e": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "HBoxModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "HBoxModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "HBoxView", "box_style": "", "children": ["IPY_MODEL_a371067a38c24c99bd15404a0b234463", "IPY_MODEL_9ccfd9ca85e
```

```
8435b8bc8e3a789d5eb9c","IPY_MODEL_ce00d050bd954fac819c5102fea9c68d",
"IPY_MODEL_332760ac71994f7baf988074f9825be1","IPY_MODEL_c487f484ba13
4a3c8c0f0d46215107b5","IPY_MODEL_edb9ea9f786f4f5a9ecfc5326b625293"],
"layout":{"IPY_MODEL_7d0ade3da1e347dcb2e76fa771a8ba7f","tabbable":nul
l,"tooltip":null}},"46eb057cf00e4db4beb1c9e4cadca23a":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}},"4812940bd97240a3a929b8e45eb85b58":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"ButtonStyleMo
del","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"ButtonStyle
Model","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","butto
n_color":null,"font_family":null,"font_size":null,"font_style":null,
"font_variant":null,"font_weight":null,"text_color":null,"text_decor
ation":null}},"49c3ebbe42ce4c89b915f182f84d55b0":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}},"4e2da9e844954ed5bf1daa4c8512bdb6":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
```

```
n_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},"4f44cb5207e9482c911c674771b158d8":{"model_module":"@jupyter-widgets/controls","model_module_version":"2.0.0","model_name":"HTMLStyleModel","state":{"_model_module":"@jupyter-widgets/controls","_model_module_version":"2.0.0","_model_name":"HTMLStyleModel","_view_count":null,"_view_module":"@jupyter-widgets/base","_view_module_version":"2.0.0","_view_name":"StyleView","background":null,"description_width":"","font_size":null,"text_color":null}},"50b72a067e3c43ecbd566ac5bfe2406e":{"model_module":"@jupyter-widgets/controls","model_module_version":"2.0.0","model_name":"HBoxModel","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/controls","_model_module_version":"2.0.0","_model_name":"HBoxModel","_view_count":null,"_view_module":"@jupyter-widgets/controls","_view_module_version":"2.0.0","_view_name":"HBoxView","box_style":"","children":["IPY_MODEL_062c76fe6dba4049938d15ec5371e799","IPY_MODEL_e3467ab4d883458f972192b6e2ed19b0"],"layout":"IPY_MODEL_8d61668749d54d8989c557eebf999c3c","tabbable":null,"tooltip":null}},"514eb7a4fb1c4240af81f8f95742a81c":{"model_module":"@jupyter-widgets/base","model_module_version":"2.0.0","model_name":"LayoutModel","state":{"_model_module":"@jupyter-widgets/base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_view_count":null,"_view_module":"@jupyter-widgets/base","_view_module_version":"2.0.0","_view_name":"LayoutView","align_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},"518a5121c8344b3db49025b4c72b99cf":{"model_module":"@jupyter-widgets/base","model_module_version":"2.0.0","model_name":"LayoutModel","state":{"_model_module":"@jupyter-widgets/base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_view_count":null,"_view_module":"@jupyter-widgets/base","_view_module_version":"2.0.0","_view_name":"LayoutView","align_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}}
```

```
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},"578e1489149143a597aa96571055272b":{"model_module":"@jupyter-widgets/controls","model_module_version":"2.0.0","model_name":"ButtonModel","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/controls","_model_module_version":"2.0.0","_model_name":"ButtonModel","_view_count":null,"_view_module":"@jupyter-widgets/controls","_view_module_version":"2.0.0","_view_name":"ButtonView","button_style":"primary","description":"Parse & Check","disabled":false,"icon":"","layout":"IPY_MODEL_83591ec70d6e4a98a365a75400636c07","style":"IPY_MODEL_624361bb9e094f74a71f20f04ec601f0","tabbable":null,"tooltip":null}},"578e91cb8b9d46138d2babc7db1a6f62":{"model_module":"@jupyter-widgets/controls","model_module_version":"2.0.0","model_name":"ButtonStyleModel","state":{"_model_module":"@jupyter-widgets/controls","_model_module_version":"2.0.0","_model_name":"ButtonStyleModel","_view_count":null,"_view_module":"@jupyter-widgets/base","_view_module_version":"2.0.0","_view_name":"StyleView","button_color":null,"font_family":null,"font_size":null,"font_style":null,"font_variant":null,"font_weight":null,"text_color":null,"text_decoration":null}},"57a34f95bbb64b49853211e77552750c":{"model_module":"@jupyter-widgets/base","model_module_version":"2.0.0","model_name":"LayoutModel","state":{"_model_module":"@jupyter-widgets/base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_view_count":null,"_view_module":"@jupyter-widgets/base","_view_module_version":"2.0.0","_view_name":"LayoutView","align_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},"57ee23f2c50f479ea7294240fe25d3dc":{"model_module":"@jupyter-widgets/controls","model_module_version":"2.0.0","model_name":"HTMLStyleModel","state":{"_model_module":"@jupyter-widgets/controls","_model_module_version":"2.0.0","_model_name":"HTMLStyleModel","_view_count":null,"_view_module":"@jupyter-widgets/base","_view_module_version":"2.0.0","_view_name":"StyleView","background":null,"description_width":"","font_size":null,"text_color":null}},"58b861ee27084f39a10ed72a78a593f8":{"model_module":"@jupyter-widgets/
```

```
controls", "model_module_version": "2.0.0", "model_name": "CheckboxModel",
"state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/controls",
"_model_module_version": "2.0.0", "_model_name": "CheckboxModel", "_view_count": null,
"_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0",
"_view_name": "CheckboxView", "description": "Calf swelling >3 cm",
"description_allow_html": false, "disabled": false, "indent": true, "layout": "IPY_MODEL_ef5dbcb03f974f4b870eb3abafb5253e",
"style": "IPY_MODEL_744201c5307a4d9a9601fc64329a8eae", "tabbable": null, "tooltip": null,
"value": false}}, "58e54b651cfa450e8bd1fc41a20f73a3": {"model_module": "@jupyter-widgets/controls",
"model_module_version": "2.0.0", "model_name": "TextareaModel", "state": {"_dom_classes": [],
"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0",
"_model_name": "TextareaModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls",
"_view_module_version": "2.0.0", "_view_name": "TextAreaView", "continuous_update": true,
"description": "Context JSON", "description_allow_html": false, "disabled": false, "layout": "IPY_MODEL_a7e366537b864608bdf7b77f9c00f1f5",
"placeholder": "", "rows": null, "style": "IPY_MODEL_bdc9d830127343b7ac4efb77502c61d4",
"tabbable": null, "tooltip": null, "value": "{\\\"pregnant\\\": true, \\\"suspected_infection\\\": true,
\\\"vasopressors\\\": false, \\\"pe_most_likely\\\": true, \\\"dvt_signs\\\": false, \\\"ascites\\\": \\\"mild\\\",
\\\"encephalopathy\\\": \\\"1-2\\\"}"}, "5ab77b4c8625493ebaafdfb434a911d8": {"model_module": "@jupyter-widgets/controls",
"model_module_version": "2.0.0", "model_name": "ButtonModel", "state": {"_dom_classes": [],
"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0",
"_model_name": "ButtonModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls",
"_view_module_version": "2.0.0", "_view_name": "ButtonView", "button_style": "", "description": "Show equipment status",
"disabled": false, "icon": "", "layout": "IPY_MODEL_e74185a5cc8d4b218b3c8b0814d876c1",
"style": "IPY_MODEL_973e519626ee424a9abeb164ae19c4bf", "tabbable": null, "tooltip": null}}, "5aed3945a6414bcd95e40f1ac9d5e01d": {"model_module": "@jupyter-widgets/controls",
"model_module_version": "2.0.0", "model_name": "ButtonStyleModel", "state": {"_model_module": "@jupyter-widgets/controls",
"_model_module_version": "2.0.0", "_model_name": "ButtonStyleModel", "_view_count": null,
"_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView",
"button_color": null, "font_family": null, "font_size": null, "font_style": null, "font_variant": null,
"font_weight": null, "text_color": null, "text_decoration": null}}, "5dc77b57b4ca47ca9d77b63c96a296f6": {"model_module": "@jupyter-widgets/controls",
"model_module_version": "2.0.0", "model_name": "CheckboxModel", "state": {"_dom_classes": [],
"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0",
"_model_name": "CheckboxModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls",
"_view_module_version": "2.0.0", "_view_name": "CheckboxView", "description": "Deep vein tenderness",
"description_allow_html": false, "disabled": false, "indent": true, "layout": "IPY_MODEL_9fa4aa4cdd054fd0a4864f9a756636cb",
"style": "IPY_MODEL_72571bda9787457991a9703f5a2718e4", "tabbable": null, "toolti
```

```
p":null,"value":false}}, "6137c34f40034564b65eaa353d487968":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"HTMLModel","s
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"HTMLModel",
"_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"HTMLView","de
scription":"","description_allow_html":false,"layout":"IPY_MODEL_514
eb7a4fb1c4240af81f8f95742a81c","placeholder":
", "style":"IPY_MODEL_2c25f650ebf6468fa0f4109d3f42749a","tabbable":nu
ll,"tooltip":null,"value":"<b>Phase-2 calculators – optimized UI
(PE/DVT/PERC, PESI/sPESI, SIRS/Sepsis3, HEART, GBS, Child-Pugh)</
b>"}}, "621836ee06554d5ba10e062a6e49e9b4":{"model_module":"@jupyter-
widgets/
controls","model_module_version":"2.0.0","model_name":"DescriptionSt
yleModel","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"Description
StyleModel","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","descr
iption_width":""}}, "624361bb9e094f74a71f20f04ec601f0":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"ButtonStyleMo
del","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"ButtonStyle
Model","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","butto
n_color":null,"font_family":null,"font_size":null,"font_style":null,
"font_variant":null,"font_weight":null,"text_color":null,"text_decor
ation":null}}, "628f75a578f2454da6d1deff1eb77324":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"CheckboxModel
","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"CheckboxMod
el","_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"CheckboxView"
,"description":"Entire leg
swollen","description_allow_html":false,"disabled":false,"indent":tr
ue,"layout":"IPY_MODEL_9e7ae7a72b2c41fa8b3799f004e5802a","style":"IP
Y_MODEL_3e6de6539d49495cb92388132aebb37b","tabbable":null,"tooltip":
null,"value":false}}, "6355d35cb4f1401e9f4ac206e26ffe4a":
{"model_module":"@jupyter-widgets/
output","model_module_version":"1.0.0","model_name":"OutputModel","s
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
output","_model_module_version":"1.0.0","_model_name":"OutputModel",
"_view_count":null,"_view_module":"@jupyter-widgets/
output","_view_module_version":"1.0.0","_view_name":"OutputView","la
yout":"IPY_MODEL_87ac5ec53bd74cfaaf1d373f5d73f314","msg_id":"","outp
uts":
[], "tabbable":null,"tooltip":null}}, "657b2a82227e4f26ba2f6f8857c3b28
b":{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"VBoxModel","s
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"VBoxModel",
"_view_count":null,"_view_module":"@jupyter-widgets/
```

```
controls", "_view_module_version": "2.0.0", "_view_name": "VBoxView", "box_style": "", "children":  
[{"IPY_MODEL_5ab77b4c8625493ebaafdfb434a911d8", "IPY_MODEL_c6686faf278d41f2a52bb67fb24c408a"}, {"layout": "IPY_MODEL_4e2da9e844954ed5bf1daa4c8512bdb6", "tabbable": null, "tooltip": null}}, {"68f1bf56a5c34a93b2e9f11c91c44d33": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, {"6994236ac01541c998db062edbada6e2": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "HTMLStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "HTMLStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": "", "font_size": null, "text_color": null}}, {"69990097297445d5be67e83ba0ccb07a": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, {"6aac149e583a4e9d8153867d7a8b4033": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "TextStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "TextStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": "", "font_size": null, "text_color": null}}
```

```
l}}, "6ab8aab939614fe4a111b55d645f4d57": {"model_module": "@jupyter-  
widgets/  
controls", "model_module_version": "2.0.0", "model_name": "DescriptionSt  
yleModel", "state": {"_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "Description  
StyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "descr  
iption_width": ""}}, "6bbf707a9a2749e3b456bd5f79d1028c":  
{"model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta  
te": {"_model_module": "@jupyter-widgets/  
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_  
view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig  
n_content": null, "align_items": null, "align_self": null, "border_bottom"  
: null, "border_left": null, "border_right": null, "border_top": null, "bott  
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul  
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nul  
l, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template  
_areas": null, "grid_template_columns": null, "grid_template_rows": null,  
"height": null, "justify_content": null, "justify_items": null, "left": nul  
l, "margin": null, "max_height": null, "max_width": null, "min_height": null  
,"min_width": null, "object_fit": null, "object_position": null, "order": n  
ull, "overflow": null, "padding": null, "right": null, "top": null, "visibili  
ty": null, "width": null}}, "6ef8e8e34a8f4d42bf97a2a9eb7e6ee4":  
{"model_module": "@jupyter-widgets/  
controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyle  
Model", "state": {"_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxSty  
leModel", "_view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "backg  
round": null, "description_width": ""}}, "70633b6e793747088261b8b788e45f  
e7": {"model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta  
te": {"_model_module": "@jupyter-widgets/  
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_  
view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig  
n_content": null, "align_items": null, "align_self": null, "border_bottom"  
: null, "border_left": null, "border_right": null, "border_top": null, "bott  
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul  
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nul  
l, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template  
_areas": null, "grid_template_columns": null, "grid_template_rows": null,  
"height": null, "justify_content": null, "justify_items": null, "left": nul  
l, "margin": null, "max_height": null, "max_width": null, "min_height": null  
,"min_width": null, "object_fit": null, "object_position": null, "order": n  
ull, "overflow": null, "padding": null, "right": null, "top": null, "visibili  
ty": null, "width": null}}, "70fd9c8a54834391af62898b924f5802":  
{"model_module": "@jupyter-widgets/  
controls", "model_module_version": "2.0.0", "model_name": "TextModel", "s  
tate": {"_dom_classes": [], "_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "TextModel",  
"_view_count": null, "_view_module": "@jupyter-widgets/
```

```
controls", "_view_module_version": "2.0.0", "_view_name": "TextView", "continuous_update": true, "description": "Name", "description_allow_html": false, "disabled": false, "layout": "IPY_MODEL_9b87d4bae1ee41b0b4bba7303f0c867f", "placeholder": "ICU-North", "style": "IPY_MODEL_f999672bd73f433e9137379b8c38fde0", "tabbable": null, "tooltip": null, "value": "1A Neurochirurgische Intensivstation"}}, "72571bda9787457991a9703f5a2718e4": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, "7366b63e56e540a99689904335d7f578": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "CheckboxView", "description": "Recent trauma (4w, PERC)", "description_allow_html": false, "disabled": false, "indent": true, "layout": "IPY_MODEL_46eb057cf00e4db4beb1c9e4cadca23a", "style": "IPY_MODEL_00fcfdb360f74a4d862dc013f56a64a6", "tabbable": null, "tooltip": null, "value": false}}, "744201c5307a4d9a9601fc64329a8eae": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, "77025d605f9e4f4185e5bac864527411": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "ButtonStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "ButtonStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "button_color": null, "font_family": null, "font_size": null, "font_style": null, "font_variant": null, "font_weight": null, "text_color": null, "text_decoration": null}}, "7d0ade3da1e347dcb2e76fa771a8ba7f": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null
```

```
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}},"7f824fbfd0614a58a0661cf7e3a566fe":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}},"809833c0115a4a949368cb4153bb6d53":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"IntTextModel"
,"state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"IntTextMode
l","_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"IntTextView",
"continuous_update":false,"description":"Occupied","description_allo
w_html":false,"disabled":false,"layout":"IPY_MODEL_a2a9625c150f4c98b
0d67e467ee2f591","step":1,"style":"IPY_MODEL_922371176cfe45789329b18
d00c17d65","tabbable":null,"tooltip":null,"value":12}},"8217ade8234d
4ce2bfff9c2589083dbd4":{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"HBoxModel","s
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"HBoxModel",
"_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"HBoxView","bo
x_style":"","children":
["IPY_MODEL_d4f3bb63235d4d358c0328ec0a635acf","IPY_MODEL_578e1489149
143a597aa96571055272b"],"layout":"IPY_MODEL_70633b6e793747088261b8b7
88e45fe7","tabbable":null,"tooltip":null}},"83591ec70d6e4a98a365a754
00636c07":{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
```

```
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}},"867b28bb3c6646eea03c085eed5f9f33":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"HBoxModel","s
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"HBoxModel",
"_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"HBoxView","bo
x_style":"","children":
["IPY_MODEL_58b861ee27084f39a10ed72a78a593f8","IPY_MODEL_412fd5f9d96
4411389cbc8f70a24435e","IPY_MODEL_a972b728351f4e5ba91aaa36868b656e",
"IPY_MODEL_1653cdfec14c486b8754d52399e5d47e","IPY_MODEL_d3c7285e15c5
40d98c95793b9bda7ddd"],"layout":"IPY_MODEL_6bbf707a9a2749e3b456bd5f7
9d1028c","tabbable":null,"tooltip":null}},"86f8e208eea24dda85c1ea584
6f820da":{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}},"87ac5ec53bd74cfaaf1d373f5d73f314":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}},"89ae5b5f6efb4c208c60136fbd19ea41":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
```

```
view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"LayoutView","align_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}}, "8b8c19532f9c41adb0b30cd1bd240e48":  
{"model_module":"@jupyter-widgets/  
base","model_module_version":"2.0.0","model_name":"LayoutModel","state":{"_model_module":"@jupyter-widgets/  
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"LayoutView","align_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}}, "8bbc81d06cb2427e8aaa5439236fad8e":  
{"model_module":"@jupyter-widgets/  
controls","model_module_version":"2.0.0","model_name":"TextStyleModel","state":{"_model_module":"@jupyter-widgets/  
controls","_model_module_version":"2.0.0","_model_name":"TextStyleModel","_view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"StyleView","background":null,"description_width":"","font_size":null,"text_color":null}}, "8cdb4f2dc9f94e37a4c93315e235bd63":{"model_module":"@jupyter-  
widgets/  
base","model_module_version":"2.0.0","model_name":"LayoutModel","state":{"_model_module":"@jupyter-widgets/  
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"LayoutView","align_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibili
```

```
ty":null,"width":null}}, "8d61668749d54d8989c557eebf999c3c":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}}, "8fe1a2877e3c41bbac262e576aba1ec7":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":"180px","justify_content":null,"justify_items":null,"left":
null,"margin":null,"max_height":null,"max_width":null,"min_height":n
ull,"min_width":null,"object_fit":null,"object_position":null,"order
":null,"overflow":null,"padding":null,"right":null,"top":null,"visib
ility":null,"width":"100%"}}, "922371176cfe45789329b18d00c17d65":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"DescriptionSt
yleModel","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"Description
StyleModel","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","descr
iption_width":""}}, "93e1800fcf2b43209e970cd40625b2b9":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"CheckboxStyle
Model","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"CheckboxSty
leModel","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","backg
round":null,"description_width":""}}, "955513af811c43e8a4979154aa460b
ae":{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"DropdownModel
","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"DropdownMod
el","_options_labels":
```

```
[{"0","1","2"],"_view_count":null,"_view_module":"@jupyter-widgets/controls", "_view_module_version":"2.0.0", "_view_name":"DropdownView", "description":"HEART: Risk", "description_allow_html":false, "disabled":false, "index":0, "layout":"IPY_MODEL_7f824fbfd0614a58a0661cf7e3a566fe", "style":"IPY_MODEL_621836ee06554d5ba10e062a6e49e9b4", "tabbable":null, "tooltip":null}}, {"973e519626ee424a9abeb164ae19c4bf":{"model_module":"@jupyter-widgets/controls", "model_module_version":"2.0.0", "model_name":"ButtonStyleModel", "state":{"_model_module":"@jupyter-widgets/controls", "_model_module_version":"2.0.0", "_model_name":"ButtonStyleModel", "_view_count":null, "_view_module":"@jupyter-widgets/base", "_view_module_version":"2.0.0", "_view_name":"StyleView", "button_color":null, "font_family":null, "font_size":null, "font_style":null, "font_variant":null, "font_weight":null, "text_color":null, "text_decoration":null}}, {"97a7b595f538466e8dd3bb1024ba8bb8":{"model_module":"@jupyter-widgets/controls", "model_module_version":"2.0.0", "model_name":"HTMLStyleModel", "state":{"_model_module":"@jupyter-widgets/controls", "_model_module_version":"2.0.0", "_model_name":"HTMLStyleModel", "_view_count":null, "_view_module":"@jupyter-widgets/base", "_view_module_version":"2.0.0", "_view_name":"StyleView", "background":null, "description_width":"","font_size":null, "text_color":null}}, {"98e36cdd33834e57ba7ad71532969581":{"model_module":"@jupyter-widgets/controls", "model_module_version":"2.0.0", "model_name":"HBoxModel", "state":{"_dom_classes":[], "_model_module":"@jupyter-widgets/controls", "_model_module_version":"2.0.0", "_model_name":"HBoxModel", "_view_count":null, "_view_module":"@jupyter-widgets/controls", "_view_module_version":"2.0.0", "_view_name":"HBoxView", "box_style":"","children":["IPY_MODEL_be9d92ed3b904b14a18c8b06fb3c7e7a", "IPY_MODEL_ac16ccfb92b943ec9ad9289c15a06f0a", "IPY_MODEL_955513af811c43e8a4979154aa460bae"]}, {"layout":"IPY_MODEL_f948864f42c14e128ce0cf75db090a8a", "tabbable":null, "tooltip":null}}, {"9a30ae47c7a3436b8465cd7b6d333a4c":{"model_module":"@jupyter-widgets/controls", "model_module_version":"2.0.0", "model_name":"TextModel", "state":{"_dom_classes":[], "_model_module":"@jupyter-widgets/controls", "_model_module_version":"2.0.0", "_model_name":"TextModel", "_view_count":null, "_view_module":"@jupyter-widgets/controls", "_view_module_version":"2.0.0", "_view_name":"TextView", "continuous_update":true, "description":"ETAs (min)", "description_allow_html":false, "disabled":false, "layout":"IPY_MODEL_518a5121c8344b3db49025b4c72b99cf", "placeholder":"e.g. 30, 90, 180", "style":"IPY_MODEL_8bbc81d06cb2427e8aaa5439236fad8e", "tabbable":null, "tooltip":null, "value":""}}, {"9b87d4bae1ee41b0b4bba7303f0c867f":{"model_module":"@jupyter-widgets/base", "model_module_version":"2.0.0", "model_name":"LayoutModel", "state":{"_model_module":"@jupyter-widgets/base", "_model_module_version":"2.0.0", "_model_name":"LayoutModel", "_view_count":null, "_view_module":"@jupyter-widgets/base", "_view_module_version":"2.0.0", "_view_name":"LayoutView", "align_content":null, "align_items":null, "align_self":null, "border_bottom":null, "border_left":null, "border_right":null, "border_top":null, "bott
```

```
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}}, "9cbd504b11604160a0f38c3531bcf8e1": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, "9ccfd9ca85e8435b8bc8e3a789d5eb9c": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxModel", "_view_count": null, "_view_module": "@jupyter-widgets/controls", "_view_module_version": "2.0.0", "_view_name": "CheckboxView", "description": "DVT signs", "description_allow_html": false, "disabled": false, "indent": true, "layout": "IPY_MODEL_28db8905b1074f90b35840b2caf40eaa", "style": "IPY_MODEL_27904b40edeb417dab6c921cf25a51ce", "tabbable": null, "tooltip": null, "value": false}}, "9e27715d8855460a92d01cdd71dd1e3a": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "9e7ae7a72b2c41fa8b3799f004e5802a": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null,
```

```
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},"9ea27c38afb040d8bfb07d43636f4e7f":{"model_module":"@jupyter-widgets/controls","model_module_version":"2.0.0","model_name":"ButtonModel","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/controls","_model_module_version":"2.0.0","_model_name":"ButtonModel","_view_count":null,"_view_module":"@jupyter-widgets/controls","_view_module_version":"2.0.0","_view_name":"ButtonView","button_style":"","description":"Save status","disabled":false,"icon":"","layout":"IPY_MODEL_ad1fdf333b1749bcab856f5307b9b82b","style":"IPY_MODEL_3cf589dab4934038b35eaec7dd4d415","tabbable":null,"tooltip":null}},"9ecbfd005f8c4220996eec650dde7b00":{"model_module":"@jupyter-widgets/base","model_module_version":"2.0.0","model_name":"LayoutModel","state":{"_model_module":"@jupyter-widgets/base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_view_count":null,"_view_module":"@jupyter-widgets/base","_view_module_version":"2.0.0","_view_name":"LayoutView","align_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},"9f0bf86bfbfd248f083964139b8f00dd4":{"model_module":"@jupyter-widgets/base","model_module_version":"2.0.0","model_name":"LayoutModel","state":{"_model_module":"@jupyter-widgets/base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_view_count":null,"_view_module":"@jupyter-widgets/base","_view_module_version":"2.0.0","_view_name":"LayoutView","align_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},"9f204cd7c63e447c9d4b723b07da148f":{"model_module":"@jupyter-widgets/controls","model_module_version":"2.0.0","model_name":"TextareaModel","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
```

```
controls", "_model_module_version": "2.0.0", "_model_name": "TextareaModel",
"_view_count": null, "_view_module": "@jupyter-widgets/controls",
"_view_module_version": "2.0.0", "_view_name": "TextareaView",
"continuous_update": true, "description": "HL7
paste", "description_allow_html": false, "disabled": false, "layout": "IPY_MODEL_1abc26d9099242e0aee3a00097a8cadf", "placeholder": "",
"rows": null, "style": "IPY_MODEL_0927fa383af84923949a0beae319f902", "tabbable": null,
"tooltip": null, "value": "OBX|1|NM|BILIRUBIN TOTAL||36|umol/L|||N||F|||20250820090000\\nOBX|2|NM|INR||1.9|||N||F|||20250820090000\\nOBX|3|NM|ALBUMIN||28|g/L|||N||F|||20250820090000\\nOBX|4|NM|D-DIMER||780|ug/L|||H||F|||20250820090000\\nOBX|5|NM|TROPONIN T||78|ng/L|||H||F|||20250820090000\\nOBX|6|NM|TROPONIN T||18|ng/L|||N||F|||20250820050000"}}, "9f9c5716d47a4ef3ab3418052dae8226": {"model_module": "@jupyter-widgets/controls",
"model_module_version": "2.0.0", "model_name": "DropdownModel", "state": {"_dom_classes": [],
"_model_module": "@jupyter-widgets/controls",
"_model_module_version": "2.0.0",
"_model_name": "DropdownModel",
"_options_labels": ["1A Neurochirurgische Intensivstation", "1B Neurologische Intensivstation", "1C Interdisziplinäre Intensivstation", "1D Interdisziplinäre Intensivstation", "1E Interdisziplinäre Intensivstation", "1F Operative Intensivstation", "1G Internistische Intensivstation", "H1b Kardiologische Intensivstation", "H1b Kardiologische Intensivstation", "H2b Intensivstation Gefäß- und Herzmedizin"],
"_view_count": null,
"_view_module": "@jupyter-widgets/controls",
"_view_module_version": "2.0.0",
"_view_name": "DropdownView",
"description": "Unit",
"description_allow_html": false,
"disabled": false,
"index": 0,
"layout": "IPY_MODEL_112565e2d4f34fddb795da4aad032c53",
"style": "IPY_MODEL_6ab8aab939614fe4a11b55d645f4d57",
"tabbable": null,
"tooltip": null}}, "9fa4aa4cdd054fd0a4864f9a756636cb": {"model_module": "@jupyter-widgets/base",
"model_module_version": "2.0.0",
"model_name": "LayoutModel",
"state": {"_model_module": "@jupyter-widgets/base",
"_model_module_version": "2.0.0",
"_model_name": "LayoutModel",
"_view_count": null,
"_view_module": "@jupyter-widgets/base",
"_view_module_version": "2.0.0",
"_view_name": "LayoutView",
"align_content": null,
"align_items": null,
"align_self": null,
"border_bottom": null,
"border_left": null,
"border_right": null,
"border_top": null,
"bottom": null,
"display": null,
"flex": null,
"flex_flow": null,
"grid_area": null,
"grid_auto_columns": null,
"grid_auto_flow": null,
"grid_auto_rows": null,
"grid_column": null,
"grid_gap": null,
"grid_row": null,
"grid_template_areas": null,
"grid_template_columns": null,
"grid_template_rows": null,
"height": null,
"justify_content": null,
"justify_items": null,
"left": null,
"margin": null,
"max_height": null,
"max_width": null,
"min_height": null,
"min_width": null,
"object_fit": null,
"object_position": null,
"order": null,
"overflow": null,
"padding": null,
"right": null,
"top": null,
"visibility": null,
"width": null}}, "9fa6931c65794362aa03f0eef93cd405": {"model_module": "@jupyter-widgets/output",
"model_module_version": "1.0.0",
"model_name": "OutputModel",
"state": {"_dom_classes": [],
"_model_module": "@jupyter-widgets/output",
"_model_module_version": "1.0.0",
"_model_name": "OutputModel",
"_view_count": null,
"_view_module": "@jupyter-widgets/output",
"_view_module_version": "1.0.0",
"_view_name": "OutputView",
"la
```

```
yout": "IPY_MODEL_afb0620f2a3942f4945d2f65528b92c6", "msg_id": "", "outputs":  
[], "tabbable": null, "tooltip": null}}, "a2a9625c150f4c98b0d67e467ee2f591": {"model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/  
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "a2c8b11707634bc89838334f7d1acf0a": {"model_module": "@jupyter-widgets/  
controls", "model_module_version": "2.0.0", "model_name": "VBoxModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "VBoxModel", "_view_count": null, "_view_module": "@jupyter-widgets/  
controls", "_view_module_version": "2.0.0", "_view_name": "VBoxView", "box_style": "", "children":  
["IPY_MODEL_0817e2629d6148e4b11015c46e3edca5", "IPY_MODEL_bcd3347d63d34c5aa84ef1943b16dd84", "IPY_MODEL_f42317e417a942fbb7452a2dcff29f3f", "IPY_MODEL_280dfe33a3d54c1caec0d68fc56b7f35", "IPY_MODEL_a8a411645de644249a790e55a9be84f5", "IPY_MODEL_b0f95ec4888145be82ed94dbee71d778"], "layout": "IPY_MODEL_ae7fdb70d389461c9939a5f04d2bccd3", "tabbable": null, "tooltip": null}}, "a371067a38c24c99bd15404a0b234463": {"model_module": "@jupyter-widgets/  
controls", "model_module_version": "2.0.0", "model_name": "CheckboxModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxModel", "_view_count": null, "_view_module": "@jupyter-widgets/  
controls", "_view_module_version": "2.0.0", "_view_name": "CheckboxView", "description": "PE most likely", "description_allow_html": false, "disabled": false, "indent": true, "layout": "IPY_MODEL_36345b2a929f4820b12da82e19831068", "style": "IPY_MODEL_b9d94302634a4ee68738dc3fd564d98f", "tabbable": null, "tooltip": null, "value": true}}, "a49b7b8c235b4c5ba892c40bd0e275e3": {"model_module": "@jupyter-widgets/  
controls", "model_module_version": "2.0.0", "model_name": "HBoxModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "HBoxModel", "_view_count": null, "_view_module": "@jupyter-widgets/  
controls", "_view_module_version": "2.0.0", "_view_name": "HBoxView", "box_style": "", "children":  
["IPY_MODEL_3c06bd5f469a466a97d74ca044597c3d", "IPY_MODEL_066b9d79860547ffabc26a71b6ea31ae", "IPY_MODEL_4606304b4fe245cda7f243b40c113fc9", "IPY_MODEL_5dc77b57b4ca47ca9d77b63c96a296f6", "IPY_MODEL_628f75a578f2
```

```
454da6d1deff1eb77324"],"layout":"IPY_MODEL_1964c0c2a3b44f54b62cc0060
962f132","tabbable":null,"tooltip":null}},{"a7b0c06d501f4a90a6756d2e3
df13fb4":{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}},{"a7e366537b864608bdf7b77f9c00f1f5":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":"90px","justify_content":null,"justify_items":null,"left":n
ull,"margin":null,"max_height":null,"max_width":null,"min_height":nu
ll,"min_width":null,"object_fit":null,"object_position":null,"order"
:null,"overflow":null,"padding":null,"right":null,"top":null,"visibi
lity":null,"width":"100%"}},{"a8257f80212045939c94d1c21bd6e0c8":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}},{"a8a411645de644249a790e55a9be84f5":
{"model_module":"@jupyter-widgets/
```

```
controls", "model_module_version": "2.0.0", "model_name": "ButtonModel",
"state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "ButtonModel",
"_view_count": null, "_view_module": "@jupyter-widgets/
controls", "_view_module_version": "2.0.0", "_view_name": "ButtonView", "
button_style": "", "description": "Compute Pregnancy YEARS +
Log", "disabled": false, "icon": "", "layout": "IPY_MODEL_c4116fd561e24b96
89939c986fd95e27", "style": "IPY_MODEL_5aed3945a6414bcd95e40f1ac9d5e01
d", "tabbable": null, "tooltip": null}}, "a972b728351f4e5ba91aaa36868b656
e": {"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "CheckboxModel",
"state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxMod
el", "_view_count": null, "_view_module": "@jupyter-widgets/
controls", "_view_module_version": "2.0.0", "_view_name": "CheckboxView",
"description": "Collateral non-
varicose", "description_allow_html": false, "disabled": false, "indent": t
rue, "layout": "IPY_MODEL_57a34f95bbb64b49853211e77552750c", "style": "I
PY_MODEL_cf7e52cb4b9f458881f6e232f1912901", "tabbable": null, "tooltip"
: null, "value": false}}, "aa50e281ba8d476893b68a2e387062cd":
{"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "HBoxModel", "s
tate": {"_dom_classes": [], "_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "HBoxModel",
"_view_count": null, "_view_module": "@jupyter-widgets/
controls", "_view_module_version": "2.0.0", "_view_name": "HBoxView", "bo
x_style": "", "children":
["IPY_MODEL_2c54a712aeff46e8ae92b47b5a9d9c2c", "IPY_MODEL_044f1791c73
c4966ba64441ed51d2fba", "IPY_MODEL_9ea27c38afb040d8bfb07d43636f4e7f"]
, "layout": "IPY_MODEL_0afb9ed37ce84de383952257972f01ee", "tabbable": nu
ll, "tooltip": null}}, "ac16ccfb92b943ec9ad9289c15a06f0a":
{"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "DropdownModel",
"state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "DropdownMod
el", "_options_labels":
["0", "1", "2"], "_view_count": null, "_view_module": "@jupyter-widgets/
controls", "_view_module_version": "2.0.0", "_view_name": "DropdownView",
"description": "HEART:
ECG", "description_allow_html": false, "disabled": false, "index": 0, "layo
ut": "IPY_MODEL_143ed71461854d6594f0b56ff94a0308", "style": "IPY_MODEL_
2213bebf52324a8faea0908b2888970e", "tabbable": null, "tooltip": null}},
ad1fdf333b1749bcab856f5307b9b82b": {"model_module": "@jupyter-widgets/
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta
te": {"_model_module": "@jupyter-widgets/
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_
view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig
n_content": null, "align_items": null, "align_self": null, "border_bottom"
: null, "border_left": null, "border_right": null, "border_top": null, "bott
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nu
ll, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template
_areas": null, "grid_template_columns": null, "grid_template_rows": null,
```

```
"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},"ae7fdb70d389461c9939a5f04d2bccd3":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","state":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","align_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},"ae89f3ca57724884b0b62141526a2a71":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"TextStyleModel","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"TextStyleModel","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","background":null,"description_width":"","font_size":null,"text_color":null}},"afb0620f2a3942f4945d2f65528b92c6":{"model_module":"@jupyter-
widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","state":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","align_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},"b0f95ec4888145be82ed94dbec71d778":
{"model_module":"@jupyter-widgets/
output","model_module_version":"1.0.0","model_name":"OutputModel","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
output","_model_module_version":"1.0.0","_model_name":"OutputModel","_view_count":null,"_view_module":"@jupyter-widgets/
output","_view_module_version":"1.0.0","_view_name":"OutputView","layout":"IPY_MODEL_ebd857d5c3e64024b9daa4050759dafb","msg_id":"","outp
```

```
uts":
[[], "tabbable": null, "tooltip": null}}, "b3829d2c861d45c3a4cdddef161554f
2": {"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "VBoxModel", "s
tate": {"_dom_classes": [], "_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "VBoxModel",
"_view_count": null, "_view_module": "@jupyter-widgets/
controls", "_view_module_version": "2.0.0", "_view_name": "VBoxView", "bo
x_style": "", "children":
["IPY_MODEL_151894b90397400db1ddcedcf7b7cc69", "IPY_MODEL_32bd503290e
9479fa53baf9c1ba570e6", "IPY_MODEL_8217ade8234d4ce2bfff9c2589083dbd4",
"IPY_MODEL_2795284af072414aa0fa6ed9e8c56682", "IPY_MODEL_1ef699aa4db8
4ff8afee13265b131a0e"], "layout": "IPY_MODEL_9e27715d8855460a92d01cdd7
1dd1e3a", "tabbable": null, "tooltip": null}}, "b4255c7dfa16499b8766662d5
d16e4f6": {"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "HTMLModel", "s
tate": {"_dom_classes": [], "_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "HTMLModel",
"_view_count": null, "_view_module": "@jupyter-widgets/
controls", "_view_module_version": "2.0.0", "_view_name": "HTMLView", "de
scription": "", "description_allow_html": false, "layout": "IPY_MODEL_9ec
bfd005f8c4220996eec650dde7b00", "placeholder": "
", "style": "IPY_MODEL_97a7b595f538466e8dd3bb1024ba8bb8", "tabbable": nu
ll, "tooltip": null, "value": "<b>ICU next-bed availability</
b>"}}, "b559e8fffc7b94ed3b17a1d140aa81b03": {"model_module": "@jupyter-
widgets/
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta
te": {"_model_module": "@jupyter-widgets/
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_
view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig
n_content": null, "align_items": null, "align_self": null, "border_bottom"
: null, "border_left": null, "border_right": null, "border_top": null, "bott
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nul
l, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template
_areas": null, "grid_template_columns": null, "grid_template_rows": null,
"height": null, "justify_content": null, "justify_items": null, "left": nul
l, "margin": null, "max_height": null, "max_width": null, "min_height": null
, "min_width": null, "object_fit": null, "object_position": null, "order": n
ull, "overflow": null, "padding": null, "right": null, "top": null, "visibili
ty": null, "width": null}}, "b9d94302634a4ee68738dc3fd564d98f":
{"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyle
Model", "state": {"_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxSty
leModel", "_view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "backg
round": null, "description_width": ""}}, "ba42ec43f8e54e89af822c8ab40021
4b": {"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "HTMLStyleMode
l", "state": {"_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "HTMLStyleMo
del", "_view_count": null, "_view_module": "@jupyter-widgets/
```

```

base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "backg
round": null, "description_width": "", "font_size": null, "text_color": nul
l}}, "bb2302f602f24743aa9c7d0279908522": {"model_module": "@jupyter-
widgets/
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta
te": {"_model_module": "@jupyter-widgets/
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_
view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "alig
n_content": null, "align_items": null, "align_self": null, "border_bottom"
: null, "border_left": null, "border_right": null, "border_top": null, "bott
om": null, "display": null, "flex": null, "flex_flow": null, "grid_area": nul
l, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": nul
l, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template
_areas": null, "grid_template_columns": null, "grid_template_rows": null,
"height": null, "justify_content": null, "justify_items": null, "left": nul
l, "margin": null, "max_height": null, "max_width": null, "min_height": null
, "min_width": null, "object_fit": null, "object_position": null, "order": n
ull, "overflow": null, "padding": null, "right": null, "top": null, "visibili
ty": null, "width": null}}, "bcd3347d63d34c5aa84ef1943b16dd84":
{"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "HTMLModel", "s
tate": {"_dom_classes": [], "_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "HTMLModel",
"_view_count": null, "_view_module": "@jupyter-widgets/
controls", "_view_module_version": "2.0.0", "_view_name": "HTMLView", "de
scription": "", "description_allow_html": false, "layout": "IPY_MODEL_252
fdf321a7e4fddbd7aedc3190f18dd", "placeholder": "
", "style": "IPY_MODEL_0cf762a039b14ec498fa9c60a33c32fd", "tabbable": nu
ll, "tooltip": null, "value": "<small><b>Note:</b> PERC is not validated
in pregnancy; use the YEARS pathway below.</
small>"}}, "bdc9d830127343b7ac4efb77502c61d4":
{"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "TextStyleMode
l", "state": {"_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "TextStyleMo
del", "_view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "backg
round": null, "description_width": "", "font_size": null, "text_color": nul
l}}, "be9d92ed3b904b14a18c8b06fb3c7e7a": {"model_module": "@jupyter-
widgets/
controls", "model_module_version": "2.0.0", "model_name": "DropdownModel
", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "DropdownMod
el", "_options_labels":
["0", "1", "2"], "_view_count": null, "_view_module": "@jupyter-widgets/
controls", "_view_module_version": "2.0.0", "_view_name": "DropdownView"
, "description": "HEART:
History", "description_allow_html": false, "disabled": false, "index": 0, "
layout": "IPY_MODEL_1be04ed5691e4069910c21ccf482f1ed", "style": "IPY_MO
DEL_ea3ad9aa7438401f8609d5d525b0ba18", "tabbable": null, "tooltip": null
}}, "c4116fd561e24b9689939c986fd95e27": {"model_module": "@jupyter-
widgets/
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "sta

```

```
te":{"_model_module":"@jupyter-widgets/  
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_  
view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig  
n_content":null,"align_items":null,"align_self":null,"border_bottom"  
:null,"border_left":null,"border_right":null,"border_top":null,"bott  
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul  
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu  
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template  
_areas":null,"grid_template_columns":null,"grid_template_rows":null,  
"height":null,"justify_content":null,"justify_items":null,"left":nul  
l,"margin":null,"max_height":null,"max_width":null,"min_height":null  
,"min_width":null,"object_fit":null,"object_position":null,"order":n  
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili  
ty":null,"width":null}},"c487f484ba134a3c8c0f0d46215107b5":  
{"model_module":"@jupyter-widgets/  
controls","model_module_version":"2.0.0","model_name":"CheckboxModel  
","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/  
controls","_model_module_version":"2.0.0","_model_name":"CheckboxMod  
el","_view_count":null,"_view_module":"@jupyter-widgets/  
controls","_view_module_version":"2.0.0","_view_name":"CheckboxView"  
,"description":"Hemoptysis","description_allow_html":false,"disabled  
":false,"indent":true,"layout":"IPY_MODEL_d7c8d389c0594d0fafa8368c69  
5895e2","style":"IPY_MODEL_10235bb45f7d41c8815c4b632ab563b9","tabbab  
le":null,"tooltip":null,"value":false}},"c49d5de4436b40a0be9a25f0978  
9e537":{"model_module":"@jupyter-widgets/  
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta  
te":{"_model_module":"@jupyter-widgets/  
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_  
view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig  
n_content":null,"align_items":null,"align_self":null,"border_bottom"  
:null,"border_left":null,"border_right":null,"border_top":null,"bott  
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul  
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu  
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template  
_areas":null,"grid_template_columns":null,"grid_template_rows":null,  
"height":null,"justify_content":null,"justify_items":null,"left":nul  
l,"margin":null,"max_height":null,"max_width":null,"min_height":null  
,"min_width":null,"object_fit":null,"object_position":null,"order":n  
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili  
ty":null,"width":null}},"c6686faf278d41f2a52bb67fb24c408a":  
{"model_module":"@jupyter-widgets/  
output","model_module_version":"1.0.0","model_name":"OutputModel","s  
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/  
output","_model_module_version":"1.0.0","_model_name":"OutputModel",  
"_view_count":null,"_view_module":"@jupyter-widgets/  
output","_view_module_version":"1.0.0","_view_name":"OutputView","la  
yout":"IPY_MODEL_8cdb4f2dc9f94e37a4c93315e235bd63","msg_id":"","outp  
uts":  
[],"tabbable":null,"tooltip":null}},"c6760c4d81be4ee8988edfabcb7b7831  
a":{"model_module":"@jupyter-widgets/  
controls","model_module_version":"2.0.0","model_name":"CheckboxStyle  
Model","state":{"_model_module":"@jupyter-widgets/
```

```
controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, "c7e2e8688d7c40aabb07394160a6c80": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "ca98ac24ef274745a601b095cfbd67b5": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "cadc032e3d444c0ba061a0411dd2335": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}
```

```
ty":null,"width":null}}, "cd1a8109dbf841a4b1491ac66a412800":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}}, "cd4adcd15ec54d968ac3ea5a66e66004":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"CheckboxModel
","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"CheckboxMod
el","_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"CheckboxView"
,"description":"Estrogen use
(PERC)","description_allow_html":false,"disabled":false,"indent":tru
e,"layout":"IPY_MODEL_49c3ebbe42ce4c89b915f182f84d55b0","style":"IPY
_MODEL_c6760c4d81be4ee8988edfab7b7831a","tabbable":null,"tooltip":n
ull,"value":false}}, "ce00d050bd954fac819c5102fea9c68d":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"CheckboxModel
","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"CheckboxMod
el","_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"CheckboxView"
,"description":"Immobilized / recent surgery
(4w)","description_allow_html":false,"disabled":false,"indent":true,
"layout":"IPY_MODEL_89ae5b5f6efb4c208c60136fbd19ea41","style":"IPY_M
ODEL_9cbd504b11604160a0f38c3531bcf8e1","tabbable":null,"tooltip":nul
l,"value":false}}, "cf7e52cb4b9f458881f6e232f1912901":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"CheckboxStyle
Model","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"CheckboxSty
leModel","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","backg
round":null,"description_width":""}}, "cf8f143ba3fb42bfba01506abc0e0f
18":{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"CheckboxStyle
Model","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"CheckboxSty
leModel","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","backg
round":null,"description_width":""}}, "d3c7285e15c540d98c95793b9bda7d
```

```
dd":{"model_module":"@jupyter-widgets/  
controls","model_module_version":"2.0.0","model_name":"CheckboxModel",  
"state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/  
controls","_model_module_version":"2.0.0","_model_name":"CheckboxMod  
el","_view_count":null,"_view_module":"@jupyter-widgets/  
controls","_view_module_version":"2.0.0","_view_name":"CheckboxView",  
"description":"Alternative dx as likely  
(subtract)","description_allow_html":false,"disabled":false,"indent"  
:true,"layout":"IPY_MODEL_3bf6c1262f514b5b85c62a4dc2351c8b","style":  
"IPY_MODEL_6ef8e8e34a8f4d42bf97a2a9eb7e6ee4","tabbable":null,"toolti  
p":null,"value":false}}, "d4f3bb63235d4d358c0328ec0a635acf":  
{"model_module":"@jupyter-widgets/  
controls","model_module_version":"2.0.0","model_name":"ButtonModel",  
"state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/  
controls","_model_module_version":"2.0.0","_model_name":"ButtonModel",  
"_view_count":null,"_view_module":"@jupyter-widgets/  
controls","_view_module_version":"2.0.0","_view_name":"ButtonView",  
"button_style":"","description":"Run  
OCR","disabled":false,"icon":"","layout":"IPY_MODEL_3d6ce717b57545bc  
9834426e29d822b1","style":"IPY_MODEL_0eb31f30adb44afbb7561751873caa1  
9","tabbable":null,"tooltip":null}}, "d5b3ab9b3f904b9e88dbc2038f8a049  
c":{"model_module":"@jupyter-widgets/  
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta  
te":{"_model_module":"@jupyter-widgets/  
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_  
view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig  
n_content":null,"align_items":null,"align_self":null,"border_bottom"  
:null,"border_left":null,"border_right":null,"border_top":null,"bott  
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul  
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nul  
l,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_  
areas":null,"grid_template_columns":null,"grid_template_rows":null,  
"height":null,"justify_content":null,"justify_items":null,"left":nul  
l,"margin":null,"max_height":null,"max_width":null,"min_height":null  
,"min_width":null,"object_fit":null,"object_position":null,"order":n  
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili  
ty":null,"width":null}}, "d6462f488db7420db3c4e5a4b5143543":  
{"model_module":"@jupyter-widgets/  
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta  
te":{"_model_module":"@jupyter-widgets/  
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_  
view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig  
n_content":null,"align_items":null,"align_self":null,"border_bottom"  
:null,"border_left":null,"border_right":null,"border_top":null,"bott  
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul  
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nul  
l,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_  
areas":null,"grid_template_columns":null,"grid_template_rows":null,  
"height":null,"justify_content":null,"justify_items":null,"left":nul  
l,"margin":null,"max_height":null,"max_width":null,"min_height":null  
,"min_width":null,"object_fit":null,"object_position":null,"order":n  
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
```

```
ty":null,"width":null}}, "d78025e056794eb39e8e98e4f580f750":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"ButtonStyleMo
del","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"ButtonStyle
Model","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","butto
n_color":null,"font_family":null,"font_size":null,"font_style":null,
"font_variant":null,"font_weight":null,"text_color":null,"text_decor
ation":null}}, "d7c8d389c0594d0fafa8368c695895e2":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}}, "d7ffef5ed9d84732b24547d0075f7e1a":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"HTMLStyleMode
l","state":{"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"HTMLStyleMo
del","_view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"StyleView","backg
round":null,"description_width":"","font_size":null,"text_color":nul
l}}, "d82b858d202e4280b8f317db1d7174ab":{"model_module":"@jupyter-
widgets/
controls","model_module_version":"2.0.0","model_name":"HTMLModel","s
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"HTMLModel",
"_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"HTMLView","de
scription":"","description_allow_html":false,"layout":"IPY_MODEL_a82
57f80212045939c94d1c21bd6e0c8","placeholder":
"","style":"IPY_MODEL_57ee23f2c50f479ea7294240fe25d3dc","tabbable":nu
ll,"tooltip":null,"value":"<b>HL7 Labs (Trop / D-Dimer / INR /
Bili / Albumin)</b>"}}, "d9e4af503e0547678a9d5d3bb9279d91":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
```

```
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},{"dc5b4cc4af754cb0abd700cc71fc63bf":{"model_module":"@jupyter-widgets/base","model_module_version":"2.0.0","model_name":"LayoutModel","state":{"_model_module":"@jupyter-widgets/base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_view_count":null,"_view_module":"@jupyter-widgets/base","_view_module_version":"2.0.0","_view_name":"LayoutView","align_content":null,"align_items":null,"align_self":null,"border_bottom":null,"border_left":null,"border_right":null,"border_top":null,"bottom":null,"display":null,"flex":null,"flex_flow":null,"grid_area":null,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":null,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template_areas":null,"grid_template_columns":null,"grid_template_rows":null,"height":null,"justify_content":null,"justify_items":null,"left":null,"margin":null,"max_height":null,"max_width":null,"min_height":null,"min_width":null,"object_fit":null,"object_position":null,"order":null,"overflow":null,"padding":null,"right":null,"top":null,"visibility":null,"width":null}},{"dca45eb8def6474a91c4f2cc05ff3c91":{"model_module":"@jupyter-widgets/controls","model_module_version":"2.0.0","model_name":"HTMLModel","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/controls","_model_module_version":"2.0.0","_model_name":"HTMLModel","_view_count":null,"_view_module":"@jupyter-widgets/controls","_view_module_version":"2.0.0","_view_name":"HTMLView","description":"","description_allow_html":false,"layout":"IPY_MODEL_9f0bf86bfbfd248f083964139b8f00dd4","placeholder":"","style":"IPY_MODEL_229861ac14594327887a3ecc1ed6ee93","tabbable":null,"tooltip":null,"value":"<b>Wells-DVT flags</b>"},"e3467ab4d883458f972192b6e2ed19b0":{"model_module":"@jupyter-widgets/controls","model_module_version":"2.0.0","model_name":"TextareaModel","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/controls","_model_module_version":"2.0.0","_model_name":"TextareaModel","_view_count":null,"_view_module":"@jupyter-widgets/controls","_view_module_version":"2.0.0","_view_name":"TextAreaView","continuous_update":true,"description":"Labs JSON","description_allow_html":false,"disabled":false,"layout":"IPY_MODEL_f76a645f4195406cb0695a010cc44ce8","placeholder":"","rows":null,"style":"IPY_MODEL_0eb05c5d88c74d08a2295e0f1766a308","tabbable":null,"tooltip":null,"value":{"\\d_dimer\\":780,\\d_dimer_unit\\":\\\"µg/L FEU\\\",\\troponin_series\\":[{\\time\\":\\\"2025-08-20T05:00:00Z\\\",\\value\\":18,\\unit\\":\\\"ng/L\\\"},{\\time\\":\\\"2025-08-20T09:00:00Z\\\",\\value\\":78,\\unit\\":\\\"ng/L\\\"}],\\creatinine\\":1.8,\\platelets\\":95,\\albumin\\":2.8,\\calcium\\":7.9,\\na\\":138,\\cl\\":105,\\hco3\\":20,\\bilirubin\\":2.1,\\inr\\":1.9}}},"e3d3f8a44f4641ceabcbaf4a8c80263d":{"model_module":"@jupyter-
```

```
widgets/
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "e5d63e456cce4058946d8d40787b060c": {"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyleModel", "state": {"_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, "e74185a5cc8d4b218b3c8b0814d876c1": {"model_module": "@jupyter-widgets/
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "e7ff180c1fff480bb25d68c6741e26fd": {"model_module": "@jupyter-widgets/
controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyleModel", "state": {"_model_module": "@jupyter-widgets/
controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, "e87d17e7a91b4b3286d31fad7c715cd1": {"model_module": "@jupyter-widgets/
output", "model_module_version": "1.0.0", "model_name": "OutputModel", "state": {"_dom_classes": [], "_model_module": "@jupyter-widgets/
output", "_model_module_version": "1.0.0", "_model_name": "OutputModel", "_view_count": null, "_view_module": "@jupyter-widgets/
output", "_view_module_version": "1.0.0", "_view_name": "OutputView", "la
```

```
yout": "IPY_MODEL_cd1a8109dbf841a4b1491ac66a412800", "msg_id": "", "outputs":  
[[], "tabbable": null, "tooltip": null}}, "ea3ad9aa7438401f8609d5d525b0ba18": {"model_module": "@jupyter-widgets/  
controls", "model_module_version": "2.0.0", "model_name": "DescriptionStyleModel", "state": {"_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "DescriptionStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "description_width": ""}}, "eb35d19682304113bcabf87f48cbc73c": {"model_module": "@jupyter-widgets/  
controls", "model_module_version": "2.0.0", "model_name": "TextStyleModel", "state": {"_model_module": "@jupyter-widgets/  
controls", "_model_module_version": "2.0.0", "_model_name": "TextStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": "", "font_size": null, "text_color": null}}, "ebd857d5c3e64024b9daa4050759dafb": {"model_module": "@jupyter-  
widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/  
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "ec00d56f7fd1448db276bbf9091f1f8e": {"model_module": "@jupyter-widgets/  
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/  
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/  
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, "ec63c072b818418388a6dec50eaeef22": {"model_module": "@jupyter-widgets/  
controls", "model_module_version": "2.0.0", "model_name": "TextStyleModel"
```

```
l","state":{"_model_module":"@jupyter-widgets/  
controls","_model_module_version":"2.0.0","_model_name":"TextStyleMo  
del","_view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"StyleView","backg  
round":null,"description_width":"","font_size":null,"text_color":nul  
l}},{"edb9ea9f786f4f5a9ecfc5326b625293":{"model_module":"@jupyter-  
widgets/  
controls","model_module_version":"2.0.0","model_name":"CheckboxModel  
","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/  
controls","_model_module_version":"2.0.0","_model_name":"CheckboxMod  
el","_view_count":null,"_view_module":"@jupyter-widgets/  
controls","_view_module_version":"2.0.0","_view_name":"CheckboxView"  
,"description":"Active  
malignancy","description_allow_html":false,"disabled":false,"indent"  
:true,"layout":"IPY_MODEL_d6462f488db7420db3c4e5a4b5143543","style":  
"IPY_MODEL_14e0e99c3d2f4bea9499f712d68f2d8c","tabbable":null,"toolti  
p":null,"value":false}},{"ee5e96aa2434416baa964390a0d1568b":  
{"model_module":"@jupyter-widgets/  
controls","model_module_version":"2.0.0","model_name":"VBoxModel","s  
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/  
controls","_model_module_version":"2.0.0","_model_name":"VBoxModel",  
"_view_count":null,"_view_module":"@jupyter-widgets/  
controls","_view_module_version":"2.0.0","_view_name":"VBoxView","bo  
x_style":"","children":  
["IPY_MODEL_b4255c7dfa16499b8766662d5d16e4f6","IPY_MODEL_9f9c5716d47  
a4ef3ab3418052dae8226","IPY_MODEL_334ecaa4063a46e8b30f3e88db966f48",  
"IPY_MODEL_9a30ae47c7a3436b8465cd7b6d333a4c","IPY_MODEL_aa50e281ba8d  
476893b68a2e387062cd","IPY_MODEL_9fa6931c65794362aa03f0eef93cd405"],  
"layout":"IPY_MODEL_c7e2e8688d7c40aabbe07394160a6c80","tabbable":nul  
l,"tooltip":null}},{"eed78e5883254d32825fbbbe625bb7308":  
{"model_module":"@jupyter-widgets/  
controls","model_module_version":"2.0.0","model_name":"CheckboxModel  
","state":{"_dom_classes":[],"_model_module":"@jupyter-widgets/  
controls","_model_module_version":"2.0.0","_model_name":"CheckboxMod  
el","_view_count":null,"_view_module":"@jupyter-widgets/  
controls","_view_module_version":"2.0.0","_view_name":"CheckboxView"  
,"description":"Unilateral leg swelling  
(PERC)","description_allow_html":false,"disabled":false,"indent":tru  
e,"layout":"IPY_MODEL_0645fad0130d4e28b163318b2657ad3a","style":"IPY  
_MODEL_f7b3a29de6d84996911c1f732a4eddaa","tabbable":null,"tooltip":n  
ull,"value":false}},{"ef5dbcb03f974f4b870eb3abafb5253e":  
{"model_module":"@jupyter-widgets/  
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta  
te":{"_model_module":"@jupyter-widgets/  
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_  
view_count":null,"_view_module":"@jupyter-widgets/  
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig  
n_content":null,"align_items":null,"align_self":null,"border_bottom"  
:null,"border_left":null,"border_right":null,"border_top":null,"bott  
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul  
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu  
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template  
_areas":null,"grid_template_columns":null,"grid_template_rows":null,  
"height":null,"justify_content":null,"justify_items":null,"left":nul
```

```
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}},"f18a06d7c1c54a0794737cebc44b5adf":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":null,"justify_content":null,"justify_items":null,"left":nul
l,"margin":null,"max_height":null,"max_width":null,"min_height":null
,"min_width":null,"object_fit":null,"object_position":null,"order":n
ull,"overflow":null,"padding":null,"right":null,"top":null,"visibili
ty":null,"width":null}},"f42317e417a942fbb7452a2dcff29f3f":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"HTMLModel","s
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"HTMLModel",
"_view_count":null,"_view_module":"@jupyter-widgets/
controls","_view_module_version":"2.0.0","_view_name":"HTMLView","de
scription":"","description_allow_html":false,"layout":"IPY_MODEL_68f
1bf56a5c34a93b2e9f11c91c44d33","placeholder":""
","style":"IPY_MODEL_6994236ac01541c998db062edbada6e2","tabbable":nu
ll,"tooltip":null,"value":"<i>Using Context JSON from Phase-2
panel</i>"}}, "f76a645f4195406cb0695a010cc44ce8":
{"model_module":"@jupyter-widgets/
base","model_module_version":"2.0.0","model_name":"LayoutModel","sta
te":{"_model_module":"@jupyter-widgets/
base","_model_module_version":"2.0.0","_model_name":"LayoutModel","_
view_count":null,"_view_module":"@jupyter-widgets/
base","_view_module_version":"2.0.0","_view_name":"LayoutView","alig
n_content":null,"align_items":null,"align_self":null,"border_bottom"
:null,"border_left":null,"border_right":null,"border_top":null,"bott
om":null,"display":null,"flex":null,"flex_flow":null,"grid_area":nul
l,"grid_auto_columns":null,"grid_auto_flow":null,"grid_auto_rows":nu
ll,"grid_column":null,"grid_gap":null,"grid_row":null,"grid_template
_areas":null,"grid_template_columns":null,"grid_template_rows":null,
"height":"130px","justify_content":null,"justify_items":null,"left":
null,"margin":null,"max_height":null,"max_width":null,"min_height":n
ull,"min_width":null,"object_fit":null,"object_position":null,"order
":null,"overflow":null,"padding":null,"right":null,"top":null,"visib
ility":null,"width":"100%"}}, "f78bae808c4f4220b339229ab8421612":
{"model_module":"@jupyter-widgets/
controls","model_module_version":"2.0.0","model_name":"VBoxModel","s
tate":{"_dom_classes":[],"_model_module":"@jupyter-widgets/
controls","_model_module_version":"2.0.0","_model_name":"VBoxModel",
"_view_count":null,"_view_module":"@jupyter-widgets/
```

```
controls", "_view_module_version": "2.0.0", "_view_name": "VBoxView", "box_style": "", "children":  
[{"IPY_MODEL_2f8613c70fec48e481405299f36c4632", "IPY_MODEL_6355d35cb4f1401e9f4ac206e26ffe4a"}, {"layout": "IPY_MODEL_f909cdd768734c44af5d9d979f64774e", "tabbable": null, "tooltip": null}], {"f7b3a29de6d84996911c1f732a4eddaa": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "CheckboxStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "CheckboxStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": ""}}, {"f909cdd768734c44af5d9d979f64774e": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, {"f948864f42c14e128ce0cf75db090a8a": {"model_module": "@jupyter-widgets/base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}, {"f999672bd73f433e9137379b8c38fde0": {"model_module": "@jupyter-widgets/controls", "model_module_version": "2.0.0", "model_name": "TextStyleModel", "state": {"_model_module": "@jupyter-widgets/controls", "_model_module_version": "2.0.0", "_model_name": "TextStyleModel", "_view_count": null, "_view_module": "@jupyter-widgets/base", "_view_module_version": "2.0.0", "_view_name": "StyleView", "background": null, "description_width": "", "font_size": null, "text_color": null}}, {"fbda97691aad43cb8d2f5889c96ce4be": {"model_module": "@jupyter-
```

```

widgets/
base", "model_module_version": "2.0.0", "model_name": "LayoutModel", "state": {"_model_module": "@jupyter-widgets/
base", "_model_module_version": "2.0.0", "_model_name": "LayoutModel", "_view_count": null, "_view_module": "@jupyter-widgets/
base", "_view_module_version": "2.0.0", "_view_name": "LayoutView", "align_content": null, "align_items": null, "align_self": null, "border_bottom": null, "border_left": null, "border_right": null, "border_top": null, "bottom": null, "display": null, "flex": null, "flex_flow": null, "grid_area": null, "grid_auto_columns": null, "grid_auto_flow": null, "grid_auto_rows": null, "grid_column": null, "grid_gap": null, "grid_row": null, "grid_template_areas": null, "grid_template_columns": null, "grid_template_rows": null, "height": null, "justify_content": null, "justify_items": null, "left": null, "margin": null, "max_height": null, "max_width": null, "min_height": null, "min_width": null, "object_fit": null, "object_position": null, "order": null, "overflow": null, "padding": null, "right": null, "top": null, "visibility": null, "width": null}}}, {"version_major": 2, "version_minor": 0}}, {"nbformat_minor": 4, "nbformat": 4, "cells": [{"cell_type": "markdown", "source": "# GPT gate-engine implementation ED-Pipeline\n\nThis notebook contains the gate-first baseline (ops + 0/1h troponin), with CONFIG guards and a Sanity Pack. Kernelspec: **python3**.\n", "metadata": {"papermill": {"duration": 0.011297, "end_time": "2025-08-24T12:38:51.044072", "exception": false, "start_time": "2025-08-24T12:38:51.032775", "status": "completed"}, "tags": []}}, {"cell_type": "code", "source": "# --- TOP-OF-NOTEBOOK GUARDS / PATHS BOOTSTRAP ---\nimport os, json, datetime as _dt\n\n# Environment root\nif os.path.exists(\"/kaggle/working\"):\n    BASE = \"/kaggle/working\"\nelif os.path.exists(\"/content\"):\n    BASE = \"/content\"\nelse:\n    BASE = \"/mnt/data\"\nprint(\"BASE =\", BASE)\n\n_p = lambda *p: os.path.join(BASE, *p)\n\n# Single source of truth\nCONFIG = {\n    \"RUN_PIPELINE\": True,\n    \"RUN_UI\": True,\n    \"RUN_MEDS\": True,\n    \"RUN_SYNTH\": False,\n    \"DATA_ROOT\": BASE,\n    \"EQUIPMENT_STATUS_PATH\": _p(\"equipment_status.csv\"),\n    \"EQUIPMENT_MOVES_LOG_PATH\": _p(\"moves_log.csv\"),\n    \"SOP_REGISTRY_PATH\": _p(\"sop_registry.csv\"),\n    \"QR_OUTPUT_DIR\": _p(\"qrs\"),\n    \"EVENT_LOG_PATH\": _p(\"event_log.json\")\n}\n\n# FS prep\nos.makedirs(CONFIG[\"QR_OUTPUT_DIR\"], exist_ok=True)\nos.makedirs(os.path.dirname(CONFIG[\"EVENT_LOG_PATH\"]), exist_ok=True)\n\n# Logger that always uses current CONFIG (no stale capture)\ndef _append_event(ev: dict):\n    ev = {\"ts\": _dt.datetime.utcnow().isoformat() + \"Z\", ** (ev or {})}\n    with open(CONFIG[\"EVENT_LOG_PATH\"], \"a\", encoding=\"utf-8\") as f:\n        f.write(json.dumps(ev, ensure_ascii=False) + \"\\n\\n\")\n    print(\"EVENT_LOG_PATH ->\", CONFIG[\"EVENT_LOG_PATH\"])\n\n\", \"metadata\": {\"execution\": {\"iopub.status.busy\": \"2025-08-25T18:40:38.236176Z\", \"iopub.execute_input\": \"2025-08-25T18:40:38.236582Z\", \"iopub.status.idle\": \"2025-08-25T18:40:38.246441Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:38.236542Z\", \"shell.execute_reply\": \"2025-08-25T18:40:38.245451Z\"}}, \"papermill\": {\"duration\": 0.024459, \"end_time\": \"2025-08-24T12:38:51.078397\", \"exception\": false, \"start_time\": \"2025-08-24T12:38:51.053938\", \"status\": \"completed\", \"tags\": [], \"trusted\": true}, \"outputs\":

```

```
[{"name":"stdout","text":"BASE = /kaggle/working\nEVENT_LOG_PATH → /
kaggle/working/
event_log.json\n","output_type":"stream"}],{"execution_count":50},
{"cell_type":"code","source":"# Freeze CONFIG so later cells can't
silently flip flags/paths.\n_ALLOWED_CONFIG_KEYS_TO_CHANGE =
{'MODEL_BUNDLE_PATH'} # bundle loader may set this\n\nclass
_FrozenConfig(dict):\n    def __setitem__(self, k, v):\n        if k
in self and k not in _ALLOWED_CONFIG_KEYS_TO_CHANGE:\n
raise RuntimeError(f\"CONFIG is frozen; attempted to modify {k}.\
Keep a single source of truth.\")\n        super().__setitem__(k, v)
\n    def update(self, *args, **kwargs):\n        if args:\n
for k in args[0].keys():\n            if k in self and k not in
_ALLOWED_CONFIG_KEYS_TO_CHANGE:\n                raise
RuntimeError(f\"CONFIG is frozen; attempted to modify {k}.\")\n
for k in list(kwargs.keys()):\n            if k in self and k not in
_ALLOWED_CONFIG_KEYS_TO_CHANGE:\n                raise
RuntimeError(f\"CONFIG is frozen; attempted to modify {k}.\")\n
return super().update(*args, **kwargs)\n\nCONFIG =
_FrozenConfig(CONFIG)\nprint(\"CONFIG frozen. Only allowed later
change:\", _ALLOWED_CONFIG_KEYS_TO_CHANGE),\"metadata\":{\"execution\":
{\"iopub.status.busy\":\"2025-08-25T18:40:38.247780Z\",\"iopub.execute_in
put\":\"2025-08-25T18:40:38.248068Z\",\"iopub.status.idle\":\"2025-08-25T1
8:40:38.266441Z\",\"shell.execute_reply.started\":\"2025-08-25T18:40:38.
248039Z\",\"shell.execute_reply\":\"2025-08-25T18:40:38.265533Z\"},\"paper
mill\":
{\"duration\":0.019004,\"end_time\":\"2025-08-24T12:38:51.107386\",\"except
ion\":false,\"start_time\":\"2025-08-24T12:38:51.088382\",\"status\":\"compl
eted\"},\"tags\":[],\"trusted\":true},\"outputs\":
[{"name":"stdout","text":"CONFIG frozen. Only allowed later change:
{'MODEL_BUNDLE_PATH'}
\n","output_type":"stream"}],{"execution_count":51},
{"cell_type":"code","source":"# == PRE-FLIGHT (compact) - stitch &
safety checks ==\ndef preflight(verbose=True):\n    import inspect,
json, pathlib, sys\n    errs=[]; pr=print if verbose else (lambda
*a,**k:None)\n    def _ok(name, cond, msg=\"\"):\n        if cond:
pr(f\"[✓] {name}\")\n        else:    errs.append(f\"[x] {name}{'(' -
'+msg) if msg else ''}\")\n        # 0) CONFIG & appender\n
_ok(\"CONFIG present\", \"CONFIG\" in globals())\n    _ok(\"CONFIG
frozen\", \"CONFIG\" in globals() and (hasattr(CONFIG,
\"_FrozenConfig\") or isinstance(CONFIG, dict)))\n
_ok(\"_append_event present\", \"_append_event\" in globals())\n
if \"_append_event\" in globals():\n        try:\n
_ok(\"_append_event signature\", \"ev\" in
inspect.signature(_append_event).parameters)\n        except
Exception:\n            errs.append(f\"[x] _append_event signature -
cannot introspect\")\n            # 1) Unique surfaces\n            for n in
(\"run_ui\", \"refresh_sop_registry\"):\n                _ok(f\"no duplicate
{n}\", n in globals(), f\"{n} missing\")\n            # 2) HL7 +
calculators\n            _ok(\"parse_hl7_labs present\", \"parse_hl7_labs\"
in globals())\n            _ok(\"phase2_bundle present\", \"phase2_bundle\"
in globals())\n            # 3) Bundle one-liners (UI if available, else
fallback)\n            import json as _json\n            if all(x in globals() for x
in (\"vitals_in\", \"labs_in\", \"ctx_in\")):\n                try:\n
vit=_json.loads(vitals_in.value or \"{}\");
```

```

lab=_json.loads(labs_in.value or "{}");
ctx=_json.loads(ctx_in.value or "{}")\n
b=phase2_bundle(vit,lab,ctx); _ok("\bundle one-liners non-empty",
isinstance(b.get("\one_liners"),list) and
len(b["one_liners"]>3)\n      except Exception as e:\n
errs.append(f"\[x] bundle on UI inputs - {e}\")\n      else:\n
try:\n
vit={"rr":24,"sbp":95,"gcs":14,"hr":120,"temp":38.6,"avpu
":"V","map":65,"age":68,"sex":"M","sao2":97}\n
lab={"d_dimer":780,"d_dimer_unit":"µg/L FEU",
"creatinine":1.8,"platelets":95,"albumin":2.8,"calcium":7.9,
"na":138,"cl":105,"hco3":20,"bilirubin":2.1,"inr":1.9}\n
ctx={"suspected_infection":True,"pe_most_likely":True,
"dvt_signs":False}\n      b=phase2_bundle(vit,lab,ctx);
_ok("\bundle one-liners non-empty",
isinstance(b.get("\one_liners"),list) and
len(b["one_liners"]>3)\n      except Exception as e:\n
errs.append(f"\[x] bundle on fallback - {e}\")\n      # 4) ICU
helpers\n      for fn in ("\_load_icu_status",
"\_compute_next_bed_eta"):\n      _ok(f"\{fn} present", fn in
globals())\n      # 5) Event log path + append\n      try:\n
p=pathlib.Path(CONFIG["EVENT_LOG_PATH"]); _ok("\Event log path
exists or creatable", bool(p.parent.exists() or True))\n
_append_event({"type":"preflight_ping","ok":True});
_ok("\Event append works", True)\n      except Exception as e:\n
errs.append(f"\[x] Event append - {e}\")\n      # 6) Optional ML
wrapper\n      if "\predict_one" in globals() and "\FEAT" in
globals(): _ok("\predict_one wired", True)\n      else: pr("\[-] ML
wrapper not loaded (ok for now)\")\n      # Result\n      if errs:\n
pr("\n\nRESULT: FAIL"); pr("\n\n".join(errs)); return False\n
pr("\n\nRESULT: PASS"); return True\n\n# auto-run once when cell
executes\npreflight("\n","metadata":{"trusted":true,"execution":
{"iopub.status.busy":"2025-08-25T18:40:38.328206Z","iopub.execute_in
put":"2025-08-25T18:40:38.329071Z","iopub.status.idle":"2025-08-25T1
8:40:38.348559Z","shell.execute_reply.started":"2025-08-25T18:40:38.
329045Z","shell.execute_reply":"2025-08-25T18:40:38.347661Z"}}, "outp
uts":[{"name":"stdout","text":"[✓] CONFIG present\n[✓] CONFIG
frozen\n[✓] _append_event present\n[✓] _append_event signature\n[✓]
no duplicate run_ui\n[✓] phase2_bundle present\n[✓] bundle one-
liners non-empty\n[✓] _load_icu_status present\n[✓]
_compute_next_bed_eta present\n[✓] Event log path exists or
creatable\n[✓] Event append works\n[✓] predict_one wired\n\nRESULT:
FAIL\n[x] no duplicate refresh_sop_registry - refresh_sop_registry
missing\n[x] parse_hl7_labs present\n","output_type":"stream"},
{"execution_count":52,"output_type":"execute_result","data":{"text/
plain":"False"},"metadata":{}}],"execution_count":52},
{"cell_type":"code","source":"# --- Load model bundle from attached
dataset → set CONFIG path ---\nimport os, shutil\nDATASET = "\ed-
pipeline-bundle-ui" # ← replace with your actual dataset slug\nSRC
= f"/kaggle/input/{DATASET}/
ed_phase2_model_thr_patched(1).joblib"\nDST = "/kaggle/working/
ed_phase2_model_thr_patched(1).joblib"\n\nassert
os.path.exists(SRC), f"\Not found: {SRC} (check dataset slug/file
name)"
\nif not os.path.exists(DST):\n

```

```

os.makedirs(os.path.dirname(DST), exist_ok=True)\n
shutil.copy2(SRC, DST)\n\nCONFIG[\"MODEL_BUNDLE_PATH\"] =
DST\nCONFIG[\"SKIP_MODEL_DISCOVERY\"] = True # prevents fallbacks
from overriding this\nprint(\"Bundle ready →\",
CONFIG[\"MODEL_BUNDLE_PATH\"])\n\n,\"metadata\":{\"execution\":
{\"iopub.status.busy\":\"2025-08-25T18:40:38.350341Z\",\"iopub.execute_in
put\":\"2025-08-25T18:40:38.350662Z\",\"iopub.status.idle\":\"2025-08-25T1
8:40:38.380926Z\",\"shell.execute_reply.started\":\"2025-08-25T18:40:38.
350638Z\",\"shell.execute_reply\":\"2025-08-25T18:40:38.380029Z\"},\"paper
mill\":
{\"duration\":0.025592,\"end_time\":\"2025-08-24T12:38:51.143650\",\"except
ion\":false,\"start_time\":\"2025-08-24T12:38:51.118058\",\"status\":\"compl
eted\"},\"tags\":[],\"trusted\":true},\"outputs\":
[{\"name\":\"stdout\",\"text\":\"Bundle ready → /kaggle/working/
ed_phase2_model_thr_patched(1).joblib\\n\",\"output_type\":\"stream\"}],\"e
xecution_count\":53},{\"cell_type\":\"code\",\"source\":\"# === Self-heal +
alias lock (idempotent, no side-effects beyond pkg setup)
===\\nimport sys, os, importlib, inspect, shutil\\nfrom pathlib import
Path\\n\\nWORK_ROOT = Path(\"/kaggle/working\")\\nPKG_ROOT =
WORK_ROOT / \"edtracker_pkg\"\\nPKG_DIR = PKG_ROOT /
\"edtracker\"\\n\\nCORE_DIR = PKG_DIR / \"core\"\\nTRACKER_CORE_DST =
CORE_DIR / \"tracker_core.py\"\\n\\nINPUT_TRACKER = Path(\"/kaggle/
input/tracker-core/tracker_core.py\")\\nOPS_INPUT = Path(\"/
kaggle/input/edtracker-ops-pg/edtracker_ops_pkg\")\\nUI_INPUT =
Path(\"/kaggle/input/edtracker-ui-pkg/edtracker_ui_pkg\")\\n\\n# 1)
Ensure package layout (no overwrite)\\nPKG_ROOT.mkdir(parents=True,
exist_ok=True)\\nPKG_DIR.mkdir(parents=True, exist_ok=True)
\\nCORE_DIR.mkdir(parents=True, exist_ok=True)\\nfor init in
(PKG_DIR / \"__init__.py\", CORE_DIR / \"__init__.py\"):\\n    if not
init.exists():\\n        init.write_text(\"# pkg init\\n\\n\",
encoding=\"utf-8\")\\n\\nif not TRACKER_CORE_DST.exists() and
INPUT_TRACKER.exists():\\n    shutil.copy2(INPUT_TRACKER,
TRACKER_CORE_DST)\\n\\n# Optional: expose ops/ui via symlinks if
available (ignore if not permitted)\\ntry:\\n    if OPS_INPUT.exists()
and not (PKG_DIR / \"ops\").exists():\\n        (PKG_DIR /
\"ops\").symlink_to(OPS_INPUT)\\nexcept Exception:\\n    pass\\ntry:\\n
if UI_INPUT.exists() and not (PKG_DIR / \"ui\").exists():\\n
(PKG_DIR / \"ui\").symlink_to(UI_INPUT)\\nexcept Exception:\\n
pass\\n\\n# 2) Ensure editable pkg root is on sys.path first\\nroot_s =
str(PKG_ROOT)\\nif sys.path[0] != root_s:\\n    if root_s in sys.path:
\\n        sys.path.remove(root_s)\\n    sys.path.insert(0, root_s)
\\n\\n# 3) Import canonical module, then bind legacy alias to the same
module object\\npkg_tc =
importlib.import_module(\"edtracker.core.tracker_core\")
\\nsys.modules[\"tracker_core\"] = pkg_tc # legacy import path stays
valid\\n\\n# 4) Prove alias == package and guard against alias
drift\\nimport tracker_core as tc\\nassert inspect.getfile(tc) ==
inspect.getfile(pkg_tc), \"Alias drift: tracker_core !=
edtracker.core.tracker_core\"\\nprint(\"ALIAS_OK:\", tc is pkg_tc)
\\nprint(\"MODULE_FILE:\", inspect.getfile(pkg_tc))\\n\n,\"metadata\":
{\"trusted\":true,\"execution\":
{\"iopub.status.busy\":\"2025-08-25T18:40:38.381985Z\",\"iopub.execute_in
put\":\"2025-08-25T18:40:38.382760Z\",\"iopub.status.idle\":\"2025-08-25T1
8:40:38.396003Z\",\"shell.execute_reply.started\":\"2025-08-25T18:40:38.

```

[illegible]

```

def update_state_from_event(self, event: Dict[str, Any]):\n            #
naive: merge event into state; track backlog\n
self.state.update(event)\n
self.backlogs.setdefault("events", []).append(event)\n\n    def
apply_event_log(self, events: List[Dict[str, Any]]):\n            for ev
in events:\n                self.update_state_from_event(ev)
\n", "metadata": {"execution":
{"iopub.status.busy": "2025-08-25T18:40:38.444815Z", "iopub.execute_in
put": "2025-08-25T18:40:38.445040Z", "iopub.status.idle": "2025-08-25T1
8:40:38.464436Z", "shell.execute_reply.started": "2025-08-25T18:40:38.
445022Z", "shell.execute_reply": "2025-08-25T18:40:38.463399Z"}, "paper
mill":
{"duration": 1.878974, "end_time": "2025-08-24T12:38:53.061552", "except
ion": false, "start_time": "2025-08-24T12:38:51.182578", "status": "compl
eted"}, "tags": [], "trusted": true}, "outputs": [], "execution_count": 57},
{"cell_type": "code", "source": "\n# TinyCritics uses
WorkflowState.feature_dict(); cold-start safe (no transform before
fit)\nimport numpy as np\n\nclass TinyCritics:\n    def
__init__(self):\n        self._fitted = False\n\n    def fit(self,
states, actions, rewards):\n        # No-op fit to keep cold-start
safe\n        self._fitted = True\n        return self\n\n    def
score(self, state: WorkflowState, actions: List[Dict[str, str]]):\n
fd = state.feature_dict()\n        n = len(actions)\n        #
Deterministic, bounded scores in [0,1]\n        base = 0.5\n
p = np.full(n, base, dtype=float)\n        bonuses = np.zeros(n,
dtype=float)\n        uncertainty = np.full(n, 0.1, dtype=float)\n
return p, bonuses, uncertainty\n", "metadata": {"execution":
{"iopub.status.busy": "2025-08-25T18:40:38.465435Z", "iopub.execute_in
put": "2025-08-25T18:40:38.465795Z", "iopub.status.idle": "2025-08-25T1
8:40:38.490997Z", "shell.execute_reply.started": "2025-08-25T18:40:38.
465762Z", "shell.execute_reply": "2025-08-25T18:40:38.489973Z"}, "paper
mill":
{"duration": 0.01876, "end_time": "2025-08-24T12:38:53.090477", "excepti
on": false, "start_time": "2025-08-24T12:38:53.071717", "status": "comple
ted"}, "tags": [], "trusted": true}, "outputs": [], "execution_count": 58},
{"cell_type": "code", "source": "from dataclasses import
dataclass\nfrom typing import Any, Dict, List, Optional\nfrom
pathlib import Path\nimport pandas as pd, numpy as np\ndef
_cfg(CONFIG: Any, key: str, default: Any=None) -> Any:\n    try:
return CONFIG.get(key, default)\n    except Exception: return
getattr(CONFIG, key, default) if hasattr(CONFIG, key) else
default\ndef _ensure_parent(p: Path): p = Path(p);\n
p.parent.mkdir(parents=True, exist_ok=True)\n@dataclass\nclass
EquipmentRecord:\n    equip_id: str; name: str = \"\"; location:
str = \"\"; status: str = \"\"; last_seen: Optional[str]=None; battery:
Optional[float]=None; confidence: Optional[float]=None\n    def
to_row(self)->Dict[str, Any]: return {\"equip_id\": self.equip_id,
\"name\": self.name, \"location\": self.location,
\"status\": self.status, \"last_seen\": self.last_seen,
\"battery\": self.battery, \"confidence\": self.confidence}\n\nclass
EquipmentRepository:\n    def __init__(self, status_csv: Path):\n
self.status_csv = Path(status_csv); _ensure_parent(self.status_csv)\n
if not self.status_csv.exists(): pd.DataFrame(columns=[\"equip_id\",
\"name\", \"location\", \"status\", \"last_seen\", \"battery\",

```

```

\"confidence\"]).to_csv(self.status_csv, index=False)\n    def
read(self)->pd.DataFrame:\n        try:
df=pd.read_csv(self.status_csv);\n            except Exception: return
pd.DataFrame(columns=[\"equip_id\", \"name\", \"location\", \"status\",
\"last_seen\", \"battery\", \"confidence\"])\n                if \"equip_id\"
in df.columns: df[\"equip_id\"] = df[\"equip_id\"].astype(str); return
df\n        def upsert(self, rec: EquipmentRecord)->None:\n
df=self.read(); row=pd.DataFrame([rec.to_row()])\n            if
df.empty: df=row\n                else:\n
mask=(df[\"equip_id\"].astype(str)==str(rec.equip_id))\n
if mask.any(): df.loc[mask,:]=row.values\n                    else:
df=pd.concat([df,row], ignore_index=True)\n
df.to_csv(self.status_csv, index=False)\n\nclass MovesLogRepository:\n
def __init__(self, moves_csv: Path):\n
self.moves_csv=Path(moves_csv); _ensure_parent(self.moves_csv)\n
if not self.moves_csv.exists(): pd.DataFrame(columns=[\"equip_id\",
\"from\", \"to\", \"ts\"]).to_csv(self.moves_csv, index=False)\n
def append(self, equip_id:str, loc_from:str, loc_to:str,
ts_iso:str)->None:\n
row=pd.DataFrame([{\"equip_id\":equip_id, \"from\":loc_from,
\"to\":loc_to, \"ts\":ts_iso}])\n        try:
prev=pd.read_csv(self.moves_csv) if self.moves_csv.exists() else
None; df=pd.concat([prev,row], ignore_index=True) if prev is not
None else row\n            except Exception: df=row\n
df.to_csv(self.moves_csv, index=False)\n        def read(self)-
>pd.DataFrame:\n            try: return pd.read_csv(self.moves_csv)\n
except Exception: return pd.DataFrame(columns=[\"equip_id\",
\"from\", \"to\", \"ts\"])\n\nclass SOPRegistry:\n    def __init__(self,
sop_csv: Path): self.sop_csv=Path(sop_csv);
_ensure_parent(self.sop_csv)\n        def read(self)->pd.DataFrame:\n
if self.sop_csv.exists():\n            try:\n
df=pd.read_csv(self.sop_csv)\n                for col in
[\"sop_id\", \"title\", \"pdf_path\"]:\n                    if col not
in df.columns: df[col]=\"\"\n                        return df\n
except Exception: pass\n                    return
pd.DataFrame(columns=[\"sop_id\", \"title\", \"pdf_path\", \"version\",
\"status\", \"keywords\", \"checklist\", \"source_url\"])\n\nclass
QRService:\n    def __init__(self, out_dir: Path): \n
self.out_dir=Path(out_dir); self.out_dir.mkdir(parents=True,
exist_ok=True)\n        def make(self, payload:str)->str:\n            try:\n
import qrcode\n                fp=self.out_dir/
f\"qr_{abs(hash(payload))}.png\"\n
img=qrcode.make(payload); img.save(fp); return str(fp)\n
except Exception: return f\"[QR fallback] {payload}\"\n        def
decode_file(self, image_bytes:bytes):\n            try:\n
from PIL import Image; import io\n
img=Image.open(io.BytesIO(image_bytes))\n                try:\n
from pyzbar.pyzbar import decode as zbar_decode\n
res=zbar_decode(img);\n                    if res: return
res[0].data.decode(\"utf-8\", \"ignore\")\n                        except
Exception: pass\n                            except Exception: pass\n                                return
None\n\nclass TrackerService:\n    def __init__(self,
equipment_repo:EquipmentRepository, moves_repo:MovesLogRepository,
sop_registry:SOPRegistry, qr:QRService, config:Any):\n

```

```

self.equipment_repo=equipment_repo; self.moves_repo=moves_repo;
self.sop_registry=sop_registry; self.qr=qr; self.CONFIG=config\n
@classmethod\n    def from_config(cls, CONFIG:Any)-
>\TrackerService\":\n        return
cls(EquipmentRepository(Path(_cfg(CONFIG,
\EQUIPMENT_STATUS_PATH\))),\n
MovesLogRepository(Path(_cfg(CONFIG,\EQUIPMENT_MOVES_LOG_PATH\))),
\n
SOPRegistry(Path(_cfg(CONFIG,
\SOP_REGISTRY_PATH\))),\n
QRService(Path(_cfg(CONFIG,\QR_OUTPUT_DIR\))), CONFIG)\n    def
equipment_status(self)->pd.DataFrame: return
self.equipment_repo.read()\n    def log_move(self, equip_id:str,
loc_from:str, loc_to:str)->None:\n
ts_iso=pd.Timestamp.utcnow().isoformat();
df=self.equipment_repo.read()\n
row=df[df[\"equip_id\"].astype(str)==str(equip_id)];
name=row[\"name\"].iloc[0] if not row.empty and \"name\" in
row.columns else \"\"\n
rec=EquipmentRecord(equip_id=equip_id,name=name,location=loc_to,stat
us=\"moved\",last_seen=ts_iso)\n
self.equipment_repo.upsert(rec); self.moves_repo.append(equip_id,
loc_from or \"\", loc_to, ts_iso)\n    def find_equipment(self,
query:str)->pd.DataFrame:\n        q=(query or
\"\\\").strip().lower(); df=self.equipment_repo.read()\n        if not
q: return df\n        def hit(r): return any(q in str(r.get(k,
\"\\\")).lower() for k in [\"equip_id\", \"name\", \"location\",
\"status\"])\n        return df[df.apply(hit, axis=1)]\n    def
overdue_equipment(self, threshold_minutes:int=120)->pd.DataFrame:\n
df=self.equipment_repo.read().copy()\n        if df.empty or
\"last_seen\" not in df.columns: return df.iloc[0:0]\n
ts=pd.to_datetime(df[\"last_seen\"],errors=\"coerce\",utc=True);
age_min=(pd.Timestamp.utcnow().tz_localize(\"UTC\")-
ts).dt.total_seconds()/60.0\n        df[\"age_min\"] = age_min; return
df[age_min>float(threshold_minutes)].sort_values(\"age_min\",
ascending=False)\n    def movement_stats(self)-
>Dict[str,pd.DataFrame]:\n        log=self.moves_repo.read()\n
if log.empty: return {\"moves_per_equipment\":log,\"routes\":log}\n
per_eq=log.groupby(\"equip_id\").size().reset_index(name=\"moves\").
sort_values(\"moves\", ascending=False)\n
routes=log.groupby([\"from\",
\"to\"]).size().reset_index(name=\"count\").sort_values(\"count\",
ascending=False)\n        return {\"moves_per_equipment\":per_eq,
\"routes\":routes}\n    def sop_table(self)->pd.DataFrame: return
self.sop_registry.read()\n    def search_sop(self, query:str)-
>pd.DataFrame:\n        df=self.sop_registry.read().copy(); q=(query
or \"\\\").strip().lower()\n        if df.empty or not q: return df\n
cols=[c for c in [\"sop_id\", \"title\", \"keywords\", \"version\",
\"status\"] if c in df.columns]\n
mask=df[cols].astype(str).apply(lambda col:
col.str.lower().str.contains(q, na=False)).any(axis=1)\n
return df[mask]\n    def make_qr(self,payload:str)->str: return
self.qr.make(payload)\n    def decode_qr_bytes(self,
image_bytes:bytes): return self.qr.decode_file(image_bytes)
\nprint(\"tracker core ready\"),\"metadata\":{\"execution\":

```

```
{"iopub.status.busy":"2025-08-25T18:40:38.492272Z","iopub.execute_in
put":"2025-08-25T18:40:38.493090Z","iopub.status.idle":"2025-08-25T1
8:40:38.530896Z","shell.execute_reply.started":"2025-08-25T18:40:38.
493063Z","shell.execute_reply":"2025-08-25T18:40:38.529945Z"},"paper
mill":
```

```
{"duration":0.110152,"end_time":"2025-08-24T12:38:53.210680","except
ion":false,"start_time":"2025-08-24T12:38:53.100528","status":"compl
eted"},"tags":[],"trusted":true},"outputs":
```

```
[{"name":"stdout","text":"tracker core
ready\n","output_type":"stream"}],"execution_count":59},
{"cell_type":"code","source":"# --- tracker_core alias shim (append-
only, no renames) ---\nimport sys, types\n_names =
[\"TrackerService\", \"QRService\", \"EquipmentRepository\",
\"MovesLogRepository\", \"SOPRegistry\"]\nif \"tracker_core\" not in
sys.modules and all(n in globals() for n in _names):\n    _m =
types.ModuleType(\"tracker_core\")\n    for n in _names:\n
setattr(_m, n, globals()[n])\n    sys.modules[\"tracker_core\"] =
_m\n    print(\"tracker_core alias ready\")\nelse:\n
print(\"tracker_core alias already present or classes missing\")
\n","metadata":{"execution":
```

```
{"iopub.status.busy":"2025-08-25T18:40:38.531950Z","iopub.execute_in
put":"2025-08-25T18:40:38.532288Z","iopub.status.idle":"2025-08-25T1
8:40:38.552906Z","shell.execute_reply.started":"2025-08-25T18:40:38.
532265Z","shell.execute_reply":"2025-08-25T18:40:38.552037Z"},"paper
mill":
```

```
{"duration":0.019937,"end_time":"2025-08-24T12:38:53.240903","except
ion":false,"start_time":"2025-08-24T12:38:53.220966","status":"compl
eted"},"tags":[],"trusted":true},"outputs":
```

```
[{"name":"stdout","text":"tracker_core alias already present or
classes missing\n","output_type":"stream"}],"execution_count":60},
{"cell_type":"code","source":"# --- Patch: make
EquipmentRepository.read() robust ---\nimport pandas as pd, numpy as
np\n\n_EQUIP_COLS = [\"equip_id\", \"name\", \"location\", \"status\",
\"last_seen\", \"battery\", \"confidence\"]\n\nndef
 equiprepo_read_fixed(self):\n    try:\n        df =
pd.read_csv(self.status_csv)\n    except Exception:\n        #
brand-new or unreadable file → start with empty, correct schema\n
return pd.DataFrame(columns=_EQUIP_COLS)\n\n    # ensure required
columns exist\n    for col in _EQUIP_COLS:\n        if col not in
df.columns:\n            df[col] = np.nan\n\n    # normalize types/
order and return ALWAYS a DataFrame\n    df[\"equip_id\"] =
df[\"equip_id\"].astype(str)\n    return df[_EQUIP_COLS]\n\n# apply
patch without renaming the class\nEquipmentRepository.read =
 equiprepo_read_fixed\nprint(\"Patched EquipmentRepository.read
(robust return)\")\n","metadata":{"execution":
```

```
{"iopub.status.busy":"2025-08-25T18:40:38.555982Z","iopub.execute_in
put":"2025-08-25T18:40:38.556249Z","iopub.status.idle":"2025-08-25T1
8:40:38.575203Z","shell.execute_reply.started":"2025-08-25T18:40:38.
556227Z","shell.execute_reply":"2025-08-25T18:40:38.574298Z"},"paper
mill":
```

```
{"duration":0.019522,"end_time":"2025-08-24T12:38:53.271360","except
ion":false,"start_time":"2025-08-24T12:38:53.251838","status":"compl
eted"},"tags":[],"trusted":true},"outputs":
```

```
[{"name":"stdout","text":"Patched EquipmentRepository.read (robust
```

```

return)\n", "output_type": "stream"}], "execution_count": 61},
{"cell_type": "code", "source": "from pathlib import Path\nfrom typing\nimport Any, Dict, List\ndef _slugify(text:str)->str:\n    import re;\ns=re.sub(r\"[^\n-a-zA-Z0-9]+\", \"\", text.strip().lower()).strip(\"-\n\"); return s or \"sop\"\ndef load_priority_flows(json_path:str)-\n>Dict[str,Any]:\n    import json\n    try:\n        with\nopen(json_path,\"r\",encoding=\"utf-8\") as f: return json.load(f)\nexcept Exception: return {}\nprint(\"sop auto ready\")\n\n", "metadata": {"execution":\n{"iopub.status.busy": "2025-08-25T18:40:38.576090Z", "iopub.execute_in\nput": "2025-08-25T18:40:38.576331Z", "iopub.status.idle": "2025-08-25T1\n8:40:38.594529Z", "shell.execute_reply.started": "2025-08-25T18:40:38.\n576312Z", "shell.execute_reply": "2025-08-25T18:40:38.593505Z"}, "paper\nmill":\n{"duration": 0.031258, "end_time": "2025-08-24T12:38:53.313146", "except\nion": false, "start_time": "2025-08-24T12:38:53.281888", "status": "compl\neted"}, "tags": [], "trusted": true}, "outputs":\n[{"name": "stdout", "text": "sop auto\nready\n", "output_type": "stream"}], "execution_count": 62},\n{"cell_type": "code", "source": "def rule_hs_tnt(value):\n    try: v =\nfloat(value)\n    except Exception: return 0.50\n    if v < 14:\nreturn 0.50\n    if 14 <= v <= 51: return 0.20\n    return\n0.50\nassert rule_hs_tnt(13.9)==0.50 and rule_hs_tnt(14.0)==0.20 and\nrule_hs_tnt(51.0)==0.20 and\nrule_hs_tnt(51.1)==0.50\nprint(\"troponin_rules ok\")", "metadata":\n{"papermill":\n{"duration": 0.018798, "end_time": "2025-08-24T12:38:53.343560", "except\nion": false, "start_time": "2025-08-24T12:38:53.324762", "status": "compl\neted"}, "tags": [], "trusted": true, "execution":\n{"iopub.status.busy": "2025-08-25T18:40:38.595800Z", "iopub.execute_in\nput": "2025-08-25T18:40:38.596173Z", "iopub.status.idle": "2025-08-25T1\n8:40:38.613755Z", "shell.execute_reply.started": "2025-08-25T18:40:38.\n596142Z", "shell.execute_reply": "2025-08-25T18:40:38.612615Z"}}, "outp\nuts": [{"name": "stdout", "text": "troponin_rules\nok\n", "output_type": "stream"}], "execution_count": 63},\n{"cell_type": "code", "source": "\n# Inline SOP surface and optional UI\nwithout external modules\nfrom typing import Optional, Dict,\nAny\nimport csv, os\nfrom pathlib import Path\n\n# Optional,\nguarded UI demo (equipment status preview uses the CSV directly)\nif\nCONFIG.get(\"RUN_UI\"):\n    try:\n        import ipywidgets as W,\npandas as pd\n        eq_path =\nPath(CONFIG[\"EQUIPMENT_STATUS_PATH\"])\n        if not\n        eq_path.exists():\n            eq_path.write_text(\"equipment_id,location,last_seen\\n\\n\")\n            btn = W.Button(description=\"Show equipment status\")\n            out =\n            W.Output()\n            def _on_click():\n                with out:\n                    out.clear_output()\n                    try:\n                        df =\n                        pd.read_csv(eq_path)\n                    except Exception:\n                        df = pd.DataFrame(columns=[\"equipment_id\", \"location\", \"\nlast_seen\"])\n                    display(df)\n            btn.on_click(_on_click)\n            display(W.VBox([btn, out]))\n        except Exception as e:\n            print(\"UI unavailable (optional):\n\", e)\n", "metadata": {"execution":\n{"iopub.status.busy": "2025-08-25T18:40:38.614719Z", "iopub.execute_in

```

```
put":"2025-08-25T18:40:38.614939Z","iopub.status.idle":"2025-08-25T18:40:38.648445Z","shell.execute_reply.started":"2025-08-25T18:40:38.614921Z","shell.execute_reply":"2025-08-25T18:40:38.647210Z"},"papermill":
```

```
{"duration":0.177118,"end_time":"2025-08-24T12:38:53.531092","exception":false,"start_time":"2025-08-24T12:38:53.353974","status":"completed"},"tags":[],"trusted":true},"outputs":
```

```
[{"output_type":"display_data","data":{"text/plain":"VBox(children=(Button(description='Show equipment status', style=ButtonStyle()), Output()))","application/vnd.jupyter.widget-view+json":
```

```
{"version_major":2,"version_minor":0,"model_id":"30fd912ab3e34c3191d6c5c27de3dc26"}}, {"metadata":{}}, {"execution_count":64},
```

```
{"cell_type":"code","source":"# --- UI helpers (split out; no side effects) ---\n\ndef run_ui_header_controls():\n    import streamlit as st\n    c0, c1, c2, c3 = st.columns([2,2,2,2])\n    with c0:\n        thresh = st.number_input(\"0Overdue threshold (min)\", min_value=5, max_value=720, value=120, step=5)\n    with c1:\n        # NEW: adjustable vitals threshold (replaces hard-coded 120)\n        vitals_thresh = st.number_input(\"Vitals overdue (min)\", min_value=5, max_value=720, value=120, step=5)\n    with c2:\n        if st.button(\"Refresh\"):\n            st.experimental_rerun()\n    return thresh, vitals_thresh\n\ndef run_ui_equipment_panel(tracker, overdue_thresh_min: float):\n    import streamlit as st, pandas as pd\n    st.header(\"Equipment\")\n    eq_df = tracker.equipment_status()\n    s1, s2 = st.columns([2,1])\n    with s1:\n        q = st.text_input(\"Find equipment (ID / name / location / status)\", \"\")\n        filt = tracker.find_equipment(q) if q else eq_df\n    st.dataframe(filt, use_container_width=True, height=260)\n    with s2:\n        overdue = tracker.overdue_equipment(int(overdue_thresh_min))\n    st.subheader(\"0Overdue\")\n    if overdue.empty:\n        st.write(\"None\")\n    else:\n        st.dataframe(overdue[[\"equip_id\", \"name\", \"location\", \"last_seen\", \"age_min\"]],\n            use_container_width=True, height=200)\n    st.markdown(\"**Update location / log move**\")\n    mc1, mc2, mc3, mc4 = st.columns([2,2,2,1])\n    with mc1:\n        sel_id = st.selectbox(\"Equipment ID\", [\"\"] + sorted(list(eq_df.get(\"equip_id\", []))))\n    with mc2:\n        loc_from = st.text_input(\"From\", \"\")\n    with mc3:\n        loc_to = st.text_input(\"To\", \"\")\n    with mc4:\n        if st.button(\"Log move\") and sel_id and loc_to:\n            tracker.log_move(sel_id, loc_from, loc_to)\n    st.success(f\"Move logged: {sel_id} → {loc_to}\")\n\ndef run_ui_qr_panel(tracker):\n    import streamlit as st\n    st.header(\"QR\")\n    qr_col1, qr_col2 = st.columns([2,2])\n    with qr_col1:\n        qr_txt = st.text_input(\"QR payload to generate\", \"\")\n        if st.button(\"Generate QR\") and qr_txt:\n            path = tracker.make_qr(qr_txt)\n    st.write(\"QR saved to:\", path)\n    with qr_col2:\n        st.write(\"Scan and update location\")\n        f = st.file_uploader(\"Upload QR image\", type=[\"png\", \"jpg\"],
```

```

"\jpeg\","\webp\"])\n
manual_payload = st.text_input("\Manual
payload (fallback if decoding fails)\","\")\n
new_loc =
st.text_input("\New location (after scan)\","\")\n
if
st.button("\Scan & Update\"): \n
equip_payload = None\n
if f is not None: \n
equip_payload =
tracker.decode_qr_bytes(f.read())\n
if not equip_payload
and manual_payload: \n
equip_payload =
manual_payload\n
if equip_payload and new_loc: \n
equip_id = equip_payload\n
if "\id=" in
equip_payload: \n
try: \n
equip_id = equip_payload.split("\id=",1)[1].split("\&",1)[0]\n
except Exception: \n
equip_id =
equip_payload\n
tracker.log_move(str(equip_id), "\",
new_loc)\n
st.success(f"\Updated via payload.
{equip_id} → {new_loc}\")\n
elif not new_loc: \n
st.error("\Provide a new location.\")\n
else: \n
st.error("\No QR payload detected (image or manual).\")\n\n
ndef
run_ui_sop_panel(): \n
import streamlit as st\n
with
st.expander("\SOP auto-pull and flows", expanded=False): \n
if st.button("\Refresh SOPs from sop-notaufnahme.de\"): \n
# assumes refresh_sop_registry(CONFIG, ...) exists in scope\n
res = refresh_sop_registry(CONFIG, base_url="\https://sop-
notaufnahme.de/sop/\")\n
st.write(res)\n
flows =
load_priority_flows("\mnt/data/priority_flows.json")\n
if
flows: \n
keys = sorted(list(flows.keys()))\n
pickf = st.selectbox("\Show flow", [\"] + keys)\n
if
pickf: \n
flow = flows[pickf]\n
st.subheader(flow.get("\title", pickf))\n
nodes =
flow.get("\nodes", []); edges = flow.get("\edges", [])\n
st.write("\Nodes: \", \", \".join([n.get("\label", n.get("\id",
\"])") for n in nodes]))\n
try: \n
import matplotlib.pyplot as plt\n
fig =
plt.figure()\n
pos = {n["id"]:(i, 0) for i,n in
enumerate(nodes)}\n
for n in nodes: \n
x,y = pos[n["id"]]; plt.scatter([x],[y])\n
plt.text(x,y+0.05,n.get("\label", n["id"]), ha="\center",
rotation=45)\n
for a,b in edges: \n
xa,ya = pos.get(a,(0,0)); xb,yb = pos.get(b,(0,0))\n
plt.plot([xa,xb],[ya,yb])\n
plt.axis("\off");
plt.title(flow.get("\title", pickf))\n
st.pyplot(fig)\n
except Exception: \n
st.info("\Graph display unavailable; showing list instead.\")\n
st.write(edges)\n\n
ndef run_ui_actions_and_critic(get_state,
get_actions, critic, vitals_thresh_min: float): \n
import
streamlit as st, pandas as pd, numpy as np\n
st.header("\SOPs")
\n
# assumes `tracker` in wrapper handles search UI; keeping your
SOP table UI below\n
# (if you want SOP search to stay here, we
can move it – keeping behavior same is safer)\n
st.header("\Actions & Critic")\n
state = get_state()\n
if
hasattr(state, "\feature_dict"): \n
feats =
state.feature_dict()\n
since_v =
feats.get("\since_vitals_min", None)\n
if since_v is not
None: \n
# NEW: use adjustable threshold instead of hard-
coded 120\n
if since_v > float(vitals_thresh_min): \n

```

```

st.error(f"Lingering patient: since_vitals_min={since_v:.0f} >
{int(vitals_thresh_min)})")\n      else:\n
st.success(f"Vitals recently checked: {since_v:.0f} min (<=
{int(vitals_thresh_min)})")\n      if st.button("Mark vitals now")
and hasattr(state, "touch_now"):\n
state.touch_now(pd.Timestamp.utcnow())\n      st.success("Vitals
timestamp updated.")\n      actions = get_actions(state)\n      if not
actions:\n          st.info("No actions available.")\n
return\n      p, benefit, burden = critic.score(state, actions)\n
view = pd.DataFrame({\n          "id": [a.get("id") for a in
actions],\n          "label": [a.get("label") for a in actions],\n
"p_accept": np.round(p,3),\n
"benefit": np.round(benefit,3),\n
"burden": np.round(burden,3)\n      }).sort_values(["p_accept",
"benefit"], ascending=[False, False])\n      st.dataframe(view,
use_container_width=True, height=240)\n\ndef
run_ui_movement_analytics(tracker):\n      import streamlit as st\n
st.header("Equipment Movement Analytics")\n      stats =
tracker.movement_stats()\n      per_eq =
stats["moves_per_equipment"]; routes = stats["routes"]\n      if
per_eq.empty:\n          st.info("No movement data yet.")\n      else:\n
\n          st.subheader("Moves per equipment")\n
st.dataframe(per_eq, use_container_width=True, height=240)\n
try:\n          import matplotlib.pyplot as plt\n          fig =
plt.figure()\n          x =
per_eq["equip_id"].astype(str).tolist()\n          y =
per_eq["moves"].tolist()\n          plt.bar(x,y);
plt.xticks(rotation=45, ha="right"); plt.title("Moves per
Equipment")\n          st.pyplot(fig)\n          except Exception:\n
\n          pass\n          st.subheader("Top routes")\n
st.dataframe(routes, use_container_width=True,
height=200)\n", "metadata": {"execution":
{"iopub.status.busy": "2025-08-25T18:40:38.649730Z", "iopub.execute_in
put": "2025-08-25T18:40:38.650073Z", "iopub.status.idle": "2025-08-25T1
8:40:38.679857Z", "shell.execute_reply.started": "2025-08-25T18:40:38.
650045Z", "shell.execute_reply": "2025-08-25T18:40:38.678851Z"}, "paper
mill":
{"duration": 0.039437, "end_time": "2025-08-24T12:38:53.581155", "except
ion": false, "start_time": "2025-08-24T12:38:53.541718", "status": "compl
eted"}, "tags": [], "trusted": true}, "outputs": [], "execution_count": 65},
{"cell_type": "code", "source": "# --- Wrapper: calls helpers; same
signature as before ---\n\ndef run_ui(tracker, get_state, get_actions,
critic):\n      import streamlit as st, pandas as pd, numpy as np\n
st.set_page_config(page_title="ED Tracker - Full",
layout="wide")\n      st.title("ED Tracker - Core Ops (Full)")\n\n
# Header controls (now returns both thresholds)\n      overdue_thresh,
vitals_thresh = run_ui_header_controls()\n\n      # Panels\n
run_ui_equipment_panel(tracker, overdue_thresh)\n
run_ui_qr_panel(tracker)\n      run_ui_sop_panel()\n\n      # SOP search
table (kept here to mirror your original flow)\n      sop_q =
st.text_input("Search SOPs (id/title/keywords)", "")\n
sop_hits = tracker.search_sop(sop_q)\n      if sop_hits.empty:\n
st.info("No SOPs found.")\n      else:\n
st.dataframe(sop_hits[["sop_id", "title", "version"],

```

```

\"status\\\"]], use_container_width=True, height=220)\n            pick =
st.selectbox(\"Open SOP\", [\"\\\"] +
sop_hits[\"sop_id\\\"].astype(str).tolist())\n            if pick:\n
row = sop_hits[sop_hits[\"sop_id\\\"].astype(str)==pick].iloc[0]\n
pdf = row.get(\"pdf_path\\\", \"\\\")\n            if pdf:\n
st.write(\"PDF path:\\\", pdf)\n            if \"checklist\" in
sop_hits.columns and isinstance(row.get(\"checklist\\\", None), str)
and row[\"checklist\\\"].strip():\n
st.subheader(\"Checklist\\\")\n                steps = [s.strip() for
s in row[\"checklist\\\"].split(\"|\")] if s.strip()\n
completed = []\n                for i, step in enumerate(steps, 1):\n
\n                    if st.checkbox(f\"{i}. {step}\",
key=f\"sop_{pick}_{i}\"):
completed.append(i)\n                    st.caption(f\"Completed
{len(completed)}/{len(steps)} steps\\\")\n\n            # Actions, critic, and
analytics (uses adjustable vitals threshold)\n
run_ui_actions_and_critic(get_state, get_actions, critic,
vitals_thresh)\n            run_ui_movement_analytics(tracker)\n
print(\"ui ready\\\")\n\", \"metadata\": {\"execution\":
{\"iopub.status.busy\": \"2025-08-25T18:40:38.681245Z\", \"iopub.execute_in
put\": \"2025-08-25T18:40:38.681621Z\", \"iopub.status.idle\": \"2025-08-25T1
8:40:38.706720Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:38.
681588Z\", \"shell.execute_reply\": \"2025-08-25T18:40:38.704966Z\"}, \"paper
mill\":
{\"duration\": 0.021702, \"end_time\": \"2025-08-24T12:38:53.613603\", \"except
ion\": false, \"start_time\": \"2025-08-24T12:38:53.591901\", \"status\": \"compl
eted\", \"tags\": [], \"trusted\": true}, \"outputs\": [], \"execution_count\": 66},
{\"cell_type\": \"code\", \"source\": \"def go(_):\n    with out:\n
out.clear_output()\n        print(\"Running...\\\")\n        try:\n
# Minimal smoke: compute Phase-2 bundle and log, then score a dummy
row with the loaded MLP\n                vitals =
{\\\"rr\\\":24,\\\"sbp\\\":95,\\\"gcs\\\":14,\\\"hr\\\":120,\\\"temp\\\":38.6,\\\"avpu\\\":
\\\"V\\\",\\\"map\\\":65,\\\"age\\\":68,\\\"sex\\\":\\\"M\\\",\\\"sao2\\\":97}\n
labs    = {\\\"d_dimer\\\":780,\\\"d_dimer_unit\\\":\\\"µg/L FEU\\\",
\\\"creatinine\\\":1.8, \\\"platelets\\\":95, \\\"albumin\\\":2.8}\n
ctx     = {\\\"suspected_infection\\\": True}\n                b =
phase2_bundle(vitals, labs, ctx)\n                print(\"\\
\n\\\".join(b[\\\"one_liners\\\"])))\n                if \"predict_one\" in
globals() and \"FEAT\" in globals():\n                    row = {f:
labs.get(f, vitals.get(f, ctx.get(f, 0.0))) for f in FEAT}\n
res = predict_one(row)\n                print(f\"ML risk
p={res['p']:.3f} → {'ALERT' if res['y'] else 'ok'}
(thr={res['thr']:.3f})\\\")\n                except Exception as e:\n
print(\"[ERROR]\", e)\n\", \"metadata\": {\"trusted\": true, \"execution\":
{\"iopub.status.busy\": \"2025-08-25T18:40:38.707516Z\", \"iopub.execute_in
put\": \"2025-08-25T18:40:38.707903Z\", \"iopub.status.idle\": \"2025-08-25T1
8:40:38.731931Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:38.
707875Z\", \"shell.execute_reply\": \"2025-08-25T18:40:38.730983Z\"}}, \"outp
uts\": [], \"execution_count\": 67}, {\"cell_type\": \"code\", \"source\": \"def
run_icu_constraints(state):\n    if not CONFIG.get(\"RUN_PIPELINE\\\",
False): return {}\n    mod = _try_import(\"icu_constraints\\\") or
_try_import(\"modeling_icu_constraints\\\")\n    if mod and
hasattr(mod, \"compute_icu_flags\\\"):\n        try: return
dict(mod.compute_icu_flags(state))\n        except Exception: return

```

```

{} \n    return {} \n \n def mesh_route_actions(state, actions): \n    if
not CONFIG.get(\\"RUN_PIPELINE\\", False): return actions \n    mod =
_try_import(\\"agent_mesh\\") or _try_import(\\"ed_agent_mesh\\") \n
if mod and hasattr(mod, \\"route\\"): \n        try: return
list(mod.route(state, actions)) \n        except Exception: return
actions \n    return actions \n \n def trainer_fit_critic(critic,
samples, y): \n    if not CONFIG.get(\\"RUN_PIPELINE\\", False): return
critic \n    mod = _try_import(\\"trainer\\") or
_try_import(\\"ed_trainer\\") \n    if mod and hasattr(mod,
\\"fit_critic\\"): \n        try: return mod.fit_critic(critic,
samples, y) \n        except Exception: return critic \n    return
critic \n", "metadata": {"trusted": true, "execution":
{"iopub.status.busy": "2025-08-25T18:40:38.733213Z", "iopub.execute_in
put": "2025-08-25T18:40:38.733710Z", "iopub.status.idle": "2025-08-25T1
8:40:38.757856Z", "shell.execute_reply.started": "2025-08-25T18:40:38.
733678Z", "shell.execute_reply": "2025-08-25T18:40:38.756898Z"}}, "outp
uts": [], "execution_count": 68}, {"cell_type": "code", "source": "# ----
Force-hydrate ML wrapper (idempotent) ---- \n import os, glob, numpy
as np, pandas as pd \n from joblib import load \n \n def
_resolve_bundle(): \n    p = CONFIG.get(\\"MODEL_BUNDLE_PATH\\") \n
if p and os.path.exists(p): return p \n    roots = [\\"/kaggle/
working\\", \\"/mnt/data\\", \\"/content\\"] \n    pats = [\\"**/
ed_phase2_model*patched(1).joblib\\", \\"**/*.joblib\\", \\"**/*.pkl\\"] \n
for r in roots: \n        for pat in pats: \n            for cand in
glob.glob(os.path.join(r, pat), recursive=True): \n                if
os.path.exists(cand): return cand \n            raise
FileNotFoundError(\\"Model bundle not found anywhere\\") \n \n # Always
(re)define predict_one, FEAT, THR from bundle \n BUNDLE =
_resolve_bundle() \n _b = load(BUNDLE) \n _pipe = _b.get(\\"pipeline\\") or
_b.get(\\"model\\") \n _cal = _b.get(\\"calibrator\\") # may be
None \n THR = float(_b.get(\\"threshold\\", 0.5)) \n FEAT =
_b.get(\\"features\\") # list of feature names \n \n def
_apply_cal(p1d: np.ndarray) -> np.ndarray: \n    try: return
cal.transform(np.asarray(p1d)) \n    except: return p1d \n \n def
score_proba(df: pd.DataFrame) -> np.ndarray: \n    X = df[FEAT] if
FEAT else df \n    p = pipe.predict_proba(X)[:, 1] \n    return
_apply_cal(p) if cal is not None else p \n \n def predict_one(row:
dict) -> dict: \n    p = float(score_proba(pd.DataFrame([row]))[0]) \n
return {\\"p\\": p, \\"y\\": int(p >= THR), \\"thr\\": THR} \n \n print(\\"ML
wrapper ready:\\", \n        \\"predict_one\\" in globals(), \n        \\"|
features:\\", len(FEAT) if FEAT is not None else None, \n        \\"|
bundle:\\", BUNDLE) \n", "metadata": {"trusted": true, "execution":
{"iopub.status.busy": "2025-08-25T18:40:38.758786Z", "iopub.execute_in
put": "2025-08-25T18:40:38.759056Z", "iopub.status.idle": "2025-08-25T1
8:40:38.785953Z", "shell.execute_reply.started": "2025-08-25T18:40:38.
759028Z", "shell.execute_reply": "2025-08-25T18:40:38.784649Z"}}, "outp
uts": [{"name": "stdout", "text": "ML wrapper ready: True | features: 16
| bundle: /kaggle/working/
ed_phase2_model_thr_patched(1).joblib \n", "output_type": "stream"},
{"name": "stderr", "text": "/usr/local/lib/python3.11/dist-packages/
sklearn/base.py:318: UserWarning: Trying to unpickle estimator
SimpleImputer from version 1.1.3 when using version 1.2.2. This
might lead to breaking code or invalid results. Use at your own
risk. For more info please refer to: \n https://scikit-learn.org/

```

```

stable/model_persistence.html#security-maintainability-limitations\n
warnings.warn(\n/usr/local/lib/python3.11/dist-packages/sklearn/
base.py:318: UserWarning: Trying to unpickle estimator
StandardScaler from version 1.1.3 when using version 1.2.2. This
might lead to breaking code or invalid results. Use at your own
risk. For more info please refer to:\nhttps://scikit-learn.org/
stable/model_persistence.html#security-maintainability-limitations\n
warnings.warn(\n/usr/local/lib/python3.11/dist-packages/sklearn/
base.py:318: UserWarning: Trying to unpickle estimator
LabelBinarizer from version 1.1.3 when using version 1.2.2. This
might lead to breaking code or invalid results. Use at your own
risk. For more info please refer to:\nhttps://scikit-learn.org/
stable/model_persistence.html#security-maintainability-limitations\n
warnings.warn(\n/usr/local/lib/python3.11/dist-packages/sklearn/
base.py:318: UserWarning: Trying to unpickle estimator MLPClassifier
from version 1.1.3 when using version 1.2.2. This might lead to
breaking code or invalid results. Use at your own risk. For more
info please refer to:\nhttps://scikit-learn.org/stable/
model_persistence.html#security-maintainability-limitations\n
warnings.warn(\n/usr/local/lib/python3.11/dist-packages/sklearn/
base.py:318: UserWarning: Trying to unpickle estimator Pipeline from
version 1.1.3 when using version 1.2.2. This might lead to breaking
code or invalid results. Use at your own risk. For more info please
refer to:\nhttps://scikit-learn.org/stable/
model_persistence.html#security-maintainability-limitations\n
warnings.warn(\n/usr/local/lib/python3.11/dist-packages/sklearn/
base.py:318: UserWarning: Trying to unpickle estimator
IsotonicRegression from version 1.1.3 when using version 1.2.2. This
might lead to breaking code or invalid results. Use at your own
risk. For more info please refer to:\nhttps://scikit-learn.org/
stable/model_persistence.html#security-maintainability-limitations\n
warnings.warn(\n", "output_type": "stream"}], "execution_count": 69},
{"cell_type": "code", "source": "# Inline UI trigger (compute from
current UI inputs, no fixtures)\nif CONFIG.get(\"RUN_UI\"):\n
try:\n
    import ipywidgets as W, json\n
    btn =
W.Button(description=\"Compute from UI inputs + log\",
button_style=\"primary\")\n
    out = W.Output()\n
    def
_go():\n
        with out:\n
            out.clear_output()\n
            print(\"Running on current UI inputs...\")\n
            try:\n
                # Use what's in the text areas right now\n
                vitals
= json.loads(vitals_in.value) if 'vitals_in' in globals() else {}\n
                labs
= json.loads(labs_in.value) if 'labs_in' in globals()
else {}\n
                ctx
= json.loads(ctx_in.value) if
'ctx_in' in globals() else {}\n
                bundle =
phase2_bundle(vitals, labs, ctx)\n
                print(\"
\n\".join(bundle[\"one_liners\"])\n
                # ML risk
if model is loaded\n
                    if \"predict_one\" in
globals() and \"FEAT\" in globals():\n
                        row =
{f: labs.get(f, vitals.get(f, ctx.get(f, 0.0))) for f in FEAT}\n
                        res = predict_one(row)\n
                        print(f\"ML risk
p={res['p']:.3f} → {'ALERT' if res['y'] else 'ok'}
(thr={res['thr']:.3f})\")\n
                        try:\n
                            _append_event({\"type\": \"ml_score_ui\", \"keys\": list(row.keys()),
                            \"res\": res})\n
                            except Exception:\n

```

```

pass\n                                else:\n                                print(\"(ML
bridge not loaded – run the bundle load cell)\")\n\n
_append_event({"type": "phase2_bundle_ui", "bundle": bundle})\n
print("\n\nLogged to:", CONFIG.get("EVENT_LOG_PATH"))\n
except Exception as e:\n                                print("[ERROR]", e)
\n\n    btn.on_click(_go)\n                                display(W.VBox([btn, out]))\n
except Exception as e:\n                                print("UI unavailable:", e)\nelse:
\n    print("UI panel deferred... set CONFIG['RUN_UI']=True.")
\n", "metadata": {"trusted": true, "execution":
{"iopub.status.busy": "2025-08-25T18:40:38.787008Z", "iopub.execute_in
put": "2025-08-25T18:40:38.787287Z", "iopub.status.idle": "2025-08-25T1
8:40:38.817201Z", "shell.execute_reply.started": "2025-08-25T18:40:38.
787265Z", "shell.execute_reply": "2025-08-25T18:40:38.816302Z"}}, "outp
uts": [{"output_type": "display_data", "data": {"text/
plain": "VBox(children=(Button(button_style='primary',
description='Compute from UI inputs + log',
style=ButtonStyle()), "application/vnd.jupyter.widget-view+json":
{"version_major": 2, "version_minor": 0, "model_id": "f85241eb4131470d9fc
ff33f3b32ecd1"}}, "metadata": {}}, {"execution_count": 70},
{"cell_type": "code", "source": "# 1) Make sure the pieces
exist\nprint("phase2_bundle:", "phase2_bundle" in globals())
\nprint("predict_one :", "predict_one" in globals(), "\n|
FEAT:", "FEAT" in globals())\n\n# 2) Click your new button once,
then tail the log:\nfrom pathlib import Path, PurePath\np =
Path(CONFIG["EVENT_LOG_PATH"])\nprint("Log exists:", p.exists(),
"\n→", str(p))\n\nif p.exists():\n    tail =
p.read_text().strip().splitlines()[-8:]\n    for ln in tail:\n
print(ln[:200])\n", "metadata": {"trusted": true, "execution":
{"iopub.status.busy": "2025-08-25T18:40:38.818522Z", "iopub.execute_in
put": "2025-08-25T18:40:38.818956Z", "iopub.status.idle": "2025-08-25T1
8:40:38.841513Z", "shell.execute_reply.started": "2025-08-25T18:40:38.
818931Z", "shell.execute_reply": "2025-08-25T18:40:38.840463Z"}}, "outp
uts": [{"name": "stdout", "text": "phase2_bundle: True\npredict_one :
True | FEAT: True\nLog exists: True → /kaggle/working/
event_log.json\n{"ts": "2025-08-25T18:11:30.050962Z", "type":
"preflight_ping", "ok": true}\n{"ts":
"2025-08-25T18:40:38.340871Z", "type": "preflight_ping",
"ok": true}\n", "output_type": "stream"}}, {"execution_count": 71},
{"cell_type": "code", "source": "# == MLP → OPS advice-only bridge
(merged, guarded, contract-compliant) ==\n\n# Runs only when
CONFIG["RUN_PIPELINE"] is True. Only CONFIG["MODEL_BUNDLE_PATH"]
may be set.\nimport os, glob, json\nfrom pathlib import Path\nimport
numpy as np, pandas as pd\n\nif CONFIG.get("RUN_PIPELINE", False):
\n    # --- 1) Locate model bundle (prefer explicit path, then
dataset, then glob) ---\n    MODEL_HINT =
CONFIG.get("MODEL_BUNDLE_PATH")\n    SEARCH_DIRS = [\n
(MODEL_HINT if MODEL_HINT else None),\n    "/kaggle/input/ed-
pipeline-bundle-ui",\n    "/kaggle/working",\n    "/mnt/
data",\n    "/content",\n    ]\n    candidates = []\n    for
root in [d for d in SEARCH_DIRS if d]:\n        p = Path(root)\n
if p.is_file() and str(p).endswith((".joblib", ".pkl")):\n
candidates.append(str(p))\n        elif p.exists():\n
candidates += [str(x) for x in
p.rglob("ed_phase2_model*_patched(1).joblib")]

```

```

candidates += [str(x) for x in p.rglob("\*.joblib\")]
candidates += [str(x) for x in p.rglob("\*.pkl\")]
BUNDLE = next((p for p in candidates if Path(p).exists()), None)
if not BUNDLE:
    raise FileNotFoundError("Model bundle not found. Set CONFIG['MODEL_BUNDLE_PATH'] or add dataset.\")
# Only allowed mutation per contract
CONFIG["MODEL_BUNDLE_PATH"] = BUNDLE
# --- 2) Load bundle (no side effects) ---
from joblib import load
b = load(BUNDLE)
pipe = b.get("pipeline") or b.get("model")
cal = b.get("calibrator", None) # may be None
THR = float(b.get("threshold", 0.5))
FEAT = b.get("features")
# list or None
if pipe is None:
    raise RuntimeError("pipeline missing in joblib bundle")
# --- 3) Build minimal legacy feature frame from MIMIC-IV-ED demo ---
MIMIC_BASE = Path("/kaggle/input/mimic-iv-demo-v2-2")
ed = pd.read_csv(MIMIC_BASE/"edstays.csv")
tri = pd.read_csv(MIMIC_BASE/"triage.csv")
# times
for c in ("intime", "outtime"):
    if c in ed.columns:
        ed[c] = pd.to_datetime(ed[c])
# triage shim → legacy names
keep = [c for c in ["stay_id", "acuity", "heartrate", "sbp", "dbp", "chiefcomplaint"] if c in tri.columns]
tri_small = tri[keep].drop_duplicates("stay_id")
tri_small["HF"] = tri_small.get("heartrate")
tri_small["MAP"] = np.nan
if {"sbp", "dbp"}.issubset(tri_small.columns):
    tri_small["MAP"] = tri_small["dbp"] + (tri_small["sbp"] - tri_small["dbp"]) / 3.0
    tri_small["Triage"] = pd.to_numeric(tri_small.get("acuity"), errors="coerce")
df = ed.merge(tri_small[["stay_id", "HF", "MAP", "Triage", "chiefcomplaint"]], on="stay_id", how="left")
# construct required feature order
feat_names = FEAT or ['Tag', 't_min', 'Triage', 'Leitsymptom', 'HF', 'MAP', 'ICU_Kap', 'Kap_veraltet', 't_norm', 'hat_Labor', 'Labor_ausstehend', 'hat_Roentgen', 'Roentgen_ausstehend', 'hat_CT', 'CT_ausstehend', 'naechste_Aktion']
X = pd.DataFrame(index=df.index, columns=feat_names, dtype="float32")
# populate available numerics
if "HF" in X.columns:
    X["HF"] = df["HF"].astype("float32")
if "MAP" in X.columns:
    X["MAP"] = df["MAP"].astype("float32")
if "Triage" in X.columns:
    X["Triage"] = df["Triage"].astype("float32")
# simple time features
if "Tag" in X.columns:
    X["Tag"] = (ed["intime"].dt.hour if "intime" in ed.columns else 0).fillna(0).astype("float32")
if "t_min" in X.columns:
    X["t_min"] = 0.0
if "t_norm" in X.columns:
    X["t_norm"] = 0.0
# defaults for others
for c in ["ICU_Kap", "Kap_veraltet", "hat_Labor", "Labor_ausstehend", "hat_Roentgen", "Roentgen_ausstehend", "hat_CT", "CT_ausstehend", "naechste_Aktion"]:
    if c in X.columns:
        X[c] = 0.0
if "Leitsymptom" in X.columns:
    # leave NaN to let the pipeline imputer handle; avoids free-text leakage
    X["Leitsymptom"] = np.nan
X = X.apply(pd.to_numeric, errors="coerce").astype("float32")
# --- 4) Score (advice-only), with calibrator fallback and tiny smoke ---
def

```

```

_score_proba(_X: pd.DataFrame) -> np.ndarray:\n        p =
pipe.predict_proba(_X)[:, 1]\n        # support sklearn
IsotonicRegression (predict) and calibrated wrappers (transform not
guaranteed)\n        if cal is None:\n            return p\n
return cal.predict(p) if hasattr(cal, \"predict\") else
(cal.transform(p) if hasattr(cal, \"transform\") else p)\n\n
p_raw = pipe.predict_proba(X)[:, 1]\n        try:\n            p_cal =
_score_proba(X)\n        except Exception:\n            p_cal = p_raw #
safe fallback\n        advice = (p_cal >= THR) if np.isfinite(THR) else
np.zeros_like(p_cal, dtype=bool)\n\n        out = pd.DataFrame({\n
\"stay_id\": df[\"stay_id\"],\n            \"p_gatepos_mlp_raw\": p_raw,\n
\n            \"p_gatepos_mlp_cal\": p_cal,\n
\n            \"advice_gatepos_bool\": advice, # advice-only; NOT wired to
gates\n        })\n        out_path = \"/kaggle/working/
advice_gatepos_mlp.csv\"\n        out.to_csv(out_path, index=False)\n\n
# --- 5) Join ops panel if present (non-blocking advice card) ---\n
panel_path = \"/kaggle/working/advice_ops_panel.csv\"\n        if
Path(panel_path).exists():\n            panel = pd.read_csv(panel_path)\n
\n            panel = panel.merge(out[[\"stay_id\",
\n            \"p_gatepos_mlp_cal\", \"advice_gatepos_bool\"]],\n
on=\"stay_id\", how=\"outer\")\n            panel.to_csv(panel_path,
index=False)\n\n        # --- 6) Tiny smoke (file exists; lengths match;
probs in [0,1]) ---\n        assert Path(out_path).exists(), \"advice
CSV not written\"\n        assert len(out) == len(df), \"row count
mismatch\"\n        assert
np.isfinite(out[\"p_gatepos_mlp_raw\"]).all(), \"NaNs in raw
probs\"\n        ok_cal = (out[\"p_gatepos_mlp_cal\"] >= 0 - 1e-9) &
(out[\"p_gatepos_mlp_cal\"] <= 1 + 1e-9)\n        assert ok_cal.all(),
\"calibrated probs out of [0,1]\"\n\n        print(f\"ML_BRIDGE_SMOKE_OK
| model={Path(BUNDLE).name} | rows={len(out)} | thr={THR}\")\n    else:
\n        print(\"ML bridge skipped (CONFIG['RUN_PIPELINE'] is False)\")
\n    ,\"metadata\":{\"execution\":
{\"iopub.status.busy\":\"2025-08-25T18:40:38.842607Z\", \"iopub.execute_in
put\":\"2025-08-25T18:40:38.842892Z\", \"iopub.status.idle\":\"2025-08-25T1
8:40:38.913150Z\", \"shell.execute_reply.started\":\"2025-08-25T18:40:38.
842866Z\", \"shell.execute_reply\":\"2025-08-25T18:40:38.911963Z\"}, \"paper
mill\":
{\"duration\":2.073242, \"end_time\":\"2025-08-24T12:38:55.794863\", \"except
ion\":false, \"start_time\":\"2025-08-24T12:38:53.721621\", \"status\":\"compl
eted\"}, \"tags\":[], \"trusted\":true}, \"outputs\":
[{\n        \"name\":\"stdout\", \"text\":\"ML_BRIDGE_SMOKE_OK |
model=ed_phase2_model_thr_patched(1).joblib | rows=222 |
thr=0.9988444286248084\\n\", \"output_type\":\"stream\"}], \"execution_count\"
:72}, {\n        \"cell_type\":\"code\", \"source\":\"# After the bridge cell\\ndef
score_and_log(row: dict):\n        \\\"\\\"\\\"row must have the 16 feature
keys in the bundle (FEAT).\\\"\\\"\\\"\\n        res = predict_one(row) # uses
THR/FEAT from the bridge\\n        try:\n            _append_event({\"type\":
\\\"ml_score\\\", \"res\": res, \"keys\": list(row.keys())})\n        except
Exception:\n            pass\\n        print(f\"ML risk p={res['p']:.3f} ->
{'ALERT' if res['y'] else 'ok'} (thr={res['thr']:.3f})\\\"\\\"\\\"\\n
return res\\n\", \"metadata\":{\"execution\":
{\"iopub.status.busy\":\"2025-08-25T18:40:38.914404Z\", \"iopub.execute_in
put\":\"2025-08-25T18:40:38.915154Z\", \"iopub.status.idle\":\"2025-08-25T1
8:40:38.920946Z\", \"shell.execute_reply.started\":\"2025-08-25T18:40:38.

```

915121Z", "shell.execute_reply": "2025-08-25T18:40:38.919807Z"}, "paper mill":

```
{"duration":0.018097,"end_time":"2025-08-24T12:38:55.823664","exception":false,"start_time":"2025-08-24T12:38:55.805567","status":"completed"},"tags":[],"trusted":true},"outputs":[],"execution_count":73}, {"cell_type":"code","source":"# ICU availability panel – replaces\n\"actionable vs blocked\" with explicit next-bed ETAs.\n\n# No sidecars; all inline; guarded by RUN_UI.\n\nfrom pathlib import Path\nfrom datetime import datetime, timezone\nimport json\n\n# Pre-seeded UAE ICUs (public info): names + capacities\nUAE_UNITS = [\n    {\"name\": \"1A Neurochirurgische Intensivstation\", \"capacity\": 12, \"occupied\": 12, \"discharge_eta_minutes\": []},\n    {\"name\": \"1B Neurologische Intensivstation\", \"capacity\": 12, \"occupied\": 12, \"discharge_eta_minutes\": []},\n    {\"name\": \"1C Interdisziplinäre Intensivstation\", \"capacity\": 12, \"occupied\": 12, \"discharge_eta_minutes\": []},\n    {\"name\": \"1D Interdisziplinäre Intensivstation\", \"capacity\": 12, \"occupied\": 12, \"discharge_eta_minutes\": []},\n    {\"name\": \"1E Interdisziplinäre Intensivstation\", \"capacity\": 12, \"occupied\": 12, \"discharge_eta_minutes\": []},\n    {\"name\": \"1F Operative Intensivstation\", \"capacity\": 12, \"occupied\": 12, \"discharge_eta_minutes\": []},\n    {\"name\": \"1G Internistische Intensivstation\", \"capacity\": 12, \"occupied\": 12, \"discharge_eta_minutes\": []},\n    {\"name\": \"H1b Kardiologische Intensivstation\", \"capacity\": 12, \"occupied\": 12, \"discharge_eta_minutes\": []},\n    {\"name\": \"H1b Kardiologische Intensivstation\", \"capacity\": 12, \"occupied\": 12, \"discharge_eta_minutes\": []},\n    {\"name\": \"H2b Intensivstation Gefäß- und Herzmedizin\", \"capacity\": 8, \"occupied\": 8, \"discharge_eta_minutes\": []},\n]\n\ndef _format_eta(mins: int) -> str:\n    if mins is None: return \"unknown\"\n    if mins <= 0: return \"now\" + h, m = divmod(int(mins), 60)\n    return f\"{m} min\" if h == 0 else f\"{h} hr\" if m == 0 else f\"{h} hr {m} min\"\n\ndef _load_icu_status(path=\"/mnt/data/icu_status.json\"):\n    p = Path(path)\n    if p.exists():\n        try:\n            js = json.loads(p.read_text())\n            if isinstance(js, dict) and js.get(\"units\"):\n                return js\n        except Exception:\n            pass\n    # default to UAE units when nothing saved\n    return {\"units\": list(UAE_UNITS), \"timestamp\": datetime.now(timezone.utc).isoformat()}\n\ndef _save_icu_status(js, path=\"/mnt/data/icu_status.json\"):\n    p = Path(path); p.parent.mkdir(parents=True, exist_ok=True)\n    p.write_text(json.dumps(js, ensure_ascii=False, indent=2))\n\ndef _compute_next_bed_eta(unit):\n    cap = int(unit.get(\"capacity\", 0) or 0)\n    occ = int(unit.get(\"occupied\", 0) or 0)\n    etas = [int(x) for x in (unit.get(\"discharge_eta_minutes\") or []) if str(x).strip().isdigit()]\n    if occ < cap: return 0\n    return min(etas) if etas else None\n\nif CONFIG.get(\"RUN_UI\"):\n    try:\n        import ipywidgets as W\n        import pandas as pd\n\n        state = _load_icu_status()\n        units = state[\"units\"]\n\n        # UI widgets\n        dd = W.Dropdown(options=[u.get(\"name\", \"(unnamed)\") for u in units] or [\"(add a unit)\"], description=\"Unit\")\n        name = W.Text(description=\"Name\",
```

```

placeholder="ICU-North")\n        cap =
W.IntText(description="Capacity", value=12)\n        occ =
W.IntText(description="Occupied", value=12)\n        eta =
W.Text(description="ETAs (min)", placeholder="e.g. 30, 90, 180")
\n\n        add_btn = W.Button(description="Add/Update unit")\n
calc_btn = W.Button(description="Estimate ETAs")\n        save_btn
= W.Button(description="Save status")\n        out = W.Output()
\n\n        def _refresh_dropdown():\n            dd.options =
[u.get("name", "(unnamed)") for u in units] or ["(add a unit)"]
\n\n        def _load_into_form(idx=0):\n            if not units:\n
name.value=""; cap.value=12; occ.value=12; eta.value="";
return\n            u = units[idx]\n            name.value =
str(u.get("name", ""))\n            cap.value =
int(u.get("capacity", 0) or 0)\n            occ.value =
int(u.get("occupied", 0) or 0)\n            seq =
u.get("discharge_eta_minutes") or []\n            eta.value = "\n",
"\n".join(str(int(x)) for x in seq)\n\n            def _parse_eta(txt:
str):\n                out = []\n                for chunk in txt.split("\n",
"\n"):\n                    chunk = chunk.strip()\n                    if
chunk:\n                        try: out.append(int(float(chunk)))\n
except: pass\n                return out\n\n            def
on_dd_change(change):\n                if change["name"]=="value"
and units:\n                    _load_into_form(dd.options.index(change["new"]))\n
dd.observe(on_dd_change)\n\n            def on_add():\n                # no
'nonlocal' needed: we mutate the existing list\n                u = {\n
"name": name.value.strip() or f"ICU-{len(units)+1}",\n
"capacity": int(cap.value or 0),\n                "occupied":
int(occ.value or 0),\n                "discharge_eta_minutes":
_parse_eta(eta.value),\n                }\n                names =
[x.get("name", "") for x in units]\n                if u["name"] in
names:\n                    units[names.index(u["name"])] = u\n
else:\n                    units.append(u)\n                _refresh_dropdown()\n                dd.value = u["name"]\n
with out:\n                print(f"Saved unit '{u['name']}'")\n\n            def on_calc():\n                rows = []\n                for u in units:\n
\n                    eta_min = _compute_next_bed_eta(u)\n
rows.append({\n                    "ICU": u.get("name", ""),\n
"capacity": int(u.get("capacity", 0) or 0),\n
"occupied": int(u.get("occupied", 0) or 0),\n
"next_bed_in": _format_eta(eta_min),\n                })\n                df = pd.DataFrame(rows) if rows else pd.DataFrame(columns=["ICU",
"capacity", "occupied", "next_bed_in"])\n                with out:\n
out.clear_output()\n                if df.empty:\n                    print("No units yet. Add a unit above.")\n                else:\n
display(df.style.hide(axis='index'))\n                # Natural-
language earliest\n                mins = [(r["ICU"],
_compute_next_bed_eta(u)) for r,u in zip(rows, units)]\n
mins = [(n,m) for n,m in mins if m is not None]\n
if mins:\n                name_min, m = sorted(mins,
key=lambda x: x[1])[0]\n                print(f"\n\nNext bed
available on {name_min} in {_format_eta(m)}")\n
else:\n                print("\n\nNext bed availability:
unknown (provide ETAs or reduce occupied < capacity).")\n\n

```

```

def on_save(_):\n                state[\"units\"] = units\nstate[\"timestamp\"] = datetime.now(timezone.utc).isoformat()\n_save_icu_status(state)\n                with out:\nprint(\"Saved to /mnt/data/icu_status.json\")\n\nadd_btn.on_click(on_add)\n                calc_btn.on_click(on_calc)\nsave_btn.on_click(on_save)\n\n                # Initial load\n_refresh_dropdown()\n                if units:\n                    dd.value =\ndd.options[0]\n                _load_into_form(0)\n\n                display(W.VBox([\n                    W.HTML(\"<b>ICU next-bed\navailability</b>\"),\n                    dd,\n                    W.HBox([name,\ncap, occ]),\n                    eta,\n                    W.HBox([add_btn,\ncalc_btn, save_btn]),\n                    out\n                ]))\n\n                except\nException as e:\n                print(\"ICU UI unavailable:\", e)\n\n            else:\nprint(\"ICU UI deferred... set CONFIG['RUN_UI']=True.\")\n\n            \"metadata\":{\"execution\":\n                {\"iopub.status.busy\":\"2025-08-25T18:40:38.922129Z\", \"iopub.execute_in\nput\":\"2025-08-25T18:40:38.922458Z\", \"iopub.status.idle\":\"2025-08-25T1\n8:40:38.977532Z\", \"shell.execute_reply.started\":\"2025-08-25T18:40:38.\n922429Z\", \"shell.execute_reply\":\"2025-08-25T18:40:38.976701Z\"}, \"paper\nmill\":\n                {\"duration\":0.064736, \"end_time\":\"2025-08-24T12:38:55.899331\", \"except\nion\":false, \"start_time\":\"2025-08-24T12:38:55.834595\", \"status\":\"compl\neted\"}, \"tags\":[], \"trusted\":true}, \"outputs\":\n                [{\"output_type\":\"display_data\", \"data\":{\"text/\nplain\":\"VBox(children=(HTML(value='<b>ICU next-bed availability</\nb>'), Dropdown(description='Unit', options=('1A\nNeuro...\", \"application/vnd.jupyter.widget-view+json\":\n                {\"version_major\":2, \"version_minor\":0, \"model_id\":\"7238ed8109544806847\nc97be8cddcdf\"}}, \"metadata\":{}}}, \"execution_count\":74},\n                {\"cell_type\":\"code\", \"source\":\"# == Synth helpers (timestamps + case\ngenerator) ==\n\n# Pure helpers. No side effects. Uses only fields\nconsumed by gate_engine/ops_intel/troponin_policy.\n\nimport random,\nfrom datetime import datetime, timezone\n\ndef _ts() -> str:\n    return datetime.now(timezone.utc).isoformat()\n\n\ndef\nsynth_patient(seed=None) -> dict:\n    \"\"\"Generate one\n    realistic ED case.\n    SBP-centric vitals, optional VBG, abdomen\nguardrails, and troponin timing for due/overdue.\n    \"\"\"\n    rng = random.Random(seed if seed is not None else random.randint(0,\n2**31-1))\n    case = {}\n    # Triage & vitals (SBP, not MAP)\n    case[\"Triage\"] = rng.choices([\"Green\", \"Yellow\", \"Orange\", \"Red\"],\nweights=[45,30,20,5], k=1)[0]\n    case[\"SBP\"] =\nmax(70, int(rng.gauss(115, 20)))\n    # mmHg\n    case[\"HR\"] = max(40, int(rng.gauss(92, 20)))\n    case[\"SpO2\"] = max(70, int(rng.gauss(95, 4)))\n    case[\"GCS\"]\n= rng.choices([15, 14, 13], weights=[90,7,3])[0]\n    # Chief\ncomplaint\n    cc = rng.choices([\n        \"Brustschmerz\", \"Bauchschmerz\", \"Dyspnoe\", \"Synkope\", \"Fieber\", \"Schwindel\"],\n        weights=[18,22,18,10,20,12], k=1)[0]\n    case[\"Leitsymptom\"] = cc\n    # VBG in ~40%: mostly normal, but\ninject tails to exercise gates/alerts\n    if rng.random() < 0.4:\n        case[\"vbg_pH\"] = round(rng.gauss(7.40, 0.06), 2)\n        case[\"vbg_pco2_mmHg\"] = round(max(30, rng.gauss(42, 12)), 1)\n        case[\"vbg_HCO3\"] = round(max(8, rng.gauss(24, 5)), 1)\n        case[\"vbg_lactate\"] = round(max(0.5, rng.gauss(1.8, 0.9)), 1)\n
```

```

case["vbg_Na"]           = round(rng.gauss(139, 6), 1)\n
case["vbg_Cl"]           = round(rng.gauss(103, 6), 1)\n
case["vbg_K"]            = round(rng.gauss(4.1, 0.7), 1)\n\n      #
Rare criticals (aligned with your ops_intel thresholds)\n      if
rng.random() < 0.05: case["vbg_pH"]           = 7.56  # alkalosis
critical (>7.55)\n      if rng.random() < 0.05: case["vbg_K"]
= 6.6  # hyperK critical (>=6.5)\n      if rng.random() < 0.05:
case["vbg_lactate"] = 4.3  # lactate critical (>=4.0)\n
if rng.random() < 0.03: case["vbg_Na"]        = 124  #
hyponatraemia critical (<=125)\n\n      # Abdomen guardrails (force
unmet requirements to trigger gates)\n      if "bauch" in cc.lower()
or "abd" in cc.lower():\n          case["US_done"] = False;
case["CT_done"] = False\n          case["labs_cycles_done"] =
1\n\n      # Troponin: enable due/overdue logic; we're not doing assay
inference here\n      if cc == "Brustschmerz" and rng.random() <
0.7:\n          case["troponin0_ngL"] = rng.choice([8, 10, 14, 20,
55]) # includes potential rule-in t0\n
case["since_trop1_min"] = rng.choice([30, 65, 80])      # to
trigger DUE/OVERDUE\n          case["admit_decision"] = rng.random()
< 0.5\n\n      return case\n", "metadata": {"execution":
{"iopub.status.busy": "2025-08-25T18:40:38.978981Z", "iopub.execute_in
put": "2025-08-25T18:40:38.979349Z", "iopub.status.idle": "2025-08-25T1
8:40:39.001172Z", "shell.execute_reply.started": "2025-08-25T18:40:38.
979316Z", "shell.execute_reply": "2025-08-25T18:40:39.000139Z"}, "paper
mill":
{"duration": 0.025682, "end_time": "2025-08-24T12:38:55.935860", "except
ion": false, "start_time": "2025-08-24T12:38:55.910178", "status": "compl
eted"}, "tags": [], "trusted": true}, "outputs": [], "execution_count": 75},
{"cell_type": "code", "source": "# == Gate-only stream emitter (no
MLP, writes JSONL) ==\n# Pure function; no side effects unless you
call it.\n\nfrom collections import Counter\nfrom pathlib import
Path\nfrom typing import Dict, Any\n\ndef emit_events(n: int = 50,
seed: int = 42, sleep_s: float = 0.0,\n\n      CONFIG:
Dict[str, Any] = None, EVENT_LOG_PATH: str | None = None):\n
assert CONFIG is not None, "CONFIG required"\n      log_path =
EVENT_LOG_PATH or CONFIG.get("EVENT_LOG_PATH", "/kaggle/working/
event_log.jsonl")\n      Path(log_path).parent.mkdir(parents=True,
exist_ok=True)\n      Path(log_path).touch(exist_ok=True)\n\n
gate_counts, prio_hist, actions = Counter(), Counter(), Counter()
\n\n      for i in range(n):\n          fd = synth_patient(seed + i)\n
pack = gate_engine(fd, CONFIG) # uses your troponin_policy +
ops_intel\n\n          for g in pack.get("gates", []):\n
gate_counts[g] += 1\n          prio_hist[pack.get("priority", 0)] +=
1\n          if pack.get("next_action"):\n
actions[pack["next_action"]] += 1\n\n          rec = {\n
"ts": _ts(),\n          "case_id": f"synth-{seed}-{i}",\n
"fd": fd,\n          "gates": pack.get("gates", []),\n
"priority": pack.get("priority", 0),\n
"next_action": pack.get("next_action", ""),\n
"ttl": pack.get("ttl", {}),\n          "explain":
pack.get("explain", []),\n          }\n          with open(log_path,
"a", encoding="utf-8") as f:\n
f.write(json.dumps(rec, ensure_ascii=False) + "\\n\\n")\n\n      if
sleep_s: # optional pacing\n          import time;

```

```

time.sleep(float(sleep_s))\n\n    # Optional tabular summary (pandas
if available)\n    try:\n        import pandas as pd\n        df =
(pd.DataFrame({"gate\:": list(gate_counts.keys()), "count\:":
list(gate_counts.values())})
\n        .sort_values("count",
ascending=False).reset_index(drop=True))\n        print("\Gate
frequency (top 10):\"); display(df.head(10))\n        except Exception:
\n        print("\Gate frequency (top 10):\n",
gate_counts.most_common(10))\n        print("\Priority histogram:\n",
dict(prio_hist))\n        print("\Top actions:\n",
actions.most_common(5))\n\n        return {"gate_counts\:": gate_counts,
"\prio_hist\:": prio_hist, "actions\:": actions, "\log_path\:":
log_path}\n", "metadata": {"execution":
{"iopub.status.busy": "2025-08-25T18:40:39.002204Z", "iopub.execute_in
put": "2025-08-25T18:40:39.002529Z", "iopub.status.idle": "2025-08-25T1
8:40:39.026895Z", "shell.execute_reply.started": "2025-08-25T18:40:39.
002480Z", "shell.execute_reply": "2025-08-25T18:40:39.025817Z"}, "paper
mill":
{"duration": 0.023043, "end_time": "2025-08-24T12:38:55.969605", "except
ion": false, "start_time": "2025-08-24T12:38:55.946562", "status": "compl
eted"}, "tags": [], "trusted": true}, "outputs": [], "execution_count": 76},
{"cell_type": "code", "source": "# === ICU capacity gate-pack (tiny &
pure; reuses ICU_PREFS if present) ===\nfrom typing import Dict,
Any, List\n\ndef _best_relevant_icu_capacity(fd: Dict[str, Any],
ICU_PREFS: Dict[str, List[str]]) -> float:\n    cond =
fd.get("suspected_condition")\n    keys = ICU_PREFS.get(cond, [])
\n    vals = []\n    for k in keys:\n        try:
vals.append(float(fd.get(k, 0.0)))\n        except Exception: pass\n
return max(vals) if vals else 0.0\n\ndef capacity_gatepack(fd:
Dict[str, Any], CONFIG: Dict[str, Any]) -> Dict[str, Any]:\n    #
Prefer your panel's ICU_PREFS if it exists; else fallback\n
ICU_PREFS = CONFIG.get("ICU_PREFS", globals().get("ICU_PREFS",
{\n        "cardiac\:": [\n            "cap_cardio_icu",
\n            "cap_cardio_surgery_icu", "cap_vascular_cardiac_icu",\n
\n            "respiratory\:": [\n            "cap_internal_medicine_icu",
\n            "cap_interdis_stage1", "cap_interdis_stage2",
\n            "cap_interdis_stage3",\n        ],\n        "neurological\:":
[\n            "cap_neurological_icu", "cap_neurochirurgical_icu",
\n            "cap_interdis_stage1",\n        ],\n        "infection\:":
[\n            "cap_internal_medicine_icu", "cap_interdis_stage2",\n
\n            "trauma\:": [\n            "cap_surgical_icu", "cap_interdis_stage3",\n
\n            "oncology\:": [\n            "cap_internal_medicine_icu",
\n            "cap_interdis_stage2",\n        ],\n        "gastrointestinal\:":
[\n            "cap_surgical_icu", "cap_internal_medicine_icu",\n
\n        ]\n    })))\n
TH_OK = float(CONFIG.get("TH_ICU_CAP_OK", 0.25)) # ≥ OK\n
TH_TIGHT = float(CONFIG.get("TH_ICU_CAP_TIGHT", 0.15)) # <
tight\n
SLA_ESC = int(CONFIG.get("SLA_ADMIT_ESCALATE_MIN",
60))\n\n    triage_txt = str(fd.get("Triage", "")).title()\n
urgent = (int(fd.get("ems_triage_code", 3)) <= 2) or (triage_txt
in ("Red", "Orange"))\n\n    cap_rel =
_best_relevant_icu_capacity(fd, ICU_PREFS) # 0..1\n    bottleneck =
1.0 - cap_rel\n\n    gates: List[str] = []; explain: List[str] = [];
ttl: Dict[str, int] = {}\n    prio, action = 0, ""\n\n    if
cap_rel <= 0.0:\n        gates += ["ICU_BLOCKED"]; explain +=

```

```

[\"Relevant ICU capacity = 0%\"]\n        prio = max(prio, 5);
action = action or \"No ICU capacity – board in ED, activate bed
manager now.\\"\n        elif cap_rel < TH_TIGHT:\n            gates +=
[\"ICU_TIGHT\"]; explain += [f\"Relevant ICU capacity <
{int(TH_TIGHT*100)}%\"]\n            prio = max(prio, 5 if urgent else
4); action = action or \"Tight ICU capacity – escalate to bed
manager.\\"\n        elif cap_rel >= TH_OK:\n            gates +=
[\"ICU_OK\"]; explain += [f\"Relevant ICU capacity ≥
{int(TH_OK*100)}%\"]\n            prio = max(prio, 3 if urgent else 2);
action = action or \"Proceed with ICU handoff planning.\\"\n\n        if
bottleneck >= 0.70:\n            gates += [\"ED_BOARDING_RISK\"];
explain += [\"High boarding risk from ICU bottleneck\"]\n
prio = max(prio, 4)\n\n        if urgent and cap_rel >= TH_OK:\n
gates += [\"ROUTE_PLAN_EMS_TO_ICU\"]; explain += [\"Urgent +
capacity OK → EMS→ICU\"]\n        else:\n            gates +=
[\"ROUTE_PLAN_ED_TO_ICU\"]; explain += [\"Default or constrained →
EMS→ED→ICU\"]\n\n        if any(g in (\"ICU_BLOCKED\", \"ICU_TIGHT\") for
g in gates):\n            for g in (\"ALERT_NURSE\", \"ALERT_DOC\"): \n
if g not in gates: gates.append(g); ttl[g] = 300\n            if
(\"ICU_BLOCKED\" in gates) or (urgent and \"ICU_TIGHT\" in gates):\n
if \"ALERT_ATTENDING\" not in gates:
gates.append(\"ALERT_ATTENDING\"); ttl[\"ALERT_ATTENDING\"] =
300\n\n        if str(fd.get(\"admit_decision\", \"\")).upper() ==
\"ICU\" and any(g in (\"ICU_BLOCKED\", \"ICU_TIGHT\") for g in
gates):\n            gates.append(\"TRANSFER_BLOCK_CAPACITY\");
explain.append(f\"Recheck capacity q{SLA_ESC}min until cleared\")\n
ttl[\"TRANSFER_BLOCK_CAPACITY\"] = SLA_ESC * 60\n\n        # de-dup
preserve order\n        seen=set(); gates=[g for g in gates if (g not in
seen and not seen.add(g))]\n        return {\"gates\": gates,
\"priority\": prio, \"next_action\": action, \"explain\": explain,
\"ttl\": ttl}\n\n\", \"metadata\": {\"execution\":
{\"iopub.status.busy\": \"2025-08-25T18:40:39.027849Z\", \"iopub.execute_in
put\": \"2025-08-25T18:40:39.028126Z\", \"iopub.status.idle\": \"2025-08-25T1
8:40:39.055542Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:39.
028099Z\", \"shell.execute_reply\": \"2025-08-25T18:40:39.054677Z\"}, \"paper
mill\":
{\"duration\": 0.028, \"end_time\": \"2025-08-24T12:38:56.008766\", \"exception
\": false, \"start_time\": \"2025-08-24T12:38:55.980766\", \"status\": \"complete
d\"}, \"tags\": [], \"trusted\": true}, \"outputs\": [], \"execution_count\": 77},
{\"cell_type\": \"code\", \"source\": \"# === Guarded execution (set
CONFIG['RUN_SYNTH']=True to run) ===\n# Interface constraint: no
side effects unless RUN_SYNTH is True.\n\nfrom pathlib import
Path\n\nif CONFIG.get(\"RUN_SYNTH\", False):\n    out =
emit_events(\n        n = int(CONFIG.get(\"SYNTH_N\", 100)),
\n        seed = int(CONFIG.get(\"SYNTH_SEED\", 42)),\n
sleep_s = float(CONFIG.get(\"SYNTH_SLEEP_S\", 0.0)),\n
CONFIG = CONFIG,\n        EVENT_LOG_PATH =
CONFIG.get(\"EVENT_LOG_PATH\", \"./kaggle/working/event_log.json\"),
\n    )\n    # Minimal smoke: file exists and not empty\n    lp =
out[\"log_path\"]\n    assert Path(lp).exists() and
Path(lp).stat().st_size > 0, \"Event log not written\"\n
print(\"SYNTH_STREAM_OK →\", lp)\nelse:\n    print(\"Deferred... set
CONFIG['RUN_SYNTH']=True to generate + run synthetic pipeline.\")
\n\", \"metadata\": {\"execution\":

```

```

{"iopub.status.busy":"2025-08-25T18:40:39.056644Z","iopub.execute_in
put":"2025-08-25T18:40:39.056905Z","iopub.status.idle":"2025-08-25T1
8:40:39.078115Z","shell.execute_reply.started":"2025-08-25T18:40:39.
056885Z","shell.execute_reply":"2025-08-25T18:40:39.077269Z"},"paper
mill":
{"duration":0.019736,"end_time":"2025-08-24T12:38:56.039250","except
ion":false,"start_time":"2025-08-24T12:38:56.019514","status":"compl
eted"},"tags":[],"trusted":true},"outputs":
[{"name":"stdout","text":"Deferred... set CONFIG['RUN_SYNTH']=True to
generate + run synthetic pipeline.
\n","output_type":"stream"}],"execution_count":78},
{"cell_type":"code","source":"# Extend CONFIG for meds integration
(idempotent, Kaggle-safe)\nCONFIG.setdefault(\"MED_RULES_PATH\",
_p(\"interaction_rules.json\"))
\nCONFIG.setdefault(\"ALLERGIES_PATH\",
_p(\"patient_allergies.json\"))\n\nfrom pathlib import Path\nimport
json, os\n\n# Ensure files exist and are valid JSON arrays (no
seeding of drug examples here)\nfor k in (\"MED_RULES_PATH\",
\"ALLERGIES_PATH\"):\n    p = Path(CONFIG[k]);
p.parent.mkdir(parents=True, exist_ok=True)\n    if not p.exists()
or os.path.getsize(p) == 0:\n        p.write_text(\"[]\",
encoding=\"utf-8\")\n    try:\n
json.loads(p.read_text(encoding=\"utf-8\") or \"[]\")\n    except
Exception:\n        p.write_text(\"[]\", encoding=\"utf-8\")
\nprint(\"Meds/allergy paths ->\", CONFIG[\"MED_RULES_PATH\"], \"|\",
CONFIG[\"ALLERGIES_PATH\"])\n","metadata":{"execution":
{"iopub.status.busy":"2025-08-25T18:40:39.082676Z","iopub.execute_in
put":"2025-08-25T18:40:39.082958Z","iopub.status.idle":"2025-08-25T1
8:40:39.105121Z","shell.execute_reply.started":"2025-08-25T18:40:39.
082933Z","shell.execute_reply":"2025-08-25T18:40:39.103754Z"},"paper
mill":
{"duration":0.020754,"end_time":"2025-08-24T12:38:56.071090","except
ion":false,"start_time":"2025-08-24T12:38:56.050336","status":"compl
eted"},"tags":[],"trusted":true},"outputs":
[{"name":"stdout","text":"Meds/allergy paths -> /mnt/data/
interaction_rules.json | /mnt/data/
patient_allergies.json\n","output_type":"stream"}],"execution_count"
:79},{"cell_type":"code","source":"# === Calc utils (shared, model-
free) ===\nfrom typing import Dict, Any, Optional, List,
Tuple\nimport re, math\nfrom datetime import datetime\n\nndef
_pick(d: Dict[str, Any], names: List[str], cast=float,
default=None):\n    for n in names:\n        if n in d and d[n] not
in (None, \"\"): \n            try:\n                return
cast(d[n])\n            except Exception:\n                try:\n
return cast(str(d[n]).replace(\"\\\", \"\\.\").replace(\"/\", \"\\.\"))\n
            except
Exception:\n                pass\n            return default\n\nndef
_bool(v) -> bool:\n    if isinstance(v, bool): return v\n    s =
str(v).strip().lower()\n    return s in {\"1\", \"true\", \"yes\",
\"y\", \"ja\", \"on\", \"oui\"}\n\nndef _safe_round(x, nd=1):\n    try:
\n        return round(float(x), nd)\n    except Exception:\n
return x\n# === Phase-2 calculators A (core scores; model-free)
===\nfrom typing import Dict, Any\n","metadata":{"execution":
{"iopub.status.busy":"2025-08-25T18:40:39.105917Z","iopub.execute_in
put":"2025-08-25T18:40:39.106191Z","iopub.status.idle":"2025-08-25T1

```

8:40:39.127221Z","shell.execute_reply.started":"2025-08-25T18:40:39.106163Z","shell.execute_reply":"2025-08-25T18:40:39.126221Z"},"paper mill":

```
{"duration":0.041175,"end_time":"2025-08-24T12:38:56.123190","exception":false,"start_time":"2025-08-24T12:38:56.082015","status":"completed"},"tags":[],"trusted":true},"outputs":[],"execution_count":80}, {"cell_type":"code","source":"# == Phase-2 calculators A (core scores; model-free) ==\nfrom typing import Dict, Any\n\ndef calc_qsofa(v: Dict[str,Any]):\n    rr=_pick(v,['rr'],\n    ['resp_rate']); sbp=_pick(v,['sbp','systolic']); gcs=_pick(v,\n    ['gcs'],float)\n    avpu = (v.get('avpu') or '').upper()[1]\n    altered = (gcs is not None and gcs<15) or avpu in {'V','P','U'}\n    return {'name':'qSofA','score':int((rr is not\n    None and rr>=22)) + int((sbp is not None and sbp<=100)) +\n    int(altered))}\n\ndef calc_mews(v: Dict[str,Any]):\n    def rr_s(x):\n    return 3 if x is not None and x<=8 else (0 if x and 9<=x<=14 else (1\n    if x and 15<=x<=20 else (2 if x and 21<=x<=29 else (3 if x and x>=30\n    else 0))))\n    def hr_s(x): return 2 if x is not None and x<=40\n    else (1 if x and 41<=x<=50 else (0 if x and 51<=x<=100 else (1 if x\n    and 101<=x<=110 else (2 if x and 111<=x<=129 else (3 if x and x>=130\n    else 0)))))\n    def sbp_s(x): return 3 if x is not None and x<=70\n    else (2 if x and 71<=x<=80 else (1 if x and 81<=x<=100 else (0 if x\n    and 101<=x<=199 else (2 if x and x>=200 else 0))))\n    def t_s(x):\n    return 2 if x is not None and x<=35.0 else (1 if x and 35.1<=x<=36.0\n    else (0 if x and 36.1<=x<=38.0 else (1 if x and 38.1<=x<=38.5 else\n    (2 if x and x>=38.6 else 0))))\n    def avpu_s(x): return\n    {'A':0,'V':1,'P':2,'U':3}.get((x or 'A').upper()[1],0)\n    return {'name':'MEWS','score':int(rr_s(_pick(v,['rr']))\n    +hr_s(_pick(v,['hr','pulse']))+sbp_s(_pick(v,['sbp']))\n    +t_s(_pick(v,['temp']))+avpu_s(v.get('avpu')))}\n\ndef calc_heart(p: Dict[str,Any]):\n    age=_pick(p,['age'],int);\n    hist=_pick(p,['heart_history'],int); ecg=_pick(p,\n    ['heart_ecg'],int); risk=_pick(p,['heart_risk'],int)\n    trop=_pick(p,['troponin','hs_troponin','trop']); uln=_pick(p,\n    ['troponin_uln'],float)\n    age_s = 2 if (age is not None and\n    age>=65) else (1 if (age is not None and 45<=age<=64) else 0)\n    ratio = (trop/uln) if (trop is not None and uln) else None\n    trop_s = 2 if (ratio is not None and ratio>3) else (1 if (ratio is\n    not None and 1<ratio<=3) else 0)\n    total = (hist or 0)+(ecg or\n    0)+age_s+(risk or 0)+trop_s\n    return {'name':'HEART',\n    'score':int(total)}\n\ndef calc_grace_coarse(p: Dict[str,Any]):\n    age=_pick(p,['age'],int); hr=_pick(p,['hr','pulse']);\n    sbp=_pick(p,['sbp']); crea=_pick(p,['creatinine'])\n    sc=0\n    if age is not None: sc += (0 if age<40 else 20 if age<60 else 40 if\n    age<80 else 60)\n    if hr is not None: sc += (0 if hr<70 else 10\n    if hr<90 else 20 if hr<110 else 30 if hr<150 else 40)\n    if sbp\n    is not None: sc += (40 if sbp<80 else 30 if sbp<100 else 10 if\n    sbp<120 else 0)\n    if crea is not None: sc += (0 if crea<1.2 else\n    10 if crea<2.0 else 20 if crea<3.0 else 30)\n    return {'name':\n    'GRACE_coarse','score':int(sc)}\n\ndef calc_sofa_min(v:\n    Dict[str,Any], labs: Dict[str,Any]):\n    pf=None; pao2=_pick(labs,\n    ['pao2']); fio2=_pick(labs,['fio2'])\n    if pao2 is not None\n    and fio2:\n        try: pf=float(pao2)/float(fio2)\n        except\n        Exception: pf=None\n        plate=_pick(labs,['platelets','plt']);
```

```

bili=_pick(labs,["bilirubin"]); mapv=_pick(v,["mapv"]);
gcs=_pick(v,["gcs"]); crea=_pick(labs,["creatinine"])\n
sc=0\n    if pf is not None:    sc += (4 if pf<100 else 3 if pf<200
else 2 if pf<300 else 1 if pf<400 else 0)\n    if plate is not None:
sc += (4 if plate<20 else 3 if plate<50 else 2 if plate<100 else 1
if plate<150 else 0)\n    if bili is not None: sc += (4 if bili>=12
else 3 if bili>=6 else 2 if bili>=2 else 1 if bili>=1.2 else 0)\n
if mapv is not None: sc += (1 if mapv<70 else 0) # presence-
guarded (MAP not required)\n    if gcs is not None: sc += (4 if
gcs<6 else 3 if gcs<10 else 2 if gcs<13 else 1 if gcs<15 else 0)\n
if crea is not None: sc += (4 if crea>=5 else 3 if crea>=3.5 else 2
if crea>=2 else 1 if crea>=1.2 else 0)\n    return {"name":
"Sofa_min","score": int(sc)}\n\nndef calc_sirs(p: Dict[str,Any]):
\n    crit = {\n        "temp>38/<36": int(((pick(p,
["temp"],float) or 37)>38) or ((pick(p,["temp"],float) or
37)<36)),\n        "hr>90": int((pick(p,["hr"],"pulse"),float)
or 0) > 90),\n        "rr>20/paco2<32": int(((pick(p,
["rr"],float) or 0) > 20) or ((pick(p,["paco2"],float) or 100)
< 32)),\n        "wbc>12/<4/bands>10%": int(((pick(p,
["wbc"],float) or 7) > 12) or ((pick(p,["wbc"],float) or 7) <
4) or ((pick(p,["bands_pct"],float) or 0) > 10)),\n    }\n
return {"name":"SIRS","score": int(sum(crit.values()))}\n\nndef
sepsis3_screen(v: Dict[str,Any], labs: Dict[str,Any], ctx:
Dict[str,Any], sofa_min: Dict[str,Any], qsofa: Dict[str,Any]):\n
sus = _bool(ctx.get("suspected_infection"));
on_pressors=_bool(ctx.get("vasopressors"))\n    mapv=_pick(v,
["mapv"]); lact=_pick(labs,["lactate"])\n    septic_shock =
bool((mapv is not None and mapv<65) and (lact is not None and
lact>2) and on_pressors)\n    return {"name":"SEPSIS3",
"sepsis_flag": bool(sus and sofa_min["score"]>=2),
"septic_shock": septic_shock}\n\n,"metadata":{"execution":
{"iopub.status.busy":"2025-08-25T18:40:39.128269Z","iopub.execute_in
put":"2025-08-25T18:40:39.128566Z","iopub.status.idle":"2025-08-25T1
8:40:39.158835Z","shell.execute_reply.started":"2025-08-25T18:40:39.
128536Z","shell.execute_reply":"2025-08-25T18:40:39.157787Z"},"paper
mill":
{"duration":0.038069,"end_time":"2025-08-24T12:38:56.172410","except
ion":false,"start_time":"2025-08-24T12:38:56.134341","status":"compl
eted"},"tags":[],"trusted":true},"outputs":[],"execution_count":81},
{"cell_type":"code","source":"# == Phase-2 calculators B (PE/VTE,
GI/hepatic, corrections + lab assess) ==\nfrom typing import Dict,
Any, List, Optional, Tuple\n\nndef calc_wells_pe(p: Dict[str,Any]):\n
pts=0.0\n    pts+=3.0 if _bool(p.get("dvt_signs")) else 0.0\n
pts+=3.0 if _bool(p.get("pe_most_likely")) else 0.0\n    pts+=1.5
if ((pick(p,["hr"],"pulse"),float) or 0)>100) else 0.0\n
pts+=1.5 if (_bool(p.get("immobilized")) or
_bool(p.get("recent_surgery_4w")))) else 0.0\n    pts+=1.5 if
_bool(p.get("prev_vte")) else 0.0\n    pts+=1.0 if
_bool(p.get("hemoptysis")) else 0.0\n    pts+=1.0 if
_bool(p.get("cancer_active")) else 0.0\n    return {"name":
"WELLS_PE","score": pts, "tier2": ("likely" if pts>4 else
"unlikely")}\n\nndef calc_wells_dvt(p: Dict[str,Any]):\n    pts=0\n
pts+=1 if _bool(p.get("cancer_active")) else 0\n    pts+=1 if
(_bool(p.get("paresis")) or _bool(p.get("plaster_cast"))) else

```

```

0\n    pts+=1 if (_bool(p.get("bedridden_3d\n")) or
_bool(p.get("surgery_12w\n"))) else 0\n    pts+=1 if
_bool(p.get("deep_vein_tenderness\n")) else 0\n    pts+=1 if
_bool(p.get("entire_leg_swollen\n")) else 0\n    pts+=1 if
_bool(p.get("calf_swelling_gt3cm\n")) else 0\n    pts+=1 if
_bool(p.get("pitting_edema\n")) else 0\n    pts+=1 if
_bool(p.get("collateral_nonvaricose\n")) else 0\n    pts+=1 if
_bool(p.get("prev_dvt\n")) else 0\n    pts-=2 if
_bool(p.get("alt_dx_as_likely\n")) else 0\n    return {"name\n":
"WELLS_DVT\n", "score\n": int(pts), "tier2\n": ("likely\n" if pts>=2
else "unlikely\n")}\n\ndef calc_perc(p: Dict[str,Any]):\n    crit =
{\n        "age<50\n": int((_pick(p, ["age\n"], int) or 10) < 50),\n
"hr<100\n": int((_pick(p, ["hr\n", "pulse\n"], float) or 0) < 100),\n
"sao2>=95\n": int((_pick(p, ["sao2\n", "spo2\n"], float) or 0) >= 95),
\n        "no_hemoptysis\n": int(not _bool(p.get("hemoptysis\n"))),
\n        "no_estrogen\n": int(not _bool(p.get("estrogen_use\n"))),
\n        "no_surg/trauma_4w\n": int(not
(_bool(p.get("recent_surgery_4w\n")) or
_bool(p.get("recent_trauma_4w\n")))),\n        "no_prior_vte\n":
int(not _bool(p.get("prev_vte\n"))),\n
"no_unilateral_swelling\n": int(not
_bool(p.get("unilateral_leg_swelling\n"))),\n    }\n    return
{"name\n": "PERC\n", "passed\n": bool(all(crit.values()))}\n\ndef
calc_pesi(p: Dict[str,Any]):\n    age=_pick(p, ["age\n"], int) or 0\n
male = 10 if (str(p.get("sex\n") or "\n").upper().startswith("M\n"))
else 0\n    cancer = 30 if _bool(p.get("cancer_active\n")) else 0\n
hf = 10 if _bool(p.get("heart_failure\n")) else 0\n    lung = 10 if
(_bool(p.get("copd\n")) or _bool(p.get("chronic_lung_disease\n")))
else 0\n    hr = 20 if ((_pick(p, ["hr\n", "pulse\n"], float) or 0)
>=110) else 0\n    sbp = 30 if ((_pick(p, ["sbp\n"], float) or 200) <
100) else 0\n    rr = 20 if ((_pick(p, ["rr\n"], float) or 0) >=30)
else 0\n    temp = 20 if ((_pick(p, ["temp\n"], float) or 37) < 36)
else 0\n    altered = 60 if ((_pick(p, ["gcs\n"], float) or 15) < 15)
else 0\n    sat = 20 if ((_pick(p, ["sao2\n", "spo2\n"], float) or 100)
< 90) else 0\n    score =
age+male+cancer+hf+lung+hr+sbp+rr+temp+altered+sat\n    klass =
"I\n" if score<=65 else "II\n" if score<=85 else "III\n" if
score<=105 else "IV\n" if score<=125 else "V\n"\n    return
{"name\n": "PESI\n", "score\n": int(score), "class\n": klass}\n\ndef
calc_spesi(p: Dict[str,Any]):\n    comps = {\n        "age>80\n":
int((_pick(p, ["age\n"], int) or 0) > 80),\n        "cancer\n":
int(_bool(p.get("cancer_active\n"))),\n        "cardiopulm\n":
int(_bool(p.get("heart_failure\n")) or _bool(p.get("copd\n")) or
_bool(p.get("chronic_lung_disease\n"))),\n        "hr>=110\n":
int((_pick(p, ["hr\n", "pulse\n"], float) or 0) >= 110),\n
"sbp<100\n": int((_pick(p, ["sbp\n"], float) or 200) < 100),\n
"o2<90\n": int((_pick(p, ["sao2\n", "spo2\n"], float) or 100) < 90),
\n    }\n    return {"name\n": "sPESI\n", "score\n":
int(sum(comps.values()))}\n\ndef calc_marburg(p: Dict[str,Any]):\n
sex=(str(p.get("sex\n") or "\n")[:1]).upper(); age=_pick(p,
["age\n"], int)\n    vasc=_bool(p.get("vasc_disease\n"));
exert=_bool(p.get("exertional\n"));
pt_thinks=_bool(p.get("patient_assumes_cardiac\n"))\n    palp =
p.get("palpation_reproducible\n"); not_repro = (palp is False)\n

```

```

age_sex = ((sex=="M" and age is not None and age>=55) or
(sex=="F" and age is not None and age>=65))\n    sc =
int(bool(age_sex)) + int(vasc) + int(exert) + int(pt_thinks) +
int(bool(not_repro))\n    return {"name\":"MARBURG\","score\":
int(sc)}\n\nndef calc_gbs(p: Dict[str,Any]):\n    score=0\nurea=_pick(p,["urea_mmol_l","\nurea"]); bun=_pick(p,
["bun_mg_dl"])\n    if urea is None and bun is not None:
urea=float(bun)/2.8\n    hb_gL=_pick(p,["hb_g_l"]); \n    if hb_gL
is None:\n        hb_gdL=_pick(p,["hb","\nhb_g_dl"]); \n        if
hb_gdL is not None: hb_gL=hb_gdL*10.0\n        sbp=_pick(p,["sbp"]);
hr=_pick(p,["hr","\npulse"]); male = str(p.get("\nsex") or
"\n").upper().startswith("\nM")\n    if urea is not None:\n
score += 2 if 6.5<=urea<=7.9 else 0; score += 3 if 8.0<=urea<=9.9
else 0\n        score += 4 if 10.0<=urea<=25.0 else 0; score += 6 if
urea>25.0 else 0\n        if hb_gL is not None:\n            if male:
score += (1 if 120<=hb_gL<=129 else 0) + (3 if 100<=hb_gL<=119 else
0) + (6 if hb_gL<100 else 0)\n            else: score += (1 if
100<=hb_gL<=119 else 0) + (6 if hb_gL<100 else 0)\n        if sbp is not
None: score += (1 if 100<=sbp<=109 else 0) + (2 if 90<=sbp<=99 else
0) + (3 if sbp<90 else 0)\n        score += 1 if (hr is not None and
hr>=100) else 0\n        score += 1 if _bool(p.get("\nmelena")) else 0\n
score += 2 if _bool(p.get("\nsyncope")) else 0\n        score += 2 if
_bool(p.get("\nhepatic_disease")) else 0\n        score += 2 if
_bool(p.get("\ncardiac_failure")) else 0\n    return {"name\":"
GBS\","score\": int(score)}\n\nndef calc_child_pugh(p:
Dict[str,Any]):\n    bili=_pick(p,["bilirubin"]); alb=_pick(p,
["albumin"]); inr=_pick(p,["inr"])\n    asc=(p.get("\nascites")
or "\n").lower(); ence=(p.get("\nencephalopathy") or "\n").lower()\n
sc=0; filled=0\n    if bili is not None: sc+=(1 if bili<2 else 2 if
bili<=3 else 3); filled+=1\n    if alb is not None: sc+=(1 if
alb>3.5 else 2 if alb>=2.8 else 3); filled+=1\n    if inr is not
None: sc+=(1 if inr<1.7 else 2 if inr<=2.3 else 3); filled+=1\n
if asc:
        sc+=(1 if asc.startswith("\nn") else 2 if
asc.startswith(("mild","\nslight")) else 3); filled+=1\n    if
ence:
        sc+=(1 if ence in {"none","\n0"} else 2 if any(x
in ence for x in ["1","\n2","\ni","\nii"]) else 3); filled+=1\n
if filled<5: return {"name\":"CHILD_PUGH\","score_partial\":
int(sc), "\nclass\":"incomplete"}\n    klass = "A" if sc<=6 else
("B" if sc<=9 else "C")\n    return {"name\":"CHILD_PUGH",
"score\": int(sc), "\nclass\": klass}\n\nndef
corrected_calcium(total_ca, albumin, units="\nmg/dL",
normal_alb=None):\n    if total_ca is None or albumin is None:
return None\n    if "\nmmol" in (units or "\n").lower():\n
alb_gl = albumin if albumin>10 else albumin*10.0\n        normal =
40.0 if normal_alb is None else float(normal_alb)\n        return
float(total_ca) + 0.02*(normal - alb_gl)\n        normal = 4.0 if
normal_alb is None else float(normal_alb)\n        return
float(total_ca) + 0.8*(normal - float(albumin))\n\nndef anion_gap(na,
cl, hco3, k=None, albumin_gdL=None):\n    if na is None or cl is
None or hco3 is None: return None\n    ag = (float(na)+(float(k) if
k is not None else 0.0)) - (float(cl)+float(hco3))\n    agc = ag +
(2.5*(4.0 - float(albumin_gdL))) if albumin_gdL is not None else
None\n    return {"ag\": _safe_round(ag,1),
"ag_albumin_corrected\": _safe_round(agc,1) if agc is not None else

```

```

None}\n\n# --- Lab assessors (reuse HL7 unit converters if
available; else safe fallback) ---\n\nndef assess_d_dimer(value,
unit, age, pregnant=False):\n    if value is None: return
{"available": False}\n    # Prefer canonical converter from HL7
cell if present\n    conv = globals().get("_to_ddimer_mgL_feu",
None)\n    if conv:\n        mgL = conv(value, unit); val_u =
mgL*1000.0 if mgL is not None else None\n    else:\n        unit_l =
(unit or "").lower()\n        # fallback assumes FEU; micro→mg\n
if "mg/l" in unit_l: val_u = float(value)*1000.0\n    else:\n
val_u = float(value) # assume µg/L FEU\n    thr = 500.0\n    if
age is not None and age>50 and not pregnant:\n        thr =
float(age)*10.0\n    return {"available": True,
"value_u_per_l": val_u, "thr_u_per_l": thr, "ok_below_thr":
bool(val_u < thr)}\n\n# Strict ESC 0/1h for Roche hs-cTnT,
backward-compatible return keys\ndef assess_troponin_delta(series):
\n    from datetime import datetime\n    if not series: \n
return {"available": False}\n    # normalize to ng/L and sort by
time\n    ser=[]\n    for ts,v,u in series:\n        if v is None:\n
\n            continue\n            try:\n                v = float(v)\n
if isinstance(u,str) and ("µg/l" in u.lower() or "ug/l" in
u.lower()):\n                v *= 1000.0\n            except Exception:\n
\n                continue\n                ser.append((ts, v))\n    ser =
sorted(ser, key=lambda x: (x[0] or datetime.min))\n    if len(ser) <
2:\n        # one sample only\n        return {"available": True,
"prev": ser[-1][1], "curr": ser[-1][1],\n
"delta_abs": 0.0, "delta_pct": None, "flag": False,\n
"dt_min": None, "rule01h": None, "reason": "needs_serial"}\n\n    # latest sample as t1; search an exact 60-min predecessor\n
t1_ts, t1 = ser[-1]\n    pair = None\n    for j in range(len(ser)-2,
-1, -1):\n        t0_ts, t0 = ser[j]\n        if t0_ts is None or
t1_ts is None:\n            continue\n            dt_min = round((t1_ts
- t0_ts).total_seconds()/60)\n            if dt_min == 60:\n
pair = (t0_ts, t0, dt_min); break\n\n        if pair:\n            _, t0, dt
= pair\n            dv = t1 - t0\n            dp = (abs(dv)/t0*100.0) if t0
else None\n            # ESC 0/1h thresholds (Roche hs-cTnT)\n            if
(t0 >= 52.0) or (dv >= 5.0):\n                label = "rule_in"\n
elif (t0 < 12.0) and (dv < 3.0):\n                label = "rule_out"\n
else:\n                label = "observe"\n            return
{"available": True, "prev": t0, "curr": t1,\n
"delta_abs": dv, "delta_pct": dp, "flag":
(label=="rule_in"),\n
"dt_min": dt, "rule01h":
label, "reason": "ok"}\n\n    # no exact 60-min pair → compute
latest delta for info, but no 0/1h label\n    t0_ts, t0 = ser[-2]\n
dv = t1 - t0\n    dp = (abs(dv)/t0*100.0) if t0 else None\n    gap =
round((t1_ts - (t0_ts or t1_ts)).total_seconds()/60) if t0_ts and
t1_ts else None\n    return {"available": True, "prev": t0,
"curr": t1,\n
"delta_abs": dv, "delta_pct": dp,
"flag": False,\n
"dt_min": gap, "rule01h": None,
"reason": "no_01h_pair"},"metadata":{"execution":
{"iopub.status.busy":"2025-08-25T18:40:39.160253Z","iopub.execute_in
put":"2025-08-25T18:40:39.160602Z","iopub.status.idle":"2025-08-25T1
8:40:39.206602Z","shell.execute_reply.started":"2025-08-25T18:40:39.
160578Z","shell.execute_reply":"2025-08-25T18:40:39.205383Z"},"paper
mill":

```

```

{"duration":0.051195,"end_time":"2025-08-24T12:38:56.234567","exception":false,"start_time":"2025-08-24T12:38:56.183372","status":"completed"},"tags":[],"trusted":true},"outputs":[],"execution_count":82},
{"cell_type":"code","source":"# == Phase-2 bundle helpers (lab
normalization + context) ==\nfrom typing import Dict, Any,
Optional, List, Tuple\n\ndef build_labs_norm(labs: Dict[str, Any])
-> Dict[str, Any]:\n    \"\"\"\n    Map canonical HL7 keys to
calculator-friendly names (presence-guarded).\n    Does NOT mutate
input; returns a new dict.\n    \"\"\"\n    labs = dict(labs or {})\n    out = dict(labs)\n    # Bilirubin: µmol/L (canonical) → mg/dL (calculators)\n    try:\n        if \"lab_bili_total_umolL\" in
labs and \"bilirubin\" not in out:\n            out[\"bilirubin\"] =
float(labs[\"lab_bili_total_umolL\"]) / 17.104\n    except
Exception:\n        pass\n    # Albumin: g/L (canonical) → g/dL
(calculators)\n    try:\n        if \"lab_albumin_gL\" in labs:\n
alb_gdL = float(labs[\"lab_albumin_gL\"]) / 10.0\n    out.setdefault(\"albumin\", alb_gdL)\n
out.setdefault(\"albumin_g_dL\", alb_gdL)\n    except Exception:\n
pass\n    # Electrolytes for anion gap; lactate for Sepsis3
(prefer explicit; fall back to VBG)\n    if \"na\" not in out and
\"vbg_Na\" in labs: out[\"na\"] = labs[\"vbg_Na\"]\n    if \"cl\"
not in out and \"vbg_Cl\" in labs: out[\"cl\"] = labs[\"vbg_Cl\"]\n
if \"hco3\" not in out and \"vbg_HCO3\" in labs: out[\"hco3\"] =
labs[\"vbg_HCO3\"]\n    if \"lactate\" not in out and
\"vbg_lactate\" in labs: out[\"lactate\"] = labs[\"vbg_lactate\"]\n
\n    return out\n\ndef _ctx_age_pregnant(vitals: Dict[str,Any],
context: Optional[Dict[str,Any]]) -> tuple[Optional[int], bool]:\n
context = context or {}\n    age = _pick({**vitals, **context},
[\"age\"], int)\n    pregnant = bool(context.get(\"pregnant\",
False))\n    return age, pregnant\n\n\"metadata\":{\"execution\":
{\"iopub.status.busy\":\"2025-08-25T18:40:39.207808Z\",\"iopub.execute_in
put\":\"2025-08-25T18:40:39.208231Z\",\"iopub.status.idle\":\"2025-08-25T1
8:40:39.232144Z\",\"shell.execute_reply.started\":\"2025-08-25T18:40:39.
208199Z\",\"shell.execute_reply\":\"2025-08-25T18:40:39.231367Z\"},\"paper
mill\":
{\"duration\":0.0218,\"end_time\":\"2025-08-24T12:38:56.267661\",\"exceptio
n\":false,\"start_time\":\"2025-08-24T12:38:56.245861\",\"status\":\"complet
ed\"},\"tags":[],\"trusted\":true,\"outputs\":[],\"execution_count\":83},
{\"cell_type\":\"code\",\"source\":\"# == Vitals normalizer (model-free,
presence-guarded) ==\nimport re, math\nimport pandas as pd\nfrom
typing import Dict, Any, Optional\n\nVITAL_SYNONYMS = {\n
\"SBP\": [\"SBP\", \"RR_sys\", \"BP_sys\", \"systolic\",
\"Systolic\"],\n
\"DBP\": [\"DBP\", \"RR_dia\", \"BP_dia\",
\"diastolic\", \"Diastolic\"],\n
\"HR\": [\"HR\", \"HF\",
\"heart_rate\"],\n
\"SpO2\": [\"SpO2\", \"SpO2\", \"SpO2_pct\",
\"Sats\"],\n
\"RR_rate\": [\"RR_rate\", \"AF\", \"Resp\",
\"respiratory_rate\"], # breaths/min (not blood pressure!)\n
\"Temp_C\": [\"Temp_C\", \"Temp\", \"temperature_c\", \"T\"],\n
\"GCS\": [\"GCS\"],\n
\"RR_str\": [\"RR\", \"RR_mmHg\",
\"RR_str\"], # e.g. \"120/80\"\n\n\ndef
_first(fd: Dict[str,Any], keys) -> Optional[Any]:\n    for k in
keys:\n        if k in fd and fd[k] is not None:\n            return
fd[k]\n    return None\n\ndef _parse_bp_pair(text: str) ->
Optional[tuple]:\n    m = re.search(r\"(\\d{2,3})\\s*/\\s*(\\d{2,3})

```

```

\ ", str(text))\n    if m: return float(m.group(1)),
float(m.group(2))\n    return None\n\ndef normalize_vitals(row:
Dict[str,Any], now_utc: Optional[pd.Timestamp]=None) ->
Dict[str,Any]:\n    \ "\ "\ "\n    Returns canonical vitals independent
of any model bundle:\n    SBP, DBP, HR, SpO2, RR_rate, Temp_C, GCS,
shock_index, hypotension_flag, hypoxia_flag, since_vitals_min.\n
\ "\ "\n    out: Dict[str,Any] = {}\n    # SBP/DBP from direct
fields or \ "RR '120/80' "\n    sbp = _first(row,
_VITAL_SYNONYMS[\ "SBP\ "])\n    dbp = _first(row,
_VITAL_SYNONYMS[\ "DBP\ "])\n    rr_pair = _first(row,
_VITAL_SYNONYMS[\ "RR_str\ "])\n    if (sbp is None or dbp is None)
and rr_pair is not None:\n    p = _parse_bp_pair(rr_pair)\n
if p: sbp = sbp or p[0]; dbp = dbp or p[1]\n    out[\ "SBP\ "] =
float(sbp) if sbp is not None else None\n    out[\ "DBP\ "] =
float(dbp) if dbp is not None else None\n\n    # Other primary
vitals\n    for canon, keys in [(\ "HR\ ", \ "HR\ "),( \ "SpO2\ ", \ "SpO2\ "),(
\ "RR_rate\ ", \ "RR_rate\ "),( \ "Temp_C\ ", \ "Temp_C\ "),( \ "GCS\ ",
\ "GCS\ ")]:\n    val = _first(row, _VITAL_SYNONYMS[canon])\n
out[canon] = float(val) if val is not None else None\n\n    #
Derived, model-free\n    hr = out.get(\ "HR\ "); sbp =
out.get(\ "SBP\ "); spo2 = out.get(\ "SpO2\ ")\n    out[\ "shock_index\ "]
= (hr/sbp) if (isinstance(hr,(int,float)) and isinstance(sbp,
(int,float)) and sbp>0) else None\n    out[\ "hypotension_flag\ "] =
(sbp is not None and sbp < 90)\n    out[\ "hypoxia_flag\ "] = (spo2
is not None and spo2 < 90)\n\n    # Age/sex NEVER used here (bias
hygiene); gates don't touch protected attrs.\n    #
since_vitals_min: prefer provided timestamp, else compute from
'vitals_ts' or now\n    now = now_utc or pd.Timestamp.utcnow()\n
ts = raw.get(\ "vitals_ts\ ") or raw.get(\ "ts\ ") # accept either if
present\n    try:\n    tsv = pd.to_datetime(ts) if ts is not
None else None\n    except Exception:\n    tsv = None\n
out[\ "since_vitals_min\ "] = (now - tsv).total_seconds()/60.0 if tsv
is not None else None\n    return out\n", "metadata": {"execution":
{"iopub.status.busy": "2025-08-25T18:40:39.233098Z", "iopub.execute_in
put": "2025-08-25T18:40:39.234063Z", "iopub.status.idle": "2025-08-25T1
8:40:39.258615Z", "shell.execute_reply.started": "2025-08-25T18:40:39.
234038Z", "shell.execute_reply": "2025-08-25T18:40:39.257517Z"}, "paper
mill":
{"duration": 0.025999, "end_time": "2025-08-24T12:38:56.304509", "except
ion": false, "start_time": "2025-08-24T12:38:56.278510", "status": "compl
eted"}, "tags": [], "trusted": true}, "outputs": [], "execution_count": 84},
{"cell_type": "code", "source": "# == Phase-2 bundle core scores
(pure) ==\nfrom typing import Dict, Any, Optional\n\ndef
compute_phase2_scores(vitals: Dict[str,Any], labs_norm:
Dict[str,Any], context: Optional[Dict[str,Any]]=None) ->
Dict[str,Any]:\n    context = context or {}\n\n    s_qsofa =
calc_qsoFA(vitals) if 'calc_qsoFA' in globals() else
calc_qsofa(vitals) # tolerate minor naming diffs\n    s_mews =
calc_mews(vitals)\n    s_heart = calc_heart(**vitals, **labs_norm)\n
\n    s_grace = calc_grace_coarse(**vitals, **labs_norm)\n
s_sofa = calc_sofa_min(vitals, labs_norm)\n    sirs =
calc_sirs(**vitals, **labs_norm)\n    sepsis3 =
sepsis3_screen(vitals, labs_norm, context, s_sofa, s_qsofa)\n\n
wells_pe = calc_wells_pe(**vitals, **labs_norm, **context)\n

```

```

wells_dvt = calc_wells_dvt(**vitals, **labs_norm, **context)\n
perc      = calc_perc(**vitals, **labs_norm, **context)\n      pesi
= calc_pesi(**vitals, **labs_norm, **context)\n      spesi
= calc_spesi(**vitals, **labs_norm, **context)\n      marburg
= calc_marburg(**vitals, **labs_norm, **context)\n      gbs
= calc_gbs(**vitals, **labs_norm, **context)\n      childpugh
= calc_child_pugh(**vitals, **labs_norm, **context)\n\n      return
{\n      \n      \"qsofa\": s_qsofa, \"mews\": s_mews, \"heart\": s_heart,
\"grace\": s_grace, \"sofa\": s_sofa,\n      \n      \"sirs\": sirs,
\"sepsis3\": sepsis3,\n      \n      \"wells_pe\": wells_pe,
\"wells_dvt\": wells_dvt, \"perc\": perc,\n      \n      \"pesi\": pesi,
\"spesi\": spesi,\n      \n      \"marburg\": marburg, \"gbs\": gbs,
\"childpugh\": childpugh\n      }\n\", \"metadata\": {\"execution\":
{\"iopub.status.busy\": \"2025-08-25T18:40:39.259641Z\", \"iopub.execute_in
put\": \"2025-08-25T18:40:39.259968Z\", \"iopub.status.idle\": \"2025-08-25T1
8:40:39.281944Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:39.
259940Z\", \"shell.execute_reply\": \"2025-08-25T18:40:39.280995Z\"}, \"paper
mill\":
{\"duration\": 0.022564, \"end_time\": \"2025-08-24T12:38:56.338115\", \"except
ion\": false, \"start_time\": \"2025-08-24T12:38:56.315551\", \"status\": \"compl
eted\", \"tags\": [], \"trusted\": true}, \"outputs\": [], \"execution_count\": 85},
{\"cell_type\": \"code\", \"source\": \"# == Phase-2 bundle (assembler)
==\nfrom typing import Dict, Any, Optional, List, Tuple\nfrom
datetime import datetime\n\ndef phase2_bundle(vitals: Dict[str, Any],
labs: Dict[str, Any], context: Optional[Dict[str, Any]]=None) ->
Dict[str, Any]:\n    context = context or {}\n    labs_norm =
build_labs_norm(labs)\n    age, pregnant = _ctx_age_pregnant(vitals,
context)\n\n    # Scores\n    sc = compute_phase2_scores(vitals,
labs_norm, context)\n    s_qsofa=sc[\"qsofa\"]; s_mews=sc[\"mews\"];
s_heart=sc[\"heart\"]; s_grace=sc[\"grace\"]; s_sofa=sc[\"sofa\"];
sirs=sc[\"sirs\"]; sepsis3=sc[\"sepsis3\"]\n
wells_pe=sc[\"wells_pe\"]; wells_dvt=sc[\"wells_dvt\"];
perc=sc[\"perc\"]\n    pesi=sc[\"pesi\"]; spesi=sc[\"spesi\"];
marburg=sc[\"marburg\"]; gbs=sc[\"gbs\"];
childpugh=sc[\"childpugh\"]\n\n    # D-dimer (canonical mg/L FEU or
legacy µg/L FEU)\n    if \"lab_ddimer_mgL_FEU\" in labs_norm:\n
d_val = float(labs_norm[\"lab_ddimer_mgL_FEU\"]);\n    d_unit = \"mg/L
FEU\"\n    else:\n        d_val = _pick(labs_norm,
[\"d_dimer_feu_ug_l\", \"d_dimer\", \"ddimer\", \"d-dimer\"])\n
d_unit = labs_norm.get(\"d_dimer_unit\") or \"µg/L FEU\"\n
d_assess = assess_d_dimer(d_val, d_unit, age, pregnant)\n    if d_val is
not None else {\"available\": False}\n\n    # Troponin Δ (legacy
series only)\n    series: List[Tuple[Optional[datetime], float,
str]] = []\n    for it in (labs_norm.get(\"troponin_series\") or
[]):\n        if isinstance(it, dict):\n            ts=None\n
if it.get(\"time\"):\n                try: ts =
datetime.fromisoformat(str(it[\"time\"]).replace(\"Z\", \"\"))\n
except Exception: ts=None\n                series.append((ts, _pick(it,
[\"value\", \"val\"]), it.get(\"unit\", \"ng/L\")))\n            t_assess =
assess_troponin_delta(series)\n    if len(series) >= 2 else
{\"available\": False}\n\n    # Corrections\n    ca_corr =
corrected_calcium(\n        _pick(labs_norm, [\"calcium\", \"ca\",
\"calcium_mg_dl\"])\n        ,\n        _pick(labs_norm, [\"albumin\", \"alb\",
\"albumin_g_dl\"])\n        ,\n        units=\"mg/dL\"\n        )\n    ag_val =

```

```

anion_gap(\n          _pick(labs_norm, [\"na\", \"sodium\"]), \n
_pick(labs_norm, [\"cl\", \"chloride\"]), \n          _pick(labs_norm,
[\"hco3\", \"bicarbonate\"]), \n          k=_pick(labs_norm, [\"k\",
\"potassium\"]), \n          albumin_gdl=_pick(labs_norm, [\"albumin\",
\"alb\", \"albumin_g_dl\"]) \n          ) \n          # One-liners \n          ones = [] \n
ones.append(f\"qSOFA={s_qsofa['score']} MEWS={s_mews['score']}
GRACE(coarse)={s_grace['score']}\") \n
ones.append(f\"HEART={s_heart['score']} SOFA(min)={s_sofa['score']}
\") \n          if d_assess.get(\"available\"): \n          ones.append(\n
f\"D-dimer {int(d_assess['value_ug_per_l'])} vs thr
{int(d_assess['thr_ug_per_l'])} µg/L → \" \n          f\"{'OK' if
d_assess['ok_below_thr'] else 'High'}\" \n          ) \n          # --- hs-
cTnT (Roche) ESC 0/1h one-liner (strict 60 min) --- \n          if
t_assess.get(\"available\"): \n          r = t_assess.get(\"rule01h\")
\n          if r is None: \n          dt = t_assess.get(\"dt_min\") \n
ones.append(\n          \"hs-cTnT 0/1h: OBSERVE - \" + \n
(\"needs serial\" if dt is None else f\"no 60-min pair (latest
gap={int(dt)} min)\") \n          ) \n          else: \n          lbl
= r.replace(\"_\", \"-\").upper() # RULE-IN | RULE-OUT | OBSERVE \n
ones.append(\n          f\"hs-cTnT 0/1h: {lbl}
(t0={t_assess['prev']:.0f}, t1={t_assess['curr']:.0f}, \" \n
f\"Δ={t_assess['delta_abs']:.0f} ng/L; 60 min)\") \n          ) \n          \n
ones.append(\n          f\"Wells-PE={_safe_round(wells_pe['score'], 1)}
({wells_pe['tier2']}) \" \n          f\"Wells-DVT={wells_dvt['score']}
({wells_dvt['tier2']})\" \n          ) \n          ones.append(f\"PERC={'pass' if
perc['passed'] else 'fail'} sPESI={spesi['score']}
PESI={pesi['class']}/{pesi['score']}\") \n          ones.append(\n
f\"SIRS={sirs['score']} Sepsis3: {'YES' if sepsis3['sepsis_flag']
else 'no'} \" \n          f\"Shock: {'YES' if sepsis3['septic_shock']
else 'no'}\" \n          ) \n          ones.append(f\"MARBURG={marburg['score']}/5
GBS={gbs['score']}\") \n          if childpugh.get(\"class\") ==
\"incomplete\": \n          ones.append(\"Child-Pugh incomplete (need
5/5 inputs)\") \n          else: \n          ones.append(f\"Child-Pugh
{childpugh['class']} ({childpugh['score']})\" \n          ) \n          if ca_corr is
not None: \n          ones.append(f\"Corrected
Ca={_safe_round(ca_corr, 2)} mg/dL\" \n          ) \n          if ag_val is not None: \n
ab = f\", AGcorr={ag_val['ag_albumin_corrected']}\", if
ag_val.get(\"ag_albumin_corrected\") is not None else \"\" \n
ones.append(f\"Anion gap={ag_val['ag']}{ab}\") \n          \n          # --- build
payload (add troponin timing + gate signals) --- \n          out = {\n
\"one_liners\": ones, \n          \"scores\": {\n          \"qsofa\":
s_qsofa[\"score\"], \"mews\": s_mews[\"score\"], \"heart\":
s_heart[\"score\"], \"grace_coarse\": s_grace[\"score\"], \n
\"sofa_min\": s_sofa[\"score\"], \"wells_pe\": wells_pe[\"score\"],
\"wells_dvt\": wells_dvt[\"score\"], \n          \"pesi\":
pesi[\"score\"], \"spesi\": spesi[\"score\"], \"sirs\":
sirs[\"score\"], \n          \"sepsis3\":
int(sepsis3[\"sepsis_flag\"]), \"gbs\": gbs[\"score\"], \"marburg\":
marburg[\"score\"], \n          \"child_pugh\":
childpugh.get(\"score\") or childpugh.get(\"score_partial\")
\n          }, \n          \"labs\": {\n          \"d_dimer\":
(d_assess.get(\"value_ug_per_l\") if d_assess.get(\"available\")
else None), \n          \"d_dimer_thr\":
d_assess.get(\"thr_ug_per_l\"), \n          \"trop_prev\":

```

```

t_assess.get("\prev\"), \trop_curr\": t_assess.get("\curr\"),\n
\trop_delta\": t_assess.get("\delta_abs\"), \trop_delta_pct\":
t_assess.get("\delta_pct\"),\n          \trop_flag\":
t_assess.get("\flag\"),\n          \trop_rule01h\":
t_assess.get("\rule01h\"),\n          \trop_dt_min\":
t_assess.get("\dt_min\"),\n          },\n          }\n\n      # minimal
advice-only signals for later gate usage\n
out.setdefault("\signals\",{}}\n      out["signals\"]
["acs_rulein\"]= (t_assess.get("\rule01h\")== "\rule_in\")\n
out["signals\"][\acs_ruleout\"]= (t_assess.get("\rule01h\")==
"\rule_out\")\n\n      return out\n", "metadata":{"execution":
{"iopub.status.busy":"2025-08-25T18:40:39.283106Z","iopub.execute_in
put":"2025-08-25T18:40:39.283433Z","iopub.status.idle":"2025-08-25T1
8:40:39.310205Z","shell.execute_reply.started":"2025-08-25T18:40:39.
283402Z","shell.execute_reply":"2025-08-25T18:40:39.309290Z"},"paper
mill":
{"duration":0.030003,"end_time":"2025-08-24T12:38:56.379069","except
ion":false,"start_time":"2025-08-24T12:38:56.349066","status":"compl
eted"},"tags":[],"trusted":true},"outputs":[],"execution_count":86},
{"cell_type":"code","source":"# ---- UI (ipywidgets) + HL7 merge
----\nif CONFIG.get("\RUN_UI\"): \n      try: \n          import
ipywidgets as W, pandas as pd, json\n          EVENT_LOG_PATH =
CONFIG.get("\EVENT_LOG_PATH\",""/mnt/data/event_log.jsonl\")\n\n
# --- Text areas ---\n          vitals_in = W.Textarea(\n
value='{\"rr\":24,\"sbp\":95,\"gcs\":14,\"hr\":120,\"temp\":38.6,\"a
vpu\": \"V\", \"map\":65,\"age\":68,\"sex\": \"M\", \"sao2\":97}',\n
description=\"Vitals JSON\", layout=W.Layout(width=\"100%\",
height=\"90px\")\n          )\n          labs_in = W.Textarea(\n
value='{\"d_dimer\":780,\"d_dimer_unit\": \"µg/L FEU\",
\"troponin_series\": [{\"time\": \"2025-08-20T05:00:00Z\",
\"value\":18,\"unit\": \"ng/L\"}, {\"time\": \"2025-08-20T09:00:00Z\",
\"value\":78,\"unit\": \"ng/L\"}],
\"creatinine\":1.8,\"platelets\":95,\"albumin\":2.8,\"calcium\":7.9,
\"na\":138,\"cl\":105,\"hco3\":20,\"bilirubin\":2.1,\"inr\":1.9}',\n
description=\"Labs JSON\", layout=W.Layout(width=\"100%\",
height=\"130px\")\n          )\n          ctx_in = W.Textarea(\n
value='{\"pregnant\": false, \"suspected_infection\": true,
\"vasopressors\": false, \"pe_most_likely\": true, \"dvt_signs\":
false, \"ascites\": \"mild\", \"encephalopathy\": \"1-2\"}',\n
description=\"Context JSON\", layout=W.Layout(width=\"100%\",
height=\"90px\")\n          )\n\n          # --- HEART component inputs
(0-2) ---\n          heart_hist = W.Dropdown(options=[0,1,2], value=0,
description=\"HEART: History\")\n          heart_ecg =
W.Dropdown(options=[0,1,2], value=0, description=\"HEART: ECG\")\n
heart_risk = W.Dropdown(options=[0,1,2], value=0,
description=\"HEART: Risk\")\n\n          # --- PE / PERC flags ---\n
chk_pe_most = W.Checkbox(description=\"PE most likely\",
value=True)\n          chk_dvt_signs= W.Checkbox(description=\"DVT
signs\", value=False)\n          chk_immob =
W.Checkbox(description=\"Immobilized / recent surgery (4w)\",
value=False)\n          chk_prev_vte = W.Checkbox(description=\"Prior
VTE\", value=False)\n          chk_hemo =
W.Checkbox(description=\"Hemoptysis\", value=False)\n
chk_cancer = W.Checkbox(description=\"Active malignancy\",

```

```

value=False)\n                chk_estrogen =
W.Checkbox(description="\Estrogen use (PERC)\", value=False)\n
chk_trauma = W.Checkbox(description="\Recent trauma (4w, PERC)\",
value=False)\n                chk_unilat =
W.Checkbox(description="\Unilateral leg swelling (PERC)\",
value=False)\n\n                # --- Wells-DVT flags ---\n
dvt_cancer = W.Checkbox(description="\Active cancer\",
value=False)\n                dvt_paresis =
W.Checkbox(description="\Paresis / plaster cast\", value=False)\n
dvt_bed_surg = W.Checkbox(description="\Bedridden ≥3d / surgery
≤12w\", value=False)\n                dvt_tender =
W.Checkbox(description="\Deep vein tenderness\", value=False)\n
dvt_entire = W.Checkbox(description="\Entire leg swollen\",
value=False)\n                dvt_calf3 = W.Checkbox(description="\Calf
swelling >3 cm\", value=False)\n                dvt_edema =
W.Checkbox(description="\Pitting edema (symptomatic leg)\",
value=False)\n                dvt_collat =
W.Checkbox(description="\Collateral non-varicose\", value=False)\n
dvt_prev = W.Checkbox(description="\Previous DVT\", value=False)\n
\n                dvt_alt_dx = W.Checkbox(description="\Alternative dx as
likely (subtract)\", value=False)\n\n                # --- HL7 paste area
---\n                hl7_in = W.Textarea(\n                value='''OBX|1|NM|
BILIRUBIN TOTAL||36|umol/L|||N||F|||20250820090000\nOBX|2|NM|INR||
1.9|||N||F|||20250820090000\nOBX|3|NM|ALBUMIN||28|g/L|||N||F|||
20250820090000\nOBX|4|NM|D-DIMER||780|ug/L|||H||F|||
20250820090000\nOBX|5|NM|TROPONIN T||78|ng/L|||H||F|||
20250820090000\nOBX|6|NM|TROPONIN T||18|ng/L|||N||F|||
20250820050000''',\n                description="\HL7 paste\",
layout=W.Layout(width="\100%\", height="\140px\")\n                )\n\n
btn_apply_hl7 = W.Button(description="\Apply HL7 → Labs JSON\")\n
btn_compute = W.Button(description="\Compute + Log\")\n                out
= W.Output()\n\n                def on_apply(_):\n                with out:\n
out.clear_output()\n                try:\n                labs =
json.loads(labs_in.value or \"{}\")\n                except
Exception as e:\n                print("\[labs parse error]\",
e)\n                return\n                parsed =
parse_hl7_labs(hl7_in.value or \"\")\n                # Merge core
others\n                oth = parsed.get("\others\") or {}\n
for k in [\"bilirubin\", \"inr\", \"albumin\"]:\n
if oth.get(k) is not None:\n                labs[k] = oth[k]\n
\n                # If available, set D-dimer (take newest) when
missing\n                dd = parsed.get("\d_dimer\") or []\n
if dd and \"d_dimer\" not in labs:\n                dd_sorted =
sorted(dd, key=lambda t: (t[0] or 0))\n                _, v, u =
dd_sorted[-1]\n                labs[\"d_dimer\"] = v\n
labs[\"d_dimer_unit\"] = u\n                # Always (re)write
troponin_series if present\n                ts =
parsed.get("\troponin\") or []\n                if ts:\n
series = []\n                for t, v, u in ts:\n
series.append({\n                \"time\":
(t.isoformat()+\"Z\") if t else None,\n                \"value\": v, \"unit\": u or \"ng/L\"
}})\n                labs[\"troponin_series\"] = series\n
json.dumps(labs, ensure_ascii=False)\n                print("\HL7

```

```
merged->\", {k: labs.get(k) for k in [\"bilirubin\", \"inr\",  
\"albumin\", \"d_dimer\", \"d_dimer_unit\"]})\n\n        def  
on_compute(_):\n        with out:\n        out.clear_output()\n        try:\n        vitals  
= json.loads(vitals_in.value or \"{ }\")\n        labs  
= json.loads(labs_in.value or \"{ }\")\n        ctx =  
json.loads(ctx_in.value or \"{ }\")\n        except Exception  
as e:\n        print(\"[parse error]\", e)\n        return\n        # HEART sub-scores from dropdowns\n        vitals[\"heart_history\"] = int(heart_hist.value)\n        vitals[\"heart_ecg\"] = int(heart_ecg.value)\n        vitals[\"heart_risk\"] = int(heart_risk.value)\n        # PE / PERC + DVT flags\n        ctx.update({\n        \"pe_most_likely\": chk_pe_most.value,\n        \"dvt_signs\": chk_dvt_signs.value,\n        \"immobilized\": chk_immob.value,\n        \"recent_surgery_4w\": chk_immob.value,\n        \"prev_vte\": chk_prev_vte.value,\n        \"hemoptysis\": chk_hemo.value,\n        \"cancer_active\": chk_cancer.value or ctx.get(\"cancer_active\",  
False),\n        \"estrogen_use\": chk_estrogen.value,\n        \"recent_trauma_4w\": chk_trauma.value,\n        \"unilateral_leg_swelling\": chk_unilat.value,\n        # Wells-DVT details\n        \"paresis\":  
dvt_paresis.value,\n        \"plaster_cast\":  
dvt_paresis.value,\n        \"bedridden_3d\":  
dvt_bed_surg.value,\n        \"surgery_12w\":  
dvt_bed_surg.value,\n        \"deep_vein_tenderness\":  
dvt_tender.value,\n        \"entire_leg_swollen\":  
dvt_entire.value,\n        \"calf_swelling_gt3cm\":  
dvt_calf3.value,\n        \"pitting_edema\":  
dvt_edema.value,\n        \"collateral_nonvaricose\":  
dvt_collat.value,\n        \"prev_dvt\": dvt_prev.value,\n        \"alt_dx_as_likely\": dvt_alt_dx.value,\n        }\n        )\n        bundle =  
phase2_bundle(vitals, labs, ctx)\n        # ML risk (only  
if bridge loaded)\n        if \"predict_one\" in globals():  
and \"FEAT\" in globals():\n        feats = FEAT\n        row = {f: labs.get(f, vitals.get(f, ctx.get(f, 0.0))) for f in  
feats}\n        res = predict_one(row)\n        bundle[\"one_liners\"].append(\n        f\"ML risk  
p={res['p']:.3f} -> {'ALERT' if res['y'] else 'ok'}  
(thr={res['thr']:.3f})\"\n        )\n        try:\n        _append_event({\"type\": \"ml_score\",  
\"keys\": list(row.keys()), \"res\": res})\n        except Exception:\n        pass\n        else:\n        bundle[\"one_liners\"].append(\"ML risk:  
(bridge not loaded)\"\n        )\n        _append_event({\"type\":  
\"phase2_bundle\", \"bundle\": bundle})\n        print(\"\\n  
\\n\".join(bundle[\"one_liners\"])\n        )\n        print(\"\\n  
\\nLogged to:\", EVENT_LOG_PATH)\n        btn_apply_hl7.on_click(on_apply)\n        btn_compute.on_click(on_compute)\n        display(W.VBox([  
W.HTML(\"<b>Phase-2 calculators - optimized UI (PE/DVT/PERC, PESI/
```

```

sPESI, SIRS/Sepsis3, HEART, GBS, Child-Pugh)</b>\"),\n
W.HBox([vitals_in, labs_in]),\n                W.HBox([heart_hist,\nheart_ecg, heart_risk]),\n                W.HTML(\"<b>PE / PERC flags</b>\"),\n                W.HBox([chk_pe_most, chk_dvt_signs, chk_immob,\nchk_prev_vte, chk_hemo, chk_cancer]),\n                W.HBox([chk_estrogen, chk_trauma, chk_unilat]),\n                W.HTML(\"<b>Wells-DVT flags</b>\"),\n                W.HBox([dvt_cancer,\ndvt_paresis, dvt_bed_surg, dvt_tender, dvt_entire]),\n                W.HBox([dvt_calf3, dvt_edema, dvt_collat, dvt_prev, dvt_alt_dx]),\n                ctx_in,\n                W.HBox([btn_compute, btn_apply_hl7]),\n                W.HTML(\"<b>HL7 Labs (Trop / D-Dimer / INR / Bili / Albumin)</b>\"),\n                hl7_in,\n                out\n                ]))\n    except\nException as e:\n    print(\"Phase-2 UI unavailable:\", e)\n\nelse:\n    # Text-mode smoke test (runs if RUN_UI=False)\n    vit\n    =\n    {\n        \"rr\":24,\"sbp\":95,\"gcs\":14,\"hr\":120,\"temp\":38.6,\"avpu\":\n        \"V\", \"map\":65,\"age\":68,\"sex\":\"M\", \"sao2\":97,\n        \"heart_history\":0,\"heart_ecg\":0,\"heart_risk\":0}\n        lab =\n        {\n            \"d_dimer\":780,\"d_dimer_unit\":\"µg/L FEU\", \n            \"troponin_series\":[\n                {\n                    \"time\":\"2025-08-20T05:00:00Z\", \n                    \"value\":18,\"unit\":\"ng/L\"},\n                {\n                    \"time\":\"2025-08-20T09:00:00Z\", \n                    \"value\":78,\"unit\":\"ng/L\"}\n                ],\n            \"creatinine\":1.8,\"platelets\":95,\"albumin\":2.8,\"calcium\":7.9,\n            \"na\":138,\"cl\":105,\"hco3\":20,\"bilirubin\":2.1,\"inr\":1.9}\n        ctx = {\n            \"pregnant\": True, \"suspected_infection\": True,\n            \"vasopressors\": False, \"pe_most_likely\": True, \"dvt_signs\":\n            False,\n            \"ascites\":\"mild\", \"encephalopathy\":\"1-2\"}\n        b = phase2_bundle(vit, lab, ctx)\n        _append_event({\"type\":\n        \"phase2_bundle_smoke\", \"bundle\": b})\n        print(\n        \"\\n\".join(b[\"one_liners\"])))\n        print(\"Logged to:\",\n        CONFIG.get(\"EVENT_LOG_PATH\", \"./mnt/data/event_log.jsonl\"))\n        \"n\", \"metadata\":{\n            \"execution\":\n            {\n                \"iopub.status.busy\":\"2025-08-25T18:40:39.311222Z\", \"iopub.execute_in\n            put\":\"2025-08-25T18:40:39.311563Z\", \"iopub.status.idle\":\"2025-08-25T1\n            8:40:39.404695Z\", \"shell.execute_reply.started\":\"2025-08-25T18:40:39.\n            311536Z\", \"shell.execute_reply\":\"2025-08-25T18:40:39.403662Z\"}, \"paper\n            mill\":\n            {\n                \"duration\":0.124553, \"end_time\":\"2025-08-24T12:38:56.514748\", \"except\n            ion\":false, \"start_time\":\"2025-08-24T12:38:56.390195\", \"status\": \"compl\n            eted\", \"tags\": [], \"trusted\":true}, \"outputs\":\n            [\n                {\n                    \"output_type\": \"display_data\", \"data\": {\n                        \"text/\n                        plain\": \"VBox(children=(HTML(value='<b>Phase-2 calculators -\n                        optimized UI (PE/DVT/PERC, PESI/sPESI, SIRS/Sepsis3,\n                        HEART...\", \"application/vnd.jupyter.widget-view+json\":\n                        {\n                            \"version_major\":2, \"version_minor\":0, \"model_id\": \"04b7b8726a994cfca7a\n                        bb4dec5c44230\"}, \"metadata\": {}}, \"execution_count\":87},\n                        {\n                            \"cell_type\": \"code\", \"source\": \"# --- Advice-only \\\"Next steps\\\" panel\n                        (ultra-minimal, no extra deps) ---\\nimport ipywidgets as W, json,\n                        html\\nfrom pathlib import Path\\n\\ndef _tail_phase2_bundle(n=120):\\n\n                        p = Path(CONFIG.get(\"EVENT_LOG_PATH\", \"./mnt/data/\n                        event_log.jsonl\"))\\n\n                        if not p.exists():\\n\n                            return None\\n\n                        lines = p.read_text().splitlines()[-n:]\\n\n                        for ln in\n                        reversed(lines):\\n\n                            try:\\n\n                                ev = json.loads(ln)\\n\n                                if ev.get(\"type\") in {\"phase2_bundle\", \"phase2_bundle_smoke\"}:

```

```

\n                return ev.get(\"bundle\") or ev.get(\"result\") or
{ }\n        except Exception:\n            pass\n            return
None\n\nndef _advice_from_bundle(b):\n    b = b or { }\n    sig =
b.get(\"signals\") or { }\n    labs = b.get(\"labs\") or { }\n    tips
= [ ]\n\n    # ACS pathway (advice-only, requires human approval for
any order/transfer)\n    if sig.get(\"acs_rulein\"):\n        tips
+= [ \n            \"Page cardiology (advice-only); requires human
approval.\",\n            \"Prepare cath-lab pre-checklist +
contraindications form (advice-only).\", \n            \"Confirm
serial ECG monitoring is active; notify if new ischemic changes.\",
\n            ]\n    elif sig.get(\"acs_ruleout\"):\n        tips += [ \n
\"De-prioritize ACS pathway; continue evaluation of non-ACS causes
per SOP.\",\n            \"If clinically stable and risks low, start
discharge planning checklist (advice-only).\", \n            ]\n    else:
\n        dt = labs.get(\"trop_dt_min\")\n        if dt is None:\n
tips.append(\"Schedule 1h hs-cTnT exactly at 60 minutes from
baseline (align phlebotomy timing).\")\n        else:\n
tips.append(f\"No exact 60-min pair (latest gap={int(dt)} min).
Align labs to strict 0/1h protocol.\")\n
tips.append(\"Continue observation and supportive care per ED SOP
(advice-only).\")\n\n    # Always include coordination hygiene\n
tips += [ \n            \"Broadcast a 1-line status update to ED team
(advice-only).\", \n            \"Append these suggestions to the audit
log.\", \n            ]\n    return tips\n\n# UI\nbtn =
W.Button(description=\"Next steps (advice-only)\",
button_style=\"info\", icon=\"check\")\nout = W.Output()\n\nndef
_on_click(_):\n    with out:\n        out.clear_output()\n        #
Prefer recomputing from current UI inputs if available\n
bundle = None\n        try:\n            vit =
json.loads(vitals_in.value)\n            lab =
json.loads(labs_in.value)\n            ctx =
json.loads(ctx_in.value)\n            bundle = phase2_bundle(vit,
lab, ctx)\n        except Exception:\n            bundle =
_tail_phase2_bundle()\n\n        if not bundle:\n
print(\"No bundle available yet – run the calculators first.\")\n
return\n\n        tips = _advice_from_bundle(bundle)\n
display(W.HTML(f\"<b>Advice-only next steps</b>\"))\n        items =
\"\".join(f\"<li>{html.escape(t)}</li>\" for t in tips)\n
display(W.HTML(f\"<ul>{items}</ul>\"))\n\nbtn.on_click(_on_click)
\ndisplay(W.VBox([btn, out]))\n\n\", \"metadata\":
{ \"trusted\": true, \"execution\":
{ \"iopub.status.busy\": \"2025-08-25T18:40:39.407863Z\", \"iopub.execute_in
put\": \"2025-08-25T18:40:39.408227Z\", \"iopub.status.idle\": \"2025-08-25T1
8:40:39.431591Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:39.
408196Z\", \"shell.execute_reply\": \"2025-08-25T18:40:39.430655Z\" } }, \"outp
uts\": [ { \"output_type\": \"display_data\", \"data\": { \"text/
plain\": \"VBox(children=(Button(button_style='info', description='Next
steps (advice-only)', icon='check', style=ButtonS...\", \"application/
vnd.jupyter.widget-view+json\":
{ \"version_major\": 2, \"version_minor\": 0, \"model_id\": \"9a570d024948480ea06
a94652e6a33fc\" } }, \"metadata\": { } }, \"execution_count\": 88 },
{ \"cell_type\": \"code\", \"source\": \"def gate_engine_with_capacity(fd,
CONFIG):\n    return merge_gatepacks(gate_engine(fd, CONFIG),
capacity_gatepack(fd, CONFIG))\n\nfd = { \"suspected_condition\":

```

```

\"respiratory\",
\"cap_internal_medicine_icu\":0.0,\"ems_triage_code\":1,\"admit_decision\": \"ICU\", \"Triage\": \"Orange\"}\npack = capacity_gatepack(fd, CONFIG)\nprint(pack[\"gates\"], \"|\", pack[\"next_action\"])\n# Expect ICU_BLOCKED, ED_BOARDING_RISK, ROUTE_PLAN_ED_TO_ICU, ALERT_* , TRANSFER_BLOCK_CAPACITY\n\", \"metadata\": {\"execution\": {\"iopub.status.busy\": \"2025-08-25T18:40:39.433079Z\", \"iopub.execute_input\": \"2025-08-25T18:40:39.433422Z\", \"iopub.status.idle\": \"2025-08-25T18:40:39.451843Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:39.433384Z\", \"shell.execute_reply\": \"2025-08-25T18:40:39.450814Z\"}, \"papermill\": {\"duration\": 0.019323, \"end_time\": \"2025-08-24T12:38:56.545422\", \"exception\": false, \"start_time\": \"2025-08-24T12:38:56.526099\", \"status\": \"completed\"}, \"tags\": [], \"trusted\": true}, \"outputs\": [{\"name\": \"stdout\", \"text\": \"['ICU_BLOCKED', 'ED_BOARDING_RISK', 'ROUTE_PLAN_ED_TO_ICU', 'ALERT_NURSE', 'ALERT_DOC', 'ALERT_ATTENDING', 'TRANSFER_BLOCK_CAPACITY'] | No ICU capacity - board in ED, activate bed manager now.\n\", \"output_type\": \"stream\"}], \"execution_count\": 89}, {\"cell_type\": \"code\", \"source\": \"# Force-sync all \\\"Context JSON\\\" textareas and the reference used by YEARS\nimport json, ipywidgets as W, gc\n\ndef _update_all_context_widgets(flag=True):\n    new_val = None\n    # If a ctx_in exists in this scope, start from it\n    try:\n        d = json.loads(ctx_in.value or '{}')\n    except Exception:\n        d = {}\n    d[\"pregnant\"] = bool(flag)\n    new_val = json.dumps(d, ensure_ascii=False)\n    # Update any Textarea whose description starts with \\\"Context JSON\\\"\n    n = 0\n    for obj in gc.get_objects():\n        try:\n            if isinstance(obj, W.Textarea) and (obj.description or '').startswith(\"Context JSON\"):\n                obj.value = new_val\n                n += 1\n        except Exception:\n            pass\n    print(f\"Updated {n} Context JSON widget(s) ->\", new_val)\n\n    # Make sure the YEARS cell reads the same widget reference\n    global ctx_src # used inside the YEARS cell handler\n    try:\n        ctx_src = ctx_in\n        print(\"ctx_src -> ctx_in (bound)\")\n    except NameError:\n        print(\"ctx_in not in scope; YEARS will still read its own ctx_src if present.\")\n\n_update_all_context_widgets(flag=True)\n\", \"metadata\": {\"execution\": {\"iopub.status.busy\": \"2025-08-25T18:40:39.453076Z\", \"iopub.execute_input\": \"2025-08-25T18:40:39.453703Z\", \"iopub.status.idle\": \"2025-08-25T18:40:39.528791Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:39.453672Z\", \"shell.execute_reply\": \"2025-08-25T18:40:39.527758Z\"}, \"papermill\": {\"duration\": 0.075072, \"end_time\": \"2025-08-24T12:38:56.631630\", \"exception\": false, \"start_time\": \"2025-08-24T12:38:56.556558\", \"status\": \"completed\"}, \"tags\": [], \"trusted\": true}, \"outputs\": [{\"name\": \"stdout\", \"text\": \"Updated 1 Context JSON widget(s) -> {\\\"pregnant\\\": true, \\\"suspected_infection\\\": true, \\\"vasopressors\\\": false, \\\"pe_most_likely\\\": true, \\\"dvt_signs\\\": false, \\\"ascites\\\": \\\"mild\\\", \\\"encephalopathy\\\": \\\"1-2\\\"}\nctx_src -> ctx_in (bound)\", \"output_type\": \"stream\"}], \"execution_count\": 90}, {\"cell_type\": \"code\", \"source\": \"\n# == Pregnancy-adapted YEARS pathway (drop-in) ==\nfrom datetime import datetime,

```

```

timezone\nimport json\n\ndef assess_pregnancy_years(ctx, labs):\n
# Preg-adapted YEARS: three items – DVT signs, hemoptysis, 'PE most
likely'.\n    # If none present → D-dimer threshold 1000 µg/L FEU;
else threshold 500 µg/L FEU.\n    preg = bool(ctx.get(\"pregnant\",
False))\n    items = {\n        \"dvt_signs\":
bool(ctx.get(\"dvt_signs\", False)),\n        \"hemoptysis\":
bool(ctx.get(\"hemoptysis\", False)),\n        \"pe_most_likely\":
bool(ctx.get(\"pe_most_likely\", False)),\n    }\n    count =
sum(int(v) for v in items.values())\n    d_val = None\n    for k in
(\"d_dimer\", \"ddimer\", \"d-dimer\", \"d_dimer_feu_ug_l\"):\n
if k in labs and labs[k] not in (None, \"\"):\n        try:\n
d_val = float(str(labs[k]).replace(\"\", \"\", \".\"))\n
break\n        except Exception:\n            pass\n
d_unit = (labs.get(\"d_dimer_unit\") or \"µg/L FEU\").lower()\n
if d_val is None:\n        return {\n            \"applies\": preg,\n
            \"needs_imaging\": True, \"reason\": \"missing D-dimer\", \"items\":
items, \"items_count\": count}\n        d_ug = float(d_val)*1000.0 if
\"mg/l\" in d_unit else float(d_val)\n        thr = 1000.0 if count==0
else 500.0\n        ruleout = bool(d_ug < thr)\n        return {\n
            \"applies\": preg, \"items\": items, \"items_count\": count,\n
            \"d_dimer_ug_l\": d_ug, \"threshold_ug_l\": thr, \"rule_out\":
ruleout,\n            \"note\": \"If DVT signs present, compression
ultrasound is recommended before applying YEARS.\"\n        }\n\n# UI
that reuses Phase-2 widget inputs if present (vitals_in / labs_in /
ctx_in).\n    try:\n        _probe = vitals_in # noqa: F401\n
_has_phase2_ui = True\n    except NameError:\n        _has_phase2_ui =
False\n\n    if CONFIG.get(\"RUN_UI\", False):\n        try:\n            import
ipywidgets as W\n            if _has_phase2_ui:\n                labs_src =
labs_in\n                ctx_src = ctx_in\n                src_note_ctx =
W.HTML('<i>Using Context JSON from Phase-2 panel</i>')\n
src_note_labs = W.HTML('<i>Using Labs JSON from Phase-2 panel</i>')\n
            else:\n                labs_src =
W.Textarea(value='{\"d_dimer\":780,\"d_dimer_unit\": \"µg/L FEU\"}',
description=\"Labs JSON\", layout=W.Layout(width=\"100%\",
height=\"80px\"))\n                ctx_src =
W.Textarea(value='{\"pregnant\": true, \"pe_most_likely\": true,
\"dvt_signs\": false, \"hemoptysis\": false}', description=\"Context
JSON\", layout=W.Layout(width=\"100%\", height=\"80px\"))\n
src_note_ctx = src_note_labs = W.HTML('')\n                out = W.Output()\n
            btn = W.Button(description=\"Compute Pregnancy YEARS +
Log\")\n            warn = W.HTML('<small><b>Note:</b> PERC is not
validated in pregnancy; use the YEARS pathway below.</small>')\n\n
def _on_click(_):\n                with out:\n
out.clear_output()\n                try:\n                    labs =
json.loads(labs_src.value if hasattr(labs_src, 'value') else
labs_src)\n                    ctx = json.loads(ctx_src.value if
hasattr(ctx_src, 'value') else ctx_src)\n                except
Exception as e:\n                    print(\"[parse error]\", e);\n
return\n                res = assess_pregnancy_years(ctx, labs)\n
if not res.get(\"applies\", False):\n
print(\"Pregnancy YEARS: not applicable (pregnant flag is false).\")\n
            else:\n                msg = f\"Pregnancy
YEARS: items={res['items_count']} → D-dimer thr
{int(res['threshold_ug_l'])} µg/L; value {int(res['d_dimer_ug_l'])}

```

```

→ \"\\n
else \"IMAGING\\\")\\n
try:\\n
    _append_event({\"type\":
    \"pregnancy_years\\\", \"result\": res})\\n
print(\"Logged to:\\\", CONFIG.get(\"EVENT_LOG_PATH\\\"))\\n
except Exception as e:\\n
    print(\"[log error]\",
    e)\\n\\n
    btn.on_click(_on_click)\\n\\n
    display(W.VBox([\\n
W.HTML(\"<b>Pregnancy-adapted YEARS pathway</b>\\\"),\\n
warn,\\n
    src_note_ctx, src_note_labs,\\n
    btn,
    out\\n
    ]))\\n
    except Exception as e:\\n
        print(\"YEARS
UI unavailable:\\\", e)\\n\\n# Text-mode smoke if RUN_UI is off\\nif not
CONFIG.get(\"RUN_UI\\\", False):\\n
    labs = {\"d_dimer\\\": 780,
    \"d_dimer_unit\\\": \"µg/L FEU\\\"}\\n
    ctx = {\"pregnant\\\": True,
    \"pe_most_likely\\\": True, \"dvt_signs\\\": False, \"hemoptysis\\\":
    False}\\n
    res = assess_pregnancy_years(ctx, labs)\\n
    msg =
    f\"Pregnancy YEARS (text-mode): items={res.get('items_count',0)};
    thr={int(res.get('threshold_u_g_l',0))} µg/L;
    value={int(res.get('d_dimer_u_g_l',0))} → \"\\n
    msg += (\"RULE-
    OUT\\\" if res.get(\"rule_out\\\") else \"IMAGING\\\")\\n
    print(msg)\\n
try:\\n
    _append_event({\"type\": \"pregnancy_years_smoke\\\",
    \"result\": res})\\n
    print(\"Logged to:\\\",
    CONFIG.get(\"EVENT_LOG_PATH\\\"))\\n
    except Exception as e:\\n
        print(\"[log error]\", e)\\n\\n
        \"metadata\": {\"execution\":
        {\"iopub.status.busy\": \"2025-08-25T18:40:39.529903Z\", \"iopub.execute_in
        put\": \"2025-08-25T18:40:39.530209Z\", \"iopub.status.idle\": \"2025-08-25T1
        8:40:39.657546Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:39.
        530179Z\", \"shell.execute_reply\": \"2025-08-25T18:40:39.656447Z\"}, \"paper
        mill\":
        {\"duration\": 0.043504, \"end_time\": \"2025-08-24T12:38:56.686312\", \"except
        ion\": false, \"start_time\": \"2025-08-24T12:38:56.642808\", \"status\": \"compl
        eted\", \"tags\": [], \"trusted\": true}, \"outputs\":
        [{\"output_type\": \"display_data\", \"data\": {\"text/
        plain\": \"VBox(children=(HTML(value='<b>Pregnancy-adapted YEARS
        pathway</b>'), HTML(value='<small><b>Note:</b> PERC is
        n...\", \"application/vnd.jupyter.widget-view+json\":
        {\"version_major\": 2, \"version_minor\": 0, \"model_id\": \"bd71fed598444e12bcd
        aacef31187348\"}}}, \"metadata\": {}}, \"execution_count\": 91},
        {\"cell_type\": \"code\", \"source\": \"# == Safe tail of EVENT_LOG_PATH
        (optional) ==\\nimport os, subprocess, shlex\\nfrom pathlib import
        Path\\nEVENT_LOG_PATH = CONFIG.get(\"EVENT_LOG_PATH\\\", \"/kaggle/
        working/event_log.json\\\")\\nif Path(EVENT_LOG_PATH).exists():\\n
        try:\\n
        subprocess.run([\"tail\\\", \"-n\\\",
        \"20\\\", EVENT_LOG_PATH], check=False)\\n
        except Exception as e:\\n
        print(\"tail failed (ignored):\\\", e)\\nelse:\\n
        print(\"No event
        log yet (will be created on first write).\\\")\", \"metadata\":
        {\"execution\":
        {\"iopub.status.busy\": \"2025-08-25T18:40:39.658692Z\", \"iopub.execute_in
        put\": \"2025-08-25T18:40:39.659037Z\", \"iopub.status.idle\": \"2025-08-25T1
        8:40:39.683948Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:39.
        659007Z\", \"shell.execute_reply\": \"2025-08-25T18:40:39.682796Z\"}, \"paper
        mill\":
        {\"duration\": 0.018801, \"end_time\": \"2025-08-24T12:38:56.716514\", \"except
        ion\": false, \"start_time\": \"2025-08-24T12:38:56.697713\", \"status\": \"compl
        eted\", \"tags\": [], \"trusted\": true}, \"outputs\":

```

```
\{"name": "stdout", "text": "\{\"ts\": \"2025-08-25T18:11:30.050962Z\",  
\\\"type\\\": \\\"preflight_ping\\\", \\\"ok\\\": true}\\n\\{\"ts\":  
\\\"2025-08-25T18:40:38.340871Z\\\", \\\"type\\\": \\\"preflight_ping\\\",  
\\\"ok\\\": true}\\n\", \"output_type\": \"stream\"}], \"execution_count\": 92},  
{\"cell_type\": \"code\", \"source\": \"\\n# == Medication Plan Import +  
Checks (inline, no sidecars) ==\\nfrom __future__ import  
annotations\\nfrom typing import Dict, Any, List, Optional,  
Tuple\\nfrom pathlib import Path\\nfrom datetime import datetime,  
timezone\\nimport json, re\\n\\nEVENT_LOG =  
Path(CONFIG[\\\"EVENT_LOG_PATH\\\"])\\n\\ndef _append_event(ev: Dict[str,  
Any]):\\n    EVENT_LOG.parent.mkdir(parents=True, exist_ok=True)\\n  
    EVENT_LOG.touch(exist_ok=True)\\n    ev = {\\\"ts\\\":  
datetime.now(timezone.utc).isoformat(), **ev}\\n    with  
EVENT_LOG.open(\\\"a\\\") as fp:\\n        fp.write(json.dumps(ev,  
ensure_ascii=False) + '\\\\n\\\\n')\\n\\n# Minimal ATC/name map for demo  
(extendable)\\n_ATC_MAP = {\\n    \\\"amoxicillin\\\": {\\\"atc\\\":  
\\\"J01CA04\\\", \\\"class\\\": \\\"penicillin\\\"},\\n    \\\"ibuprofen\\\":  
{\\\"atc\\\": \\\"M01AE01\\\", \\\"class\\\": \\\"nsaid\\\"},\\n    \\\"warfarin\\\":  
{\\\"atc\\\": \\\"B01AA03\\\", \\\"class\\\": \\\"coumarin\\\"},\\n    \\\"ramipril\\\":  
{\\\"atc\\\": \\\"C09AA05\\\", \\\"class\\\": \\\"ace\\\"},\\n    \\\"spironolacton\\\":  
{\\\"atc\\\": \\\"C03DA01\\\", \\\"class\\\": \\\"aldosterone_antagonist\\\"},\\n  
\\\"metoprolol\\\": {\\\"atc\\\": \\\"C07AB02\\\", \\\"class\\\": \\\"beta_blocker\\\"},  
\\n    \\\"simvastatin\\\": {\\\"atc\\\": \\\"C10AA01\\\", \\\"class\\\":  
\\\"statin\\\"},\\n    \\\"clarithromycin\\\": {\\\"atc\\\": \\\"J01FA09\\\",  
\\\"class\\\": \\\"macrolide\\\"},\\n    \\\"azithromycin\\\": {\\\"atc\\\":  
\\\"J01FA10\\\", \\\"class\\\": \\\"macrolide\\\"},\\n}\\n\\n# Demo interaction  
rules (replace via MED_RULES_PATH for your own rules)\\n_DEMO_RULES =  
{\\n    \\\"a\\\": \\\"ibuprofen\\\", \\\"b\\\": \\\"warfarin\\\", \\\"severity\\\":  
\\\"major\\\", \\\"message\\\": \\\"Bleeding risk (NSAID + warfarin)\\\"},\\n  
    \\\"a\\\": \\\"ramipril\\\", \\\"b\\\": \\\"spironolacton\\\", \\\"severity\\\":  
\\\"moderate\\\", \\\"message\\\": \\\"Hyperkalemia risk (ACE + aldosterone  
antagonist)\\\"},\\n    \\\"a\\\": \\\"amoxicillin\\\", \\\"b\\\": \\\"warfarin\\\",  
\\\"severity\\\": \\\"moderate\\\", \\\"message\\\": \\\"Antibiotic may potentiate  
warfarin effect\\\"},\\n    \\\"a\\\": \\\"clarithromycin\\\", \\\"b\\\":  
\\\"simvastatin\\\", \\\"severity\\\": \\\"major\\\", \\\"message\\\":  
\\\"Rhabdomyolysis risk (CYP3A4 inhibition)\\\"},\\n}\\n\\n\\n_ROUTES =  
[\\\"p.o.\\\", \\\"po\\\", \\\"i.v.\\\", \\\"iv\\\", \\\"i.m.\\\", \\\"im\\\", \\\"s.c.\\\",  
\\\"sc\\\", \\\"inhalativ\\\", \\\"topisch\\\", \\\"nasal\\\", \\\"otic\\\",  
\\\"ophthalmic\\\"]\\n_FREQ_WORDS = [\\\"morgens\\\", \\\"mittags\\\",  
\\\"abends\\\", \\\"nachts\\\"]\\n_FREQ_PAT = re.compile(r'\\\\\\\\b(\\\\\\\\d+)[-./](\\\\\\\\  
\\\\\\\\d+)[-./](\\\\\\\\d+)\\\\\\\\b')\\n_DOSE_PAT = re.compile(r'\\\\\\\\(\\\\\\\\d+(?:[.,]\\\\\\\\d+)?)  
\\\\\\\\s*(mg|g|mcg|ug|ml|IE|Einheiten)\\\\\\\\b', flags=re.I)\\n\\n\\ndef  
_normalize_name(s: str) -> str:\\n    s = s.strip().lower()\\n    s =  
s.replace(\\\"ä\\\", \\\"ae\\\").replace(\\\"ö\\\", \\\"oe\\\").replace(\\\"ü\\\",  
\\\"ue\\\").replace(\\\"ß\\\", \\\"ss\\\")\\n    return re.sub(r'^a-z0-9]+', '\\'  
\\', s).strip()\\n\\n\\ndef _map_to_atc(name_norm: str) -> Dict[str, Any]:  
\\n    for key, meta in _ATC_MAP.items():\\n        if key in  
name_norm:\\n            return {\\\"name_norm\\\": key, **meta}\\n  
return {\\\"name_norm\\\": name_norm, \\\"atc\\\": None, \\\"class\\\": None}  
\\n\\n\\ndef parse_med_line(line: str) -> Optional[Dict[str, Any]]:\\n  
raw = line.strip()\\n    if not raw or raw.startswith(\\\"#\\\"):\\n  
return None\\n    name = raw.split(\\\",\\\", \\\" \")[0].split(\\\" \")[0] # up  
to comma or double spaces\\n    name_norm = _normalize_name(name)\\n  
dose_val, dose_unit = None, None\\n    m = DOSE_PAT.search(raw)\\n
```

```
if m:\n        dose_val = float(m.group(1).replace("\\", "\\.\\."))\n    dose_unit = m.group(2).lower()\n    freq = None\n    m = _FREQ_PAT.search(raw)\n    if m:\n        freq = f"{m.group(1)}-{m.group(2)}-{m.group(3)}"\n    else:\n        words = [w for w in _FREQ_WORDS if w in raw.lower()]\n        if words:\n            freq = "\",\".join(words)\n        route = None\n        for r in _ROUTES:\n            if r in raw.lower():\n                route = r\n                break\n        prn = bool(re.search(r"\\b(prn|bei bedarf)\\b", raw, flags=re.I))\n    atc_meta = _map_to_atc(name_norm)\n    return {\n        "raw": raw,\n        "name": name.strip(),\n        "name_norm": atc_meta["name_norm"],\n        "atc": atc_meta["atc"],\n        "class": atc_meta["class"],\n        "dose_value": dose_val,\n        "dose_unit": dose_unit,\n        "frequency": freq,\n        "route": route,\n        "prn": prn,\n    }\n\ndef parse_med_text(text: str) -> List[Dict[str, Any]]:\n    meds = []\n    for line in text.splitlines():\n        rec = parse_med_line(line)\n        if rec:\n            meds.append(rec)\n    return meds\n\ndef load_rules(path: str) -> List[Dict[str, Any]]:\n    p = Path(path)\n    if p.exists():\n        try:\n            js = json.loads(p.read_text())\n            if isinstance(js, list):\n                return js\n            except Exception:\n                pass\n            return list(_DEMO_RULES)\n        \ndef load_allergies(path: str) -> List[str]:\n            p = Path(path)\n            if p.exists():\n                try:\n                    js = json.loads(p.read_text())\n                    if isinstance(js, dict) and "allergies" in js and isinstance(js["allergies"], list):\n                        return [str(x) for x in js["allergies"]]\n                    except Exception:\n                        pass\n            return []\n        \ndef check_interactions(meds: List[Dict[str, Any]], rules: List[Dict[str, Any]]) -> List[Dict[str, Any]]:\n            out = []\n            names = [m["name_norm"] for m in meds]\n            for r in rules:\n                a, b = r["a"], r["b"]\n                if (a in names and b in names) or (b in names and a in names):\n                    out.append({"type": "med_interaction_warning", *r})\n            return out\n        \ndef check_allergies(meds: List[Dict[str, Any]], allergy_terms: List[str]) -> List[Dict[str, Any]]:\n            out = []\n            terms = [t.lower() for t in allergy_terms]\n            for m in meds:\n                # direct name match\n                if any(t in m["name"].lower() for t in terms):\n                    out.append({"type": "med_allergy_warning", "med": m["name"], "match": "name"})\n                    continue\n                # conservative penicillin class heuristic\n                if any("penicillin" in t for t in terms):\n                    if m["class"] == "penicillin" or m["name"].lower().endswith("cillin"):\n                        out.append({"type": "med_allergy_warning", "med": m["name"], "match": "class_penicillin"})\n            return out\n        \ndef ocr_file(path: str) -> Optional[str]:\n            p = Path(path)\n            if not p.exists():\n                return None\n            try:\n                from PIL import Image\n                import pytesseract\n                if p.suffix.lower() in [".png", ".jpg", ".jpeg", ".tif", ".tiff"]:\n                    return pytesseract.image_to_string(Image.open(p))\n                if p.suffix.lower() == ".pdf":\n                    try:\n                        from pdf2image import convert_from_path\n                        pages =
```

```

convert_from_path(str(p))\n                text = \"\"\nfor img in pages:\n                text +=\npytesseract.image_to_string(img) + \"\\n\\n\"\n                return\ntext\n                except Exception:\n                return None\nexcept Exception:\n                return None\n                return None\n\ndef\nemit_meds_events(patient_id: str, meds: List[Dict[str, Any]],\nwarnings: List[Dict[str, Any]]):\n    _append_event({\"type\":\n\"med_plan_import\", \"patient_id\": patient_id, \"n_meds\":\nlen(meds)})\n    for m in meds:\n        _append_event({\"type\":\n\"med_entry\", \"patient_id\": patient_id, **m})\n    for w in\nwarnings:\n        _append_event({**w, \"patient_id\": patient_id})\n\n\nif CONFIG.get(\"RUN_MEDS\"):\n    print(\"Medication import/\nchecks enabled. Use the UI panel (if RUN_UI=True) or call the\nfunctions above.\")\nelse:\n    print(\"Deferred... set\nCONFIG['RUN_MEDS']=True to enable medication import & checks.\")\n\n\n# Optional UI\nif CONFIG.get(\"RUN_UI\") and\nCONFIG.get(\"RUN_MEDS\"):\n    try:\n        import ipywidgets as\nW\n        import pandas as pd\n        pid =\nW.Text(description=\"Patient ID\", placeholder=\"e.g., UKE-12345\")\n        path = W.Text(description=\"Scan path\", placeholder=\"/\nmnt/data/scan.pdf (optional)\")\n        ocr_btn =\nW.Button(description=\"Run OCR\")\n        ta =\nW.Textarea(description=\"Plan text\", layout=W.Layout(width=\"100%\n\", height=\"180px\"))\n        parse_btn =\nW.Button(description=\"Parse & Check\", button_style=\"primary\")\n        out = W.Output()\n        def on_ocr(_):\n            with out:\n                out.clear_output()\n                txt =\nocr_file(path.value.strip())\n                if txt:\n                    ta.value = txt\n                    print(\"OCR complete.\")\n                else:\n                    print(\"OCR unavailable or failed. Paste\nthe text instead.\")\n        def on_parse(_):\n            with\nout:\n                out.clear_output()\n                text =\n                ta.value.strip()\n                if not text:\n                    print(\"No text provided.\"); return\n                    meds =\n                    parse_med_text(text)\n                    rules =\n                    load_rules(CONFIG[\"MED_RULES_PATH\"])\n                    allergies =\n                    load_allergies(CONFIG[\"ALLERGIES_PATH\"])\n                    warnings\n= check_interactions(meds, rules) + check_allergies(meds, allergies)\n                    emit_meds_events(pid.value or \"DEMO\", meds,\nwarnings)\n                    if meds:\n                        display(pd.DataFrame(meds))\n                    else:\n                        print(\"No medications parsed.\")\n                        if warnings:\n                            print(\"\\nWarnings:\\n\")\n                            display(pd.DataFrame(warnings))\n                        else:\n                            print(\"\\n\\nNo interaction/allergy warnings.\")\n                    ocr_btn.on_click(on_ocr)\n                    parse_btn.on_click(on_parse)\n                    display(W.VBox([pid, path, W.HBox([ocr_btn, parse_btn]), ta, out]))\n    except Exception as e:\n        print(\"Meds UI unavailable:\n\", e)\n    \"\", \"metadata\": {\"execution\":\n{\"iopub.status.busy\": \"2025-08-25T18:40:39.685014Z\", \"iopub.execute_in\nput\": \"2025-08-25T18:40:39.685969Z\", \"iopub.status.idle\": \"2025-08-25T1\n8:40:39.737224Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:39.\n685937Z\", \"shell.execute_reply\": \"2025-08-25T18:40:39.736399Z\"}, \"paper\nmill\":\n
```

```

{"duration":0.062555,"end_time":"2025-08-24T12:38:56.790421","exception":false,"start_time":"2025-08-24T12:38:56.727866","status":"completed"},"tags":[],"trusted":true},"outputs":
[{"name":"stdout","text":"Medication import/checks enabled. Use the
UI panel (if RUN_UI=True) or call the functions above.
\n","output_type":"stream"},{"output_type":"display_data","data":
{"text/plain":"VBox(children=(Text(value='', description='Patient
ID', placeholder='e.g., UKE-12345'), Text(value='',
descrip...", "application/vnd.jupyter.widget-view+json":
{"version_major":2,"version_minor":0,"model_id":"4460d63be5944b92b88
b82ab40b91857"}}, "metadata":{}}}, {"execution_count":93},
{"cell_type":"code","source":"# == Gate-pack merger (pure; TTL=max;
order-preserving dedup) ==\nfrom typing import Dict, Any\n\ndef
_merge_ttl_max(a: Dict[str,int]|None, b: Dict[str,int]|None) ->
Dict[str,int]:\n    out = dict(a or {})\n    for k, v in (b or
{}).items():\n        try:\n            out[k] = max(out.get(k, 0),
int(v))\n        except Exception:\n            out[k] = out.get(k,
0)\n    return out\n\ndef merge_gatepacks(*packs: Dict[str, Any]) ->
Dict[str, Any]:\n    out: Dict[str, Any] = {"gates\\": [],
\\"priority\\": 0, \\"next_action\\": \\"\\", \\"explain\\": [], \\"ttl\\":
{}}\n    for p in packs:\n        if not p: \n            continue\n    out[\"gates\\\"].extend(p.get(\"gates\\\", []))\n    out[\"priority\\\"] = max(out[\"priority\\\"], int(p.get(\"priority\\\",
0)))\n    if p.get(\"next_action\\\"):\n    out[\"next_action\\\"] = p[\"next_action\\\"]\n        if
p.get(\"explain\\\"):\n    out[\"explain\\\"].extend(p[\"explain\\\"])\n        out[\"ttl\\\"] =
_merge_ttl_max(out[\"ttl\\\"], p.get(\"ttl\\\"))\n    # order-preserving
dedup\n    seen, dedup = set(), []\n    for g in out[\"gates\\\"]:\n
if g not in seen:\n        seen.add(g); dedup.append(g)\n    out[\"gates\\\"] = dedup\n    return out\n", "metadata":{"execution":
{"iopub.status.busy":"2025-08-25T18:40:39.738621Z","iopub.execute_in
put":"2025-08-25T18:40:39.738946Z","iopub.status.idle":"2025-08-25T1
8:40:39.771544Z","shell.execute_reply.started":"2025-08-25T18:40:39.
738919Z","shell.execute_reply":"2025-08-25T18:40:39.770503Z"},"paper
mill":
{"duration":0.021592,"end_time":"2025-08-24T12:38:56.823278","except
ion":false,"start_time":"2025-08-24T12:38:56.801686","status":"compl
eted"},"tags":[],"trusted":true},"outputs":[],"execution_count":94},
{"cell_type":"code","source":"# == Gate-engine (composes packs; ICU
& BGA fallback if packs absent) ==\nfrom typing import Dict,
Any\n\ndef _fallback_pack_icu(fd: Dict[str,Any], CONFIG:
Dict[str,Any]) -> Dict[str,Any]:\n    # thresholds (do not mutate
CONFIG)\n    th_ok = float(CONFIG.get(\"TH_ICU_CAP_OK\\\", 0.15))
# ≥15% = OK\n    cap = fd.get(\"cap_internal_medicine_icu\\\")\n
need_icu = str(fd.get(\"admit_decision\\\", \"\")).upper() ==
\"ICU\"\n    blocked = (cap is not None) and (float(cap) <= 0.0)\n
tight = (cap is not None) and (0.0 < float(cap) < th_ok)\n
gates, explain = [], []\n    if need_icu and (blocked or tight):\n
gates += [\"ROUTE_PLAN_ED_TO_ICU\"]\n        if blocked:\n
gates += [\"ICU_BLOCKED\\\", \"ED_BOARDING_RISK\\\",
\"TRANSFER_BLOCK_CAPACITY\\\"]\n            explain.append(\"No ICU
capacity – board in ED, activate bed manager now.\")\n        elif
tight:\n            gates += [\"ICU_TIGHT\"]\n

```

```

explain.append("\nICU capacity tight – pre-alert bed manager and
accepting team.\n")\n      # alerts for capacity issue\n
gates += ["\nALERT_NURSE\n","\nALERT_DOC\n"]\n      if not gates:\n
return {"\ngates\n": [], "\npriority\n": 0, "\nnext_action\n": "\n",
"\nexplain\n": [], "\nttl\n": {}}\n      return {\n          "\ngates\n":
gates,\n          "\npriority\n": 1,\n          "\nnext_action\n":
explain[0] if explain else "\n",\n          "\nexplain\n": explain,\n
"\nttl\n": {"\nALERT_NURSE\n":300, "\nALERT_DOC\n":300}\n      }\n\nndef
_fallback_pack_bga(fd: Dict[str,Any], CONFIG: Dict[str,Any]) ->
Dict[str,Any]:\n      # thresholds from your ops policy\n      pH =
fd.get("\nvbg_pH\n")\n      K = fd.get("\nvbg_K\n")\n      Na =
fd.get("\nvbg_Na\n")\n      Lac = fd.get("\nvbg_lactate\n")\n      gates,
explain, ttl = [], [], {}\n      # pH\n      if pH is not None:\n
if float(pH) > 7.55:\n
gates.append("\nBGA_ALKALOSIS_CRITICAL\n"); explain.append(f"\npH {pH}
> 7.55\n")\n      elif float(pH) >= 7.5:\n
gates.append("\nBGA_ALKALOSIS_HIGH\n"); explain.append(f"\npH {pH} ≥
7.5\n")\n      # Potassium\n      if K is not None:\n          kf =
float(K)\n          if kf >= 6.5:\n
gates.append("\nBGA_HYPERKALAEMIA_CRITICAL\n"); explain.append(f"\nK+
{kf} ≥ 6.5 mmol/L\n")\n          elif kf >= 6.0:\n
gates.append("\nBGA_HYPERKALAEMIA_HIGH\n"); explain.append(f"\nK+ {kf}
≥ 6.0 mmol/L\n")\n          elif kf < 2.5:\n
gates.append("\nBGA_HYPOKALAEMIA_CRITICAL\n"); explain.append(f"\nK+
{kf} < 2.5 mmol/L\n")\n      # Sodium\n      if Na is not None:\n
naf = float(Na)\n      if naf <= 125:\n
gates.append("\nBGA_DYSNATRAEMIA_CRITICAL\n"); explain.append(f"\nNa+
{naf} ≤ 125 mmol/L\n")\n      elif (naf <= 129) or (naf >= 145):\n
gates.append("\nBGA_DYSNATRAEMIA\n"); explain.append(f"\nNa+ {naf}
outside 130–144 mmol/L\n")\n      # Lactate (typical critical)\n      if
Lac is not None and float(Lac) >= 4.0:\n
gates.append("\nBGA_LACTATE_CRITICAL\n"); explain.append(f"\nLactate
{Lac} ≥ 4.0 mmol/L\n")\n      # escalate alerts for any critical\n
if any(g.endswith("\nCRITICAL\n") for g in gates):\n          gates +=
["\nALERT_NURSE\n","\nALERT_DOC\n","\nALERT_ATTENDING\n"]\n
ttl.update({"\nALERT_NURSE\n":300, "\nALERT_DOC\n":300, "\nALERT_ATTENDING\n":300})\n
if not gates:\n          return {"\ngates\n": [],
"\npriority\n": 0, "\nnext_action\n": "\n", "\nexplain\n": [], "\nttl\n":
{}}\n      return {\n          "\ngates\n": gates,\n          "\npriority\n":
2 if any(g.endswith("\nCRITICAL\n") for g in gates) else 1,\n
"\nnext_action\n": "\nAddress critical VBG derangements now.\n" if
any(g.endswith("\nCRITICAL\n") for g in gates) else "\n",\n
"\nexplain\n": explain,\n          "\nttl\n": ttl\n      }\n\nndef
gate_engine(fd: Dict[str,Any], CONFIG: Dict[str,Any]) ->
Dict[str,Any]:\n      "\n"\n      Compose gate-packs if present;
otherwise use ICU+BGA fallbacks to guarantee safety signals.\n      No
I/O. Pure function.\n      "\n"\n      packs = []\n      # Prefer your
real pack producers if they're defined in the notebook/module\n
for name in [\n          # add your real producers here if present,
e.g.:\n          "\nops_capacity_gatepack\n",          # ICU/boarding
pack\n          "\nbga_gatepack\n",          # VBG/
electrolytes pack\n          "\nabdomen_guardrails_gatepack\n",          #
imaging + labs policy\n          "\noverdue_gatepack\n",
# last_seen & vitals overdues\n          "\ntroponin_policy_gatepack\n",

```

```

# troponin timing/delta policy\n    ]:\n        fn =
globals().get(name)\n        if callable(fn):\n            try:\n
packs.append(fn(fd, CONFIG))\n            except Exception as e:\n
# non-fatal: continue composing with other packs\n
packs.append({"gates\":[], "priority\":0, "next_action\":"\",
"explain\":[f"{name} failed: {e}\", "ttl\":{}})\n\n    # If key
packs are missing, add safe fallbacks\n    if not any("\ICU\" in g
for p in packs for g in p.get("gates\", [])):\n
packs.append(_fallback_pack_icu(fd, CONFIG))\n    if not
any(g.startswith("\BGA\") for p in packs for g in p.get("gates\",
[])):\n        packs.append(_fallback_pack_bga(fd, CONFIG))\n\n    #
Merge all packs\n    return merge_gatepacks(*packs)\n", "metadata":
{"execution":
{"iopub.status.busy":"2025-08-25T18:40:39.772592Z", "iopub.execute_in
put":"2025-08-25T18:40:39.772906Z", "iopub.status.idle":"2025-08-25T1
8:40:39.801946Z", "shell.execute_reply.started":"2025-08-25T18:40:39.
772880Z", "shell.execute_reply":"2025-08-25T18:40:39.801112Z"}, "paper
mill":
{"duration":0.030236, "end_time":"2025-08-24T12:38:56.865131", "except
ion":false, "start_time":"2025-08-24T12:38:56.834895", "status":"compl
eted"}, "tags":[], "trusted":true}, "outputs":[], "execution_count":95},
{"cell_type":"code", "source":"# == Failure Triage Harness (gate
engine + Phase-2 split) ==\nimport sys, types, inspect\nfrom
datetime import datetime\n\ndef _have(*names):\n    return {n: (n in
globals()) for n in names}\n\ndef _where(fn_name):\n    obj =
globals().get(fn_name, None)\n    if not callable(obj): return
None\n    try:\n        src = inspect.getsourcefile(obj) or
inspect.getfile(obj)\n        ln = obj.__code__.co_firstlineno\n
return f\"{src}:{ln}\" except Exception:\n        return
\"<dynamic>\"\n\nprint(f\"PY {sys.version.split()[0]}\")
\nprint("\PRESENT:\", _have("\merge_gatepacks\", "\gate_engine\",
\"_pick\", \"_bool\", \"_safe_round\", \n
\"build_labs_norm\", \"compute_phase2_scores\", \"phase2_bundle\", \n
\"parse_hl7_all\", \"parse_hl7_labs\"))\n\n# Alias tolerance for
qSOFA func name drift\nif "\calc_qsoFA\" not in globals() and
\"calc_qsofa\" in globals():\n    calc_qsoFA = calc_qsofa\nelif
\"calc_qsoFA\" in globals() and \"calc_qsofa\" not in globals():\n
calc_qsofa = calc_qsoFA\n\nprint("\LOC:\", {n:_where(n) for n in
[\"merge_gatepacks\", \"gate_engine\", \"build_labs_norm\", \n
\"compute_phase2_scores\", \"phase2_bundle\"]})\n\ndef _probe(title,
fn):\n    try:\n        r = fn()\n        print(f\"{title}: OK\")\n
return True, r\n    except Exception as e:\n        et =
type(e).__name__\n        print(f\"{title}: FAIL -> {et}: {e}\")\n
return False, e\n\n# 1) TTL merge sanity\ndef _t1():\n    p1 =
{\"gates\":[\"ALERT_NURSE\"], \"priority\":0, \"next_action\":"\",
\"explain\":[], \"ttl\":{\"ALERT_NURSE\":120}}\n    p2 = {\"gates\":
[\"ALERT_NURSE\"], \"priority\":0, \"next_action\":"\",
\"explain\":[], \"ttl\":{\"ALERT_NURSE\":300}}\n    m =
merge_gatepacks(p1, p2)\n    assert
m[\"ttl\"].get(\"ALERT_NURSE\")==300\n    return
m\n\n_probe(\"TTL_MAX_OK\", _t1)\n\n# 2) Gate engine probe (no HL7
needed)\ndef _t2():\n    fd = {\"Triage\":"\"Orange\",
\"admit_decision\":"\"ICU\", \"suspected_condition\":"\"respiratory\",
\n

```

```

\cap_internal_medicine_icu\":0.0,\ems_triage_code\:1,\vbg_pH\:7
.56,\vbg_K\:6.6}\n    out = gate_engine(fd, CONFIG)\n    gs =
set(out.get(\"gates\",[]))\n    assert {\\"ALERT_NURSE\",
\\"ALERT_DOC\"}.intersection(gs)\n    return sorted(list(gs))
[:8]\n_probe(\\"GATES_WIRED_OK\", _t2)\n\n# 3) Phase-2 bundle probe
(uses canonical HL7 if available; otherwise minimal labs)\ndef
_t3():\n    if \\"parse_hl7_all\" in globals():\n        hl7 =
(\\"OBX|1|NM|D-DIMER^D-Dimer FEU||2000|ug/L FEU|||F\\n\\n
\\"OBX|2|NM|TROP^Troponin T (hs)||70|ng/L|||F\\n\\n
\\"OBX|3|NM|BILITOT^Bilirubin Total||2.3|mg/dL|||F\\n\\n
\\"OBX|4|NM|ALB^Albumin||3.6|g/dL|||F\\n\\n)\n        labs =
parse_hl7_all(hl7)\n    else:\n        labs =
{\\"lab_ddimer_mgL_FEU\:2.0,\\"lab_bili_total_umolL\:2.3*17.104,\\"la
b_albumin_gL\:36.0}\n    vit =
{\\"sbp\:110,\\"hr\:95,\\"rr\:18,\\"temp\:37.1,\\"gcs\:15}\n    out
= phase2_bundle(vit, labs, {\\"age\:62})\n    assert \\"one_liners\"
in out and \\"scores\" in out\n    return
out[\\"scores\"].get(\"qsofa\"), out[\\"one_liners\"]
[:2]\n_probe(\\"PHASE2_BUNDLE_SMOKE_OK\", _t3)\n\nprint(\\"HINTS:\")
\nprint(\\"- NameError merge_gatepacks/gate_engine => run your gate
packs/engine cell or add the merger shim above.\")\nprint(\\"-
NameError _pick/_bool/_safe_round => run Calc utils cell (defines
those three).\")\nprint(\\"- calc_qsoFA vs calc_qsofa mismatch =>
alias applied above; make it permanent later.\")\nprint(\\"-
NameError parse_hl7_all => run unified HL7 cell; bundle fallback used
here until you do.\")\n", "metadata": {"execution":
{"iopub.status.busy": "2025-08-25T18:40:39.802994Z", "iopub.execute_in
put": "2025-08-25T18:40:39.803576Z", "iopub.status.idle": "2025-08-25T1
8:40:39.830139Z", "shell.execute_reply.started": "2025-08-25T18:40:39.
803545Z", "shell.execute_reply": "2025-08-25T18:40:39.829067Z"}, "paper
mill":
{"duration": 0.028528, "end_time": "2025-08-24T12:38:56.905006", "except
ion": false, "start_time": "2025-08-24T12:38:56.876478", "status": "compl
eted"}, "tags": [], "trusted": true}, "outputs":
[{"name": "stdout", "text": "PY 3.11.13\nPRESENT: {'merge_gatepacks':
True, 'gate_engine': True, '_pick': True, '_bool': True,
'_safe_round': True, 'build_labs_norm': True,
'compute_phase2_scores': True, 'phase2_bundle': True,
'parse_hl7_all': False, 'parse_hl7_labs': False}\nL0C:
{'merge_gatepacks': '/tmp/ipykernel_36/1608344827.py:13',
'gate_engine': '/tmp/ipykernel_36/2783023910.py:73',
'build_labs_norm': '/tmp/ipykernel_36/275384517.py:4',
'compute_phase2_scores': '/tmp/ipykernel_36/1671391004.py:4',
'phase2_bundle': '/tmp/ipykernel_36/1148578841.py:5'}\nTTL_MAX_OK:
OK\nGATES_WIRED_OK: OK\nPHASE2_BUNDLE_SMOKE_OK: OK\nHINTS:\n-
NameError merge_gatepacks/gate_engine => run your gate packs/engine
cell or add the merger shim above.\n- NameError _pick/_bool/_
_safe_round => run Calc utils cell (defines those three).\n-
calc_qsoFA vs calc_qsofa mismatch => alias applied above; make it
permanent later.\n- NameError parse_hl7_all => run unified HL7 cell;
bundle fallback used here until you do.
\n", "output_type": "stream"}], "execution_count": 96},
{"cell_type": "code", "source": "# Gate dedup + TTL max policy\np1 =
{\\"gates\":[\\"ALERT_NURSE\"], \\"priority\:0, \\"next_action\":"\"\",

```

```

\explain\":[], \ttl\":{\\"ALERT_NURSE\\":120}}\np2 = {\\"gates\\":
[\\\"ALERT_NURSE\\\"], \\"priority\\":0, \\"next_action\\":\\\"\\\",
\\\"explain\\":[], \\"ttl\\":{\\"ALERT_NURSE\\":300}}}\nm =
merge_gatepacks(p1, p2)\nassert m[\\\"ttl\\\"].get(\\\"ALERT_NURSE\\\") ==
300\nprint(\\\"TTL_MAX_OK\\\")\n\n# Gate pack smoke (ICU + BGA path
still wired)\nfd = {\\"suspected_condition\\":\\\"respiratory\\\",
\\\"cap_internal_medicine_icu\\":0.0, \\"ems_triage_code\\":1, \n
\\\"admit_decision\\":\\\"ICU\\\", \\"Triage\\":\\\"Orange\\\", \\"vbg_pH\\":7.56,
\\\"vbg_K\\":6.6}\npack = gate_engine(fd, CONFIG)\nassert
{\\"ICU_BLOCKED\\\", \\"ED_BOARDING_RISK\\\",
\\\"ALERT_NURSE\\\"}.issubset(set(pack[\\\"gates\\"]))
\nprint(\\\"GATES_WIRED_OK\\\")\n\n,\"metadata\":{\\"execution\":
{\\"iopub.status.busy\\":\"2025-08-25T18:40:39.831060Z\", \\"iopub.execute_in
put\\":\"2025-08-25T18:40:39.831411Z\", \\"iopub.status.idle\\":\"2025-08-25T1
8:40:39.854774Z\", \\"shell.execute_reply.started\\":\"2025-08-25T18:40:39.
831343Z\", \\"shell.execute_reply\\":\"2025-08-25T18:40:39.853841Z\"}, \\"paper
mill\":
{\\"duration\\":0.021086, \\"end_time\\":\"2025-08-24T12:38:56.937811\", \\"except
ion\\":false, \\"start_time\\":\"2025-08-24T12:38:56.916725\", \\"status\\":\\\"compl
eted\\\"}, \\"tags\\":[], \\"trusted\\":true}, \\"outputs\":
[{\\"name\\":\\\"stdout\\\", \\"text\\\":\\\"TTL_MAX_OK\\nGATES_WIRED_OK\\n\\\", \\"output_type
\\":\\\"stream\\\"}], \\"execution_count\\":97}, {\\"cell_type\\":\\\"code\\\", \\"source\\\":\\\"#
== Tiny patch: canonical refresh_sop_registry + parse_hl7_labs
==\nfrom pathlib import Path\nfrom datetime import datetime\nfrom
typing import Dict, Any, Optional\nimport re, json\n\nndef
refresh_sop_registry(CONFIG: dict, base_url:
Optional[str]=\\\"https://sop-notaufnahme.de/sop/\\\")->Dict[str, Any]:\n
# Fetch product pages, grab first PDF, write canonical CSV\n
out_csv = Path(CONFIG[\\\"SOP_REGISTRY_PATH\\\"]);\n pdf_dir =
Path(CONFIG[\\\"DATA_ROOT\\\"])/\\\"sop_pdfs\\\";\n
pdf_dir.mkdir(parents=True, exist_ok=True)\n
try:\n
import
requests; from bs4 import BeautifulSoup\n
except Exception as e:\n
\n
return
{\\"found\\":0, \\"saved\\":0, \\"errors\\":1, \\"error\\":f\\\"missing libs: {e}
\\\", \\"csv\\": str(out_csv)}\n
found=saved=errors=0; recs=[]\n
try:\n
soup = BeautifulSoup(requests.get(base_url,
timeout=15).text, \\"html.parser\\")\n
links =
sorted({a[\\\"href\\\"] for a in soup.select('a[href*=\\\"/product/\\\"]')
if a[\\\"href\\\"].startswith(\\\"http\\\")})\n
for u in links:\n
try:\n
ps = BeautifulSoup(requests.get(u,
timeout=15).text, \\"html.parser\\")\n
title =
(ps.find([\\\"h1\\\", \\\"h2\\\"] or ps.find(\\\"title\\\") or
\\\"\\\"))\n
.get_text(strip=True)\n
sid = re.sub(r\\\"[^a-
z0-9]+\\\", \\\"-\\\", (title or u).lower()).strip(\\\"-\\\")\n
a = ps.select_one('a[href$=\\\".pdf\\\"]'); pdf_path=\\\"\\\"\\n
if a:\n
try:\n
fp =
pdf_dir/f\\\"{sid}.pdf\\\"\\n
with
requests.get(a[\\\"href\\\"], stream=True, timeout=30) as r:\n
r.raise_for_status()\n
with open(fp,
\\\"wb\\\") as f:\n
for c in
r.iter_content(8192): \n
if c:
f.write(c)\n
pdf_path=str(fp); saved+=1\n
except Exception: errors+=1; pdf_path=a[\\\"href\\\"]\n
recs.append({\\\"sop_id\\\":sid, \\"title\\\":title or sid,

```

```

def pdf_path":pdf_path,"version":"","status":"","checked" if
pdf_path else "\\linked\\","keywords":"","checklist":"","\n",
"source_url":u}})\n\n found+=1\n except
Exception: errors+=1\n except Exception as e:\n return
{"found":0,"saved":0,"errors":1,"error":str(e), "csv":
str(out_csv)}\n import pandas as pd\n cols=["sop_id",
"title","pdf_path","version","status","keywords",
"checklist","source_url"]\n df = pd.read_csv(out_csv) if
out_csv.exists() else pd.DataFrame(columns=cols)\n for c in cols:
\n if c not in df.columns: df[c]=""\n for r in recs:\n
m = (df["sop_id"].astype(str)==r["sop_id"])\n if m.any():
\n for k,v in r.items():\n if k in
df.columns and (pd.isna(df.loc[m,k]).all() or
str(df.loc[m,k].iloc[0]).strip()=="\" or k in ["pdf_path",
"status","source_url"]):\n df.loc[m,k]=v\n
else:\n df = pd.concat([df, pd.DataFrame([r])],
ignore_index=True)\n df[cols].to_csv(out_csv, index=False)\n
return {"found":found,"saved":saved,"errors":errors,
"csv":str(out_csv),"dir":str(pdf_dir)}\n\ndef
parse_hl7_labs(txt: str)->Dict[str,Any]:\n tropes, dd, oth = [],
[], {"bilirubin":None,"inr":None,"albumin":None}\n for ln
in re.split(r'[\r\n]+' , (txt or "").strip()):\n if not
ln.startswith("\tOBX|"): continue\n p = ln.split("|");
obx3, obx5, obx6, obx14 = (p+[None]*15)[3], (p+[None]*15)[5], (p+
[None]*15)[6], (p+[None]*15)[14]\n ts=None; m=re.search(r'(\d{8})(\d{6})?', obx14 or "")\n if m:\n try:
ts=datetime.strptime(m.group(1)+(m.group(2) or "000000"),
"%Y%m%d%H%M%S")\n except: pass\n try:
v=float(str(obx5).replace(",",".", ""))\n except:\n
mm=re.search(r'[+-]?[d*\.?][d+', str(obx5)); v=float(mm.group(0))
if mm else None\n if v is None: continue\n u=(obx6 or
 ""); name=(obx3 or ln).upper()\n if "\tTROP" in name:
tropes.append((ts,v,u or "\tng/L"))\n if any(k in name for k
in ["D-DIMER","DDIMER","D DIMER"]): dd.append((ts,v,u or "\tμg/
L FEU"))\n if "\tBILIRUBIN" in name or "\tBILI" in name:\n
if oth["bilirubin"] is None: oth["bilirubin"] = v/17.1 if
u.lower() in {"umol/l","μmol/l"} else v\n if "\tINR" in
name and oth["inr"] is None: oth["inr"]=v\n if
"\tALBUMIN" in name and oth["albumin"] is None: oth["albumin"]=
v/10.0 if u.lower() in {"g/l","g l","gl"} else v\n return
{"troponin":tropes,"d_dimer":dd,"others":oth}\n","metadata":
{"trusted":true,"execution":
{"iopub.status.busy":"2025-08-25T18:40:39.856014Z","iopub.execute_in
put":"2025-08-25T18:40:39.856327Z","iopub.status.idle":"2025-08-25T1
8:40:39.879569Z","shell.execute_reply.started":"2025-08-25T18:40:39.
856299Z","shell.execute_reply":"2025-08-25T18:40:39.878648Z"}}, "outp
uts":[],"execution_count":98}, {"cell_type":"code", "source":"# ---
Phase-1 quickcheck (overlay; runs only if you call it) ---\nfrom
pathlib import Path\nimport inspect, pandas as pd\n\ndef
phase1_quickcheck(config) -> dict:\n CFG = dict(config)
# do NOT mutate CONFIG\n CFG["RUN_UI"] = False\n
CFG["RUN_PIPELINE"] = False\n\n # Ensure CSV/dir semantics
without side effects\n for k in ("QR_OUTPUT_DIR",
"EVENT_LOG_PATH","SOP_REGISTRY_PATH",\n

```

```

\EQUIPMENT_STATUS_PATH\", \"EQUIPMENT_MOVES_LOG_PATH\"): \n      p
= Path(CFG[k]); (p.parent if p.suffix else p).mkdir(parents=True,
exist_ok=True)\n\n      # Import tracker from alias or globals\n
try:\n          from tracker_core import TrackerService, QRSservice,
SOPRegistry # alias shim if present\n          except Exception:\n
TrackerService, QRSservice, SOPRegistry = TrackerService, QRSservice,
SOPRegistry # use in-notebook classes\n\n      # 1) Moves log writes
(no None-indexing thanks to read()) patch)\n          t =
TrackerService.from_config(CFG)\n          t.log_move(\"__smoke__\", \"\",
\"ZZ\")\n          moves_ok =
Path(CFG[\"EQUIPMENT_MOVES_LOG_PATH\"]).exists()\n\n      # 2) SOP
auto-pull surface (offline-safe summary)\n          try:\n              sop_ok =
isinstance(refresh_sop_registry(CFG, base_url= \"http://127.0.0.1:9/
offline\"), dict)\n          except Exception:\n              sop_ok = False\n\n
# 3) Adjustable vitals threshold present (split-aware; no regex)\n
try:\n          ui_src = \"\"\n          if \"run_ui_header_controls\"
in globals():\n              ui_src =
inspect.getsource(run_ui_header_controls)\n          elif \"run_ui\"
in globals():\n              ui_src = inspect.getsource(run_ui)\n
ui_ok = \"Vitals overdue (min)\" in ui_src\n          except Exception:\n
ui_ok = False\n\n          return {\"moves\": moves_ok, \"sop\": sop_ok,
\"ui_vitals_threshold\": ui_ok}\n\n# Example opt-in usage:\n# res =
phase1_quickcheck(CONFIG); print(res); assert all(res.values()),
res\n\", \"metadata\": {\"execution\":
{\"iopub.status.busy\": \"2025-08-25T18:40:39.880538Z\", \"iopub.execute_in
put\": \"2025-08-25T18:40:39.880827Z\", \"iopub.status.idle\": \"2025-08-25T1
8:40:39.904919Z\", \"shell.execute_reply.started\": \"2025-08-25T18:40:39.
880797Z\", \"shell.execute_reply\": \"2025-08-25T18:40:39.903981Z\"}, \"paper
mill\":
{\"duration\": 0.021843, \"end_time\": \"2025-08-24T12:38:56.971393\", \"except
ion\": false, \"start_time\": \"2025-08-24T12:38:56.949550\", \"status\": \"compl
eted\"}, \"tags\": [], \"trusted\": true}, \"outputs\": [], \"execution_count\": 99},
{\"cell_type\": \"code\", \"source\": \"# one-ping-per-episode (arrival-only)
\\nfrom datetime import datetime, timezone\\n\\nFLOW =
dict(monitor_paused=False, episode=0, notified_ep=None,
current_service= \"ED\")\\n\\ndef orderly(event, dest=None, now=None):
\\n    t = now or datetime.now(timezone.utc)\\n    if event ==
\"PICKUP\":\\n        FLOW[\"episode\"] += 1\\n
FLOW[\"monitor_paused\"] = True\\n        if dest:
FLOW[\"current_service\"] = dest\\n        FLOW[\"notified_ep\"] =
None # reset notifier for this episode\\n        # no nurse
ping here\\n    elif event == \"DROPOFF_BACK_ED\":\\n
FLOW[\"current_service\"] = \"ED\"\\n        # send exactly one
arrival ping per episode\\n        if FLOW.get(\"notified_ep\") !=
FLOW[\"episode\"]:\\n            print(f\"[NURSE] {t:%H:%M}Z Patient
returned to ED - please resume monitoring.\\n\")\\n
FLOW[\"notified_ep\"] = FLOW[\"episode\"]\\n    elif event ==
\"RESUME\":\\n        FLOW[\"monitor_paused\"] = False # optional:
no ping\\n    else:\\n        pass\\n\\n# demo:\\n# orderly(\"PICKUP\",
dest= \"MRI\")\\n# orderly(\"DROPOFF_BACK_ED\")\\n#
orderly(\"DROPOFF_BACK_ED\") # (no second ping)\\n#
orderly(\"RESUME\")\\n\", \"metadata\": {\"execution\":
{\"iopub.status.busy\": \"2025-08-25T18:40:39.905942Z\", \"iopub.execute_in
put\": \"2025-08-25T18:40:39.906265Z\", \"iopub.status.idle\": \"2025-08-25T1

```



```

{"duration":0.022342,"end_time":"2025-08-24T12:38:57.077043","exception":false,"start_time":"2025-08-24T12:38:57.054701","status":"completed"},"tags":[],"trusted":true},"outputs":
[{"name":"stdout","text":"UI_VITALS_INPUT_OK: True\nFound in:
run_ui_header_controls\n","output_type":"stream"}],"execution_count":102}, {"cell_type":"code","source":"# --- Shadowing diff for
tracker_core & edtracker package ---\nimport sys, hashlib\nfrom
pathlib import Path\n\ndef sha16(p: Path):\n    h = hashlib.sha256()\n    with p.open(\"rb\") as f:\n        for chunk in iter(lambda:
f.read(65536), b\"\"): \n            h.update(chunk)\n    return
h.hexdigest()[:16]\n\ncandidates = []\n\n# Where Python *would*
import \"tracker_core\" from if those dirs were on sys.path\nfor
base in map(Path, sys.path):\n    mod = base / \"tracker_core.py\"\n
if mod.is_file():\n        candidates.append(mod)\n\n# Known dataset
mounts you pasted (may or may not be on sys.path)\nextra = [\n
Path(\"/kaggle/input/tracker-core/tracker_core.py\"),\n    Path(\"/
kaggle/working/tracker_core.py\"), # in case a shim file exists\n]
\nfor p in extra:\n    if p.is_file() and p not in candidates:\n
candidates.append(p)\n\nprint(\"# tracker_core candidates on disk\")
\nfor p in candidates:\n    try:\n        print(\".\", p, \"sha=\",
sha16(p))\n    except Exception:\n        print(\".\", p,
\"(unreadable)\")\n\n# Also show the package file actually used by
edtracker\nimport importlib, inspect\npkg =
importlib.import_module(\"edtracker.core.tracker_core\")\nprint(\"
\n# edtracker.core.tracker_core actual file\")
\nprint(inspect.getfile(pkg))\n","metadata":
{"trusted":true,"execution":
{"iopub.status.busy":"2025-08-25T18:40:39.970893Z","iopub.execute_in
put":"2025-08-25T18:40:39.971257Z","iopub.status.idle":"2025-08-25T1
8:40:39.996224Z","shell.execute_reply.started":"2025-08-25T18:40:39.
971147Z","shell.execute_reply":"2025-08-25T18:40:39.995293Z"}}, "outp
uts":[{"name":"stdout","text":"# tracker_core candidates on
disk\n. /kaggle/input/tracker-core/tracker_core.py sha=
afa6c40d0ca3117e\n\n# edtracker.core.tracker_core actual file\n/
kaggle/working/edtracker_pkg/edtracker/core/
tracker_core.py\n","output_type":"stream"}],"execution_count":103},
{"cell_type":"code","source":"# === PIPELINE INTEGRITY SELF-TEST
(fast, no side-effects) ===\nimport json, inspect, types, pandas as
pd, sys\n\nerrors = []\n\ndef _ok(name, cond, msg=\"\"): \n    if not
cond:\n        errors.append(f\"[x] {name} - {msg}\")\n    else:\n
print(f\"[✓] {name}\")\n\n# 1) Invariants\n_ok(\"CONFIG frozen\",
hasattr(CONFIG, \"_FrozenConfig\") or isinstance(CONFIG, dict))
\n_ok(\"_append_event present\", \"_append_event\" in globals())\n#
check consistent signature\n_ok(\"_append_event signature\", \"ev\"
in inspect.signature(_append_event).parameters)\n\n# 2) Unique
surfaces\ndups = [n for n in [\"run_ui\", \"refresh_sop_registry\"]
if len([k for k in globals().keys() if k==n])!=1]\n_ok(\"no
duplicate run_ui/refresh_sop_registry\", len(dups)==0,
f\"dups={dups}\")\n\n# 3) HL7 & calculators\n_ok(\"parse_hl7_labs
present\", \"parse_hl7_labs\" in globals())\n_ok(\"phase2_bundle
present\", \"phase2_bundle\" in globals())\n\n# 4) UI widgets wired
(if running UI)\n_ui_ok = True\nfor w in [\"vitals_in\", \"labs_in\",
\"ctx_in\"]:\n    if w not in globals():\n        _ui_ok = False\nif
_ui_ok:\n    try:\n        vit = json.loads(vitals_in.value or \"{

```

```

\")\n        lab = json.loads(labs_in.value or "{}")\n            ctx
= json.loads(ctx_in.value or "{}")\n        bund =
phase2_bundle(vit, lab, ctx)\n            _ok("bundle one-liners non-
empty", isinstance(bund.get("one_liners"), list) and
len(bund["one_liners"])>3)\n        except Exception as e:\n
errors.append(f"[x] UI inputs parse/bundle - {e}")\n    else:\n
print("[ - ] UI textareas not found (ok if RUN_UI is off)")\n\n# 5)
ICU panel helpers\nfor fn in ["_load_icu_status",
"_compute_next_bed_eta"]:\n    _ok(f"{fn} present", fn in
globals())\n\n# 6) Event schema sanity\nimport pathlib\np =
pathlib.Path(CONFIG["EVENT_LOG_PATH"])\n_ok("Event log path
exists or creatable", p.parent.exists() or True)\n# write a dry run
record\ntry:\n    _append_event({"type": "selftest", "ok": True})
\n    _ok("Event append works", True)\nexcept Exception as e:\n
errors.append(f"[x] Event append - {e}")\n\n# 7) Optional ML
surface\nif "predict_one" in globals() and "FEAT" in globals():
\n    _ok("predict_one wired", True)\nelse:\n    print("[ - ] ML
wrapper not loaded (ok for now)")\n\nprint("\n\nRESULT:", "PASS"
if not errors else "FAIL")\nif errors:\n    print("\n
\n".join(errors))\n", "metadata": {"trusted": true, "execution":
{"iopub.status.busy": "2025-08-25T18:40:39.997440Z", "iopub.execute_in
put": "2025-08-25T18:40:39.997834Z", "iopub.status.idle": "2025-08-25T1
8:40:40.022546Z", "shell.execute_reply.started": "2025-08-25T18:40:39.
997803Z", "shell.execute_reply": "2025-08-25T18:40:40.021732Z"}}, "outp
uts": [{"name": "stdout", "text": "[✓] CONFIG frozen\n[✓] _append_event
present\n[✓] _append_event signature\n[✓] no duplicate run_ui/
refresh_sop_registry\n[✓] parse_hl7_labs present\n[✓] phase2_bundle
present\n[✓] bundle one-liners non-empty\n[✓] _load_icu_status
present\n[✓] _compute_next_bed_eta present\n[✓] Event log path
exists or creatable\n[✓] Event append works\n[✓] predict_one
wired\n\nRESULT:
PASS\n", "output_type": "stream"}}, {"execution_count": 104},
{"cell_type": "code", "source": "# == NO-SURPRISES GUARDS ==\n\n# Use
config guards everywhere (prevents NameError & unintended runs)\ndef
_on(flag): return bool(CONFIG.get(flag, False))\ndef
run_icu_constraints(state):\n    if not _on("RUN_PIPELINE"):\n
return {}\n    mod = _try_import("icu_constraints") or
_try_import("modeling_icu_constraints")\n    if mod and
hasattr(mod, "compute_icu_flags"):\n        try: return
dict(mod.compute_icu_flags(state))\n        except Exception: return
{}\n    return {}\ndef mesh_route_actions(state, actions):\n    if
not _on("RUN_PIPELINE"): return actions\n    mod =
_try_import("agent_mesh") or _try_import("ed_agent_mesh")\n    if mod and hasattr(mod, "route"):\n        try: return
list(mod.route(state, actions))\n        except Exception: return
actions\n    return actions\ndef trainer_fit_critic(critic,
samples, y):\n    if not _on("RUN_PIPELINE"): return critic\n
mod = _try_import("trainer") or _try_import("ed_trainer")\n    if mod and hasattr(mod, "fit_critic"):\n        try: return
mod.fit_critic(critic, samples, y)\n        except Exception: return
critic\n    return critic\n\n# Single event appender (choose one
timestamp style; this is UTC with 'Z')\ndef _append_event(ev: dict):
\n    import json, datetime as _dt, os\n    ev = {"ts":
_dt.datetime.utcnow().isoformat()+"Z", **(ev or {})}\n    with

```

```
open(CONFIG["EVENT_LOG_PATH"], "a", encoding="utf-8") as f:\n
f.write(json.dumps(ev, ensure_ascii=False) + "\\n\\n")\n", "metadata":\n{"trusted":true,"execution":\n{"iopub.status.busy":"2025-08-25T18:40:40.023419Z","iopub.execute_in\nput":"2025-08-25T18:40:40.023697Z","iopub.status.idle":"2025-08-25T1\n8:40:40.043589Z","shell.execute_reply.started":"2025-08-25T18:40:40.\n023677Z","shell.execute_reply":"2025-08-25T18:40:40.042598Z"}}, "outp\nuts":[],"execution_count":105}}}
```